Computation Graphs, Backpropagation and Autodiff

How gradients flow through neural networks

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Backprop vocabulary

Lecture 2 built the forward map; today we reverse it

V VISUAL

Last time, an MLP turned an input into a prediction and then one scalar loss. Today we reuse that **same graph** in reverse:

Lecture 2 built the forward map; today we reverse it

V VISUAL

Last time, an MLP turned an input into a prediction and then one scalar loss. Today we reuse that **same graph** in reverse:

General pattern

Forward: compute values

$$\boldsymbol{x} \to f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \hat{\boldsymbol{y}} \to \mathcal{L}$$

Backward: compute sensitivities loss sensitivity for every parameter θ_j

Concrete scalar example

Forward: with $(w, x, b, y) = (2, 3, 1, 10)$

$$\hat{y} = 2 \cdot 3 + 1 = 7, \quad \mathcal{L} = (7 - 10)^2 = 9$$

Backward:

$$\left(\frac{\partial \mathcal{L}}{\partial w}, \frac{\partial \mathcal{L}}{\partial b}\right) = (-18, -6)$$

Lecture 2 built the forward map; today we reverse it

V VISUAL

Last time, an MLP turned an input into a prediction and then one scalar loss. Today we reuse that **same graph** in reverse:

General pattern

Forward: compute values

$$x \to f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \hat{\boldsymbol{y}} \to \mathcal{L}$$

Backward: compute sensitivities loss sensitivity for every parameter θ_j

Concrete scalar example

Forward: with $(w, x, b, y) = (2, 3, 1, 10)$

$$\hat{y} = 2 \cdot 3 + 1 = 7, \quad \mathcal{L} = (7 - 10)^2 = 9$$

Backward:

$$\left(\frac{\partial \mathcal{L}}{\partial w}, \frac{\partial \mathcal{L}}{\partial b}\right) = (-18, -6)$$

same executed graph · values flow forward, gradients flow backward

Read the notation before the algorithm

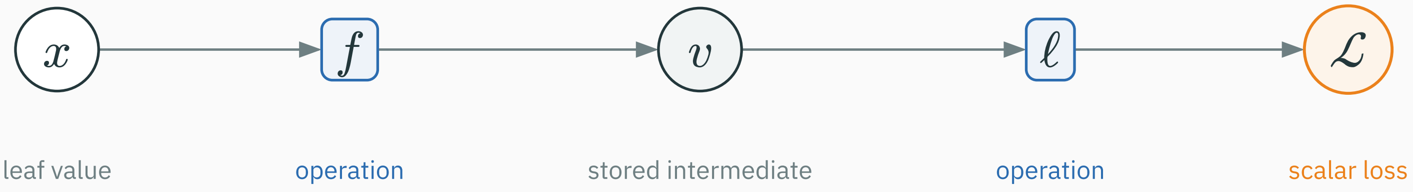
Symbol	Meaning in this lecture
u, v, z	scalars
$\boldsymbol{x}, \boldsymbol{z}, \boldsymbol{\theta}$	vectors (bold lowercase)
$\boldsymbol{W}, \boldsymbol{X}$	matrices (bold uppercase)
$\partial \mathcal{L} / \partial u$	how the loss changes when scalar u changes
$\partial \mathcal{L} / \partial \boldsymbol{\theta}$	all parameter derivatives, arranged like $\boldsymbol{\theta}$

Read the notation before the algorithm

Symbol	Meaning in this lecture
u, v, z	scalars
$\boldsymbol{x}, \boldsymbol{z}, \boldsymbol{\theta}$	vectors (bold lowercase)
$\boldsymbol{W}, \boldsymbol{X}$	matrices (bold uppercase)
$\partial \mathcal{L} / \partial u$	how the loss changes when scalar u changes
$\partial \mathcal{L} / \partial \boldsymbol{\theta}$	all parameter derivatives, arranged like $\boldsymbol{\theta}$

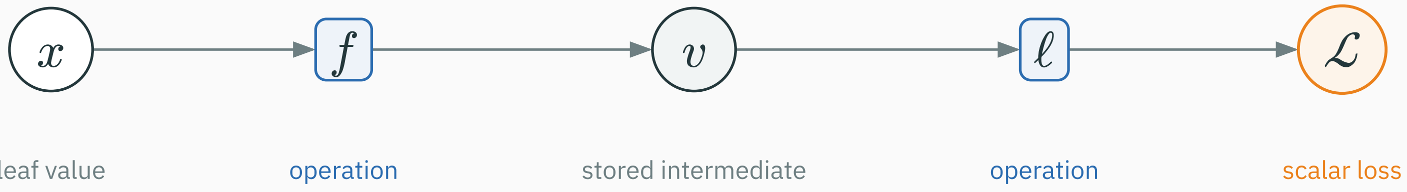
> We keep the full $\partial \mathcal{L} / \partial(\cdot)$ notation visible throughout: the numerator is always the final loss; the denominator names the stored value whose derivative we want.

Our computation-graph convention



Our computation-graph convention

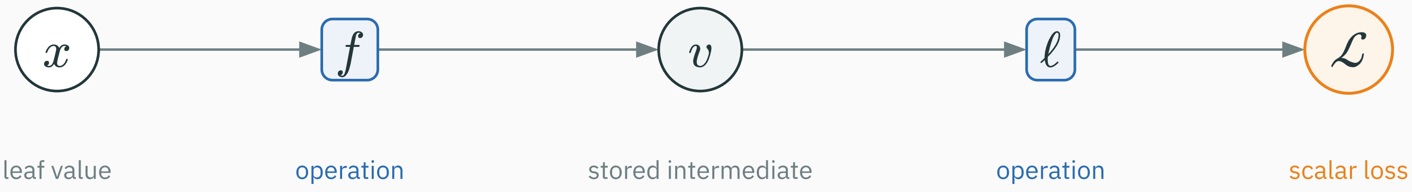
V VISUAL



white circle	given value: input, parameter, or target — a graph leaf
gray circle	computed intermediate value, saved for backward when needed
blue box	operation: consumes value(s) and produces a value
gray arrow	forward dependency from a value to the operation that uses it
orange circle	final scalar loss $\mathcal{L}$; seeds backward with $\partial\mathcal{L}/\partial\mathcal{L} = 1$

Our computation-graph convention

V VISUAL



white circle	given value: input, parameter, or target — a graph leaf
gray circle	computed intermediate value, saved for backward when needed
blue box	operation: consumes value(s) and produces a value
gray arrow	forward dependency from a value to the operation that uses it
orange circle	final scalar loss $\mathcal{L}$; seeds backward with $\partial\mathcal{L}/\partial\mathcal{L} = 1$

circles store values · boxes transform them · arrows show dependency

One derivative, three routes

D DERIVATION

Symbolic = formula $\to$ formula. A person or a computer-algebra system may do the algebra.

$$\mathcal{L} = (wx + b - y)^2, \quad (w, x, b, y) = (2, 3, 1, 10).$$

One derivative, three routes

Symbolic = formula $\to$ formula. A person or a computer-algebra system may do the algebra.

$$\mathcal{L} = (wx + b - y)^2, \quad (w, x, b, y) = (2, 3, 1, 10).$$

Route	What it does	What it returns here
symbolic	derive a formula	$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x$; substitute $\to -18$

One derivative, three routes

D DERIVATION

Symbolic = formula → formula. A person or a computer-algebra system may do the algebra.

$$\mathcal{L} = (wx + b - y)^2, \quad (w, x, b, y) = (2, 3, 1, 10).$$

Route	What it does	What it returns here
symbolic	derive a formula	$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x$; substitute $\to -18$
finite difference	evaluate $\mathcal{L}(w \pm \varepsilon)$	approximate number at $w = 2$: -18

One derivative, three routes

Symbolic = formula $\to$ formula. A person or a computer-algebra system may do the algebra.

$$\mathcal{L} = (wx + b - y)^2, \quad (w, x, b, y) = (2, 3, 1, 10).$$

Route	What it does	What it returns here
symbolic	derive a formula	$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x$; substitute $\to -18$
finite difference	evaluate $\mathcal{L}(w \pm \varepsilon)$	approximate number at $w = 2$: -18
reverse-mode AD	run local rules backward	number for this run: $(-6) \cdot 3 = -18$

One derivative, three routes

Symbolic = formula → formula. A person or a computer-algebra system may do the algebra.

$$\mathcal{L} = (wx + b - y)^2, \quad (w, x, b, y) = (2, 3, 1, 10).$$

Route	What it does	What it returns here
symbolic	derive a formula	$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x$; substitute $\to -18$
finite difference	evaluate $\mathcal{L}(w \pm \varepsilon)$	approximate number at $w = 2$: -18
reverse-mode AD	run local rules backward	number for this run: $(-6) \cdot 3 = -18$

formula · approximate value · reverse sweep through the executed graph

Gradient checking: compare one coordinate two ways

INTERACTIVE

Reverse-mode AD computes the derivative from local rules, up to floating-point arithmetic. Central difference independently approximates the same value: $g_{\mathrm{FD}} = \frac{\mathcal{L}(\theta_j+\varepsilon)-\mathcal{L}(\theta_j-\varepsilon)}{2\varepsilon}$.

Gradient checking: compare one coordinate two ways

I INTERACTIVE

Reverse-mode AD computes the derivative from local rules, up to floating-point arithmetic. Central difference independently approximates the same value: $g_{\mathrm{FD}} = \frac{\mathcal{L}(\theta_j+\varepsilon)-\mathcal{L}(\theta_j-\varepsilon)}{2\varepsilon}$.

```
import torch
def loss(w): return (w*3.0 + 1.0 - 10.0)**2
w, eps = 2.0, 1e-5
g_fd = (loss(w + eps) - loss(w - eps)) / (2*eps)
wt = torch.tensor(w, dtype=torch.float64, requires_grad=True)
loss(wt).backward()
g_ad = wt.grad.item()
print(g_fd, g_ad)  # both approximately -18
```

Gradient checking: compare one coordinate two ways

I INTERACTIVE

Reverse-mode AD computes the derivative from local rules, up to floating-point arithmetic. Central difference independently approximates the same value: $g_{\mathrm{FD}} = \frac{\mathcal{L}(\theta_j+\varepsilon)-\mathcal{L}(\theta_j-\varepsilon)}{2\varepsilon}$.

```
import torch
def loss(w): return (w*3.0 + 1.0 - 10.0)**2
w, eps = 2.0, 1e-5
g_fd = (loss(w + eps) - loss(w - eps)) / (2*eps)
wt = torch.tensor(w, dtype=torch.float64, requires_grad=True)
loss(wt).backward()
g_ad = wt.grad.item()
print(g_fd, g_ad)  # both approximately -18
```

> Use float64 and check only a few coordinates. Finite differences debug gradients; they do not train models because every checked coordinate needs two extra forward passes.

Why train with reverse-mode autodiff?

D DERIVATION

Method	Exact?	Cost for P parameters	Use it for
symbolic	exact formula before numerical evaluation	expression-dependent; algebra can grow	deriving / reasoning

Why train with reverse-mode autodiff?

D DERIVATION

Method	Exact?	Cost for P parameters	Use it for
symbolic	exact formula before numerical evaluation	expression-dependent; algebra can grow	deriving / reasoning
finite difference	approximate	$2P$ loss evaluations	checking

Why train with reverse-mode autodiff?

D DERIVATION

Method	Exact?	Cost for P parameters	Use it for
symbolic	exact formula before numerical evaluation	expression-dependent; algebra can grow	deriving / reasoning
finite difference	approximate	$2P$ loss evaluations	checking
reverse mode	exact local rules; floating-point values	one reverse sweep for scalar $\mathcal{L}$	training

Why train with reverse-mode autodiff?

D DERIVATION

Method	Exact?	Cost for P parameters	Use it for
symbolic	exact formula before numerical evaluation	expression-dependent; algebra can grow	deriving / reasoning
finite difference	approximate	$2P$ loss evaluations	checking
reverse mode	exact local rules; floating-point values	one reverse sweep for scalar $\mathcal{L}$	training

Symbolic to reason · finite differences to check · reverse mode to train.

Backprop returns every parameter sensitivity in one sweep

For each parameter coordinate, gradient descent uses

$$\theta_{j,t+1} = \theta_{j,t} - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}.$$

Backprop returns every parameter sensitivity in one sweep

For each parameter coordinate, gradient descent uses

$$\theta_{j,t+1} = \theta_{j,t} - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}.$$

η is the learning rate; the derivative supplies the direction and magnitude of the local change.

Backprop returns every parameter sensitivity in one sweep

For each parameter coordinate, gradient descent uses

$$\theta_{j,t+1} = \theta_{j,t} - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}.$$

η is the learning rate; the derivative supplies the direction and magnitude of the local change. A modern network has $P = 10^7$ to 10^{11} parameters.

Backprop returns every parameter sensitivity in one sweep

For each parameter coordinate, gradient descent uses

$$\theta_{j,t+1} = \theta_{j,t} - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}.$$

η is the learning rate; the derivative supplies the direction and magnitude of the local change. A modern network has $P = 10^7$ to 10^{11} parameters.

We need all P derivatives $\partial \mathcal{L}/\partial \theta_j$. Backprop computes them together in **one reverse sweep**, without perturbing parameters one at a time.

The route from one local operation to an entire network

V VISUAL

Stage	Question we answer
1 · local rule	How does one operation pass a gradient backward?

The route from one local operation to an entire network

V VISUAL

Stage	Question we answer
1 · local rule	How does one operation pass a gradient backward?
2 · branches	Why do gradient contributions add?

The route from one local operation to an entire network

V VISUAL

Stage	Question we answer
1 · local rule	How does one operation pass a gradient backward?
2 · branches	Why do gradient contributions add?
3 · worked graph	Can we reproduce every derivative numerically?

The route from one local operation to an entire network

V VISUAL

Stage	Question we answer
1 · local rule	How does one operation pass a gradient backward?
2 · branches	Why do gradient contributions add?
3 · worked graph	Can we reproduce every derivative numerically?
4 · layers + MLP	How do scalar rules become vector and matrix formulas?

The route from one local operation to an entire network

V VISUAL

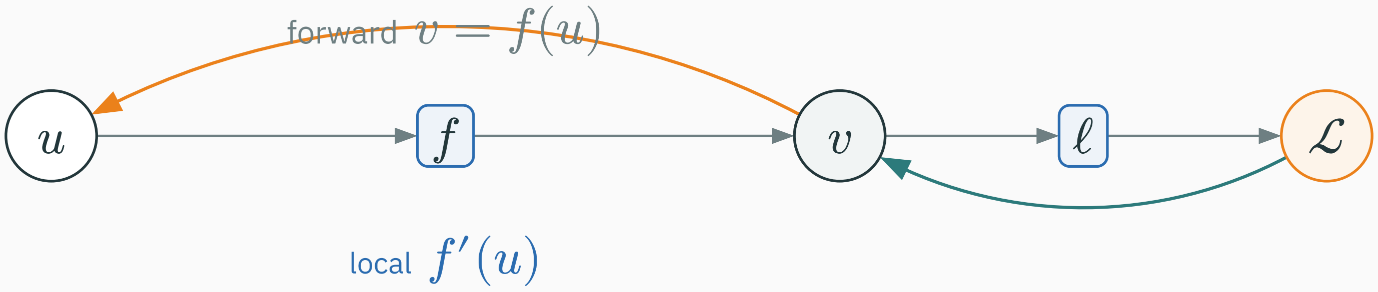
Stage	Question we answer
1 · local rule	How does one operation pass a gradient backward?
2 · branches	Why do gradient contributions add?
3 · worked graph	Can we reproduce every derivative numerically?
4 · layers + MLP	How do scalar rules become vector and matrix formulas?
5 · depth + gradient flow	What happens when many local gradients multiply?

One local operation

Zoom in on one operation. The value circle u enters the boxed operation f; its output is stored in the value circle v.

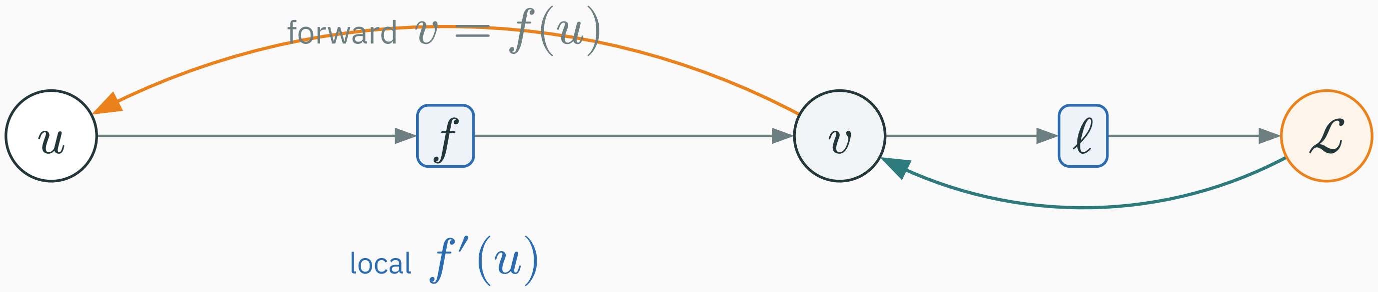
One local operation

Zoom in on one operation. The value circle u enters the boxed operation f; its output is stored in the value circle v.



One local operation

Zoom in on one operation. The value circle u enters the boxed operation f; its output is stored in the value circle v.

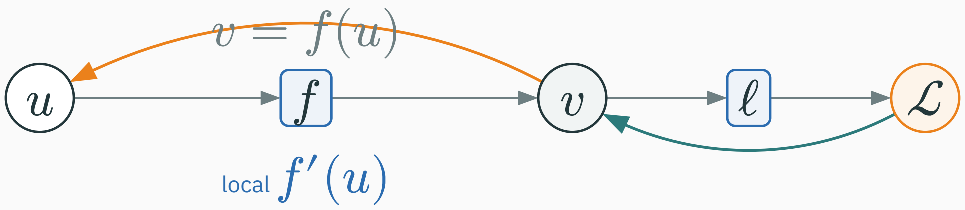


backward computes $\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot f'(u)$

Every operation multiplies the arriving gradient by its local gradient

D DERIVATION

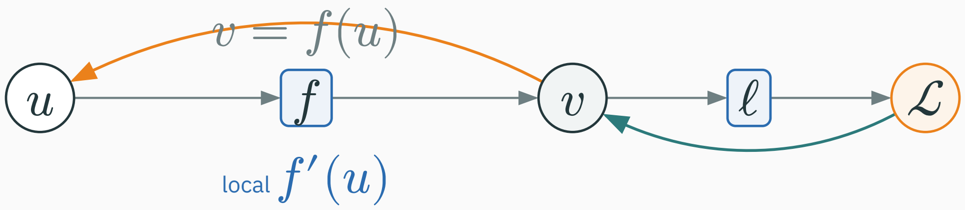
For the operation $v = f(u)$, the chain rule is



Every operation multiplies the arriving gradient by its local gradient

D DERIVATION

For the operation $v = f(u)$, the chain rule is

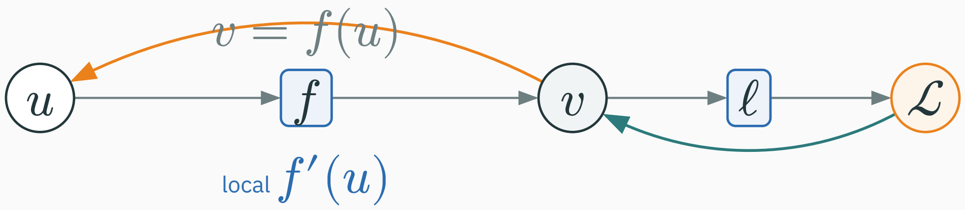


$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot f'(u)$$

Every operation multiplies the arriving gradient by its local gradient

D DERIVATION

For the operation $v = f(u)$, the chain rule is



$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot f'(u)$$

Downstream gradient

sent back to u

$\frac{\partial \mathcal{L}}{\partial u}$

Upstream gradient

arriving at v

$\frac{\partial \mathcal{L}}{\partial v}$

Local gradient

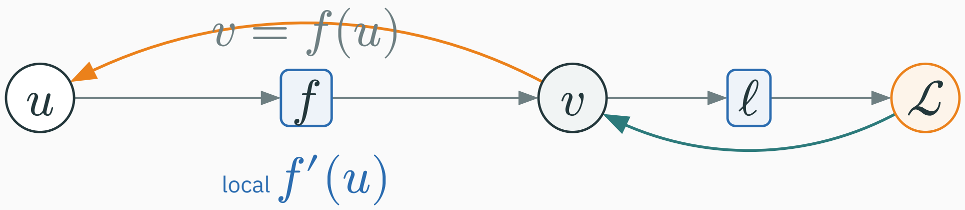
this operation's slope

$f'(u) = \frac{\partial v}{\partial u}$

The chain rule is one reusable local rule

D DERIVATION

For every scalar operation $v = f(u)$:

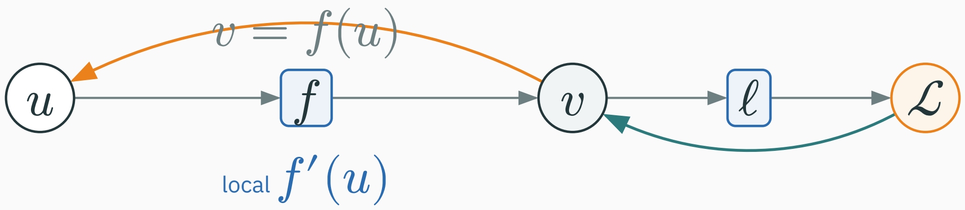


$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot \frac{\partial v}{\partial u}.$$

The chain rule is one reusable local rule

D DERIVATION

For every scalar operation $v = f(u)$:



$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot \frac{\partial v}{\partial u}.$$

> The operation multiplies its **upstream gradient** by its **local gradient** to produce the **downstream gradient**. It needs no global formula for the whole network.

Example: a square operation

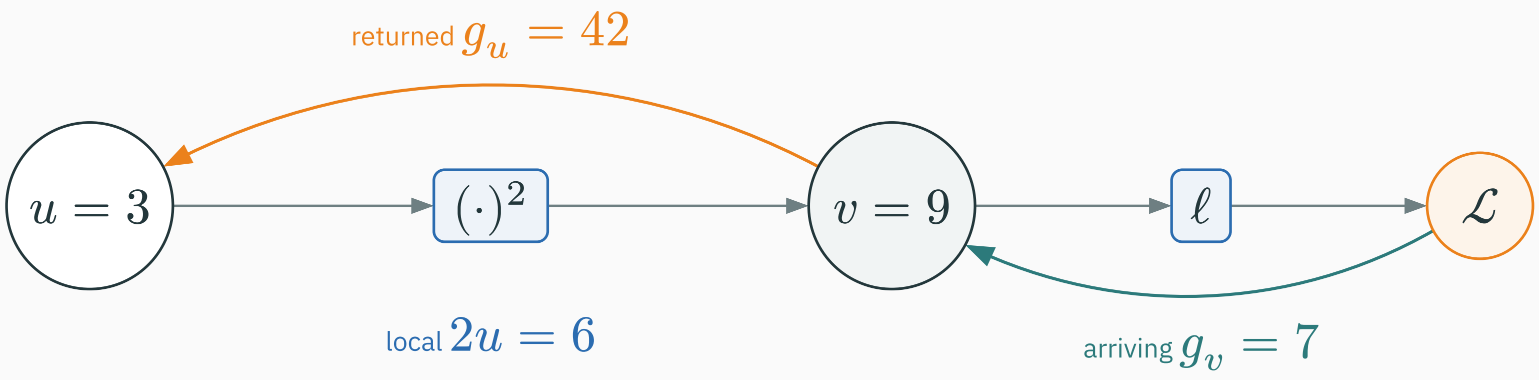
D DERIVATION

For $v = u^2$ at $u = 3$, the forward value is $v = 9$ and the local slope is $\partial v / \partial u = 2u = 6$. Suppose the rest of the graph sends $g_v = \partial \mathcal{L} / \partial v = 7$ back to this square.

Example: a square operation

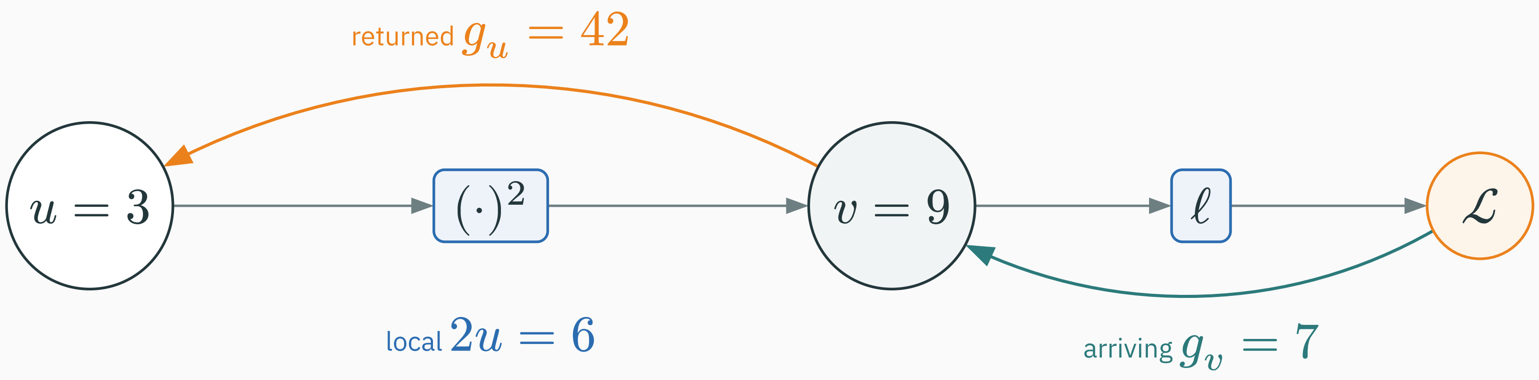
D DERIVATION

For $v = u^2$ at $u = 3$, the forward value is $v = 9$ and the local slope is $\partial v/\partial u = 2u = 6$. Suppose the rest of the graph sends $g_v = \partial\mathcal{L}/\partial v = 7$ back to this square.



Example: a square operation

For $v = u^2$ at $u = 3$, the forward value is $v = 9$ and the local slope is $\partial v/\partial u = 2u = 6$. Suppose the rest of the graph sends $g_v = \partial\mathcal{L}/\partial v = 7$ back to this square.

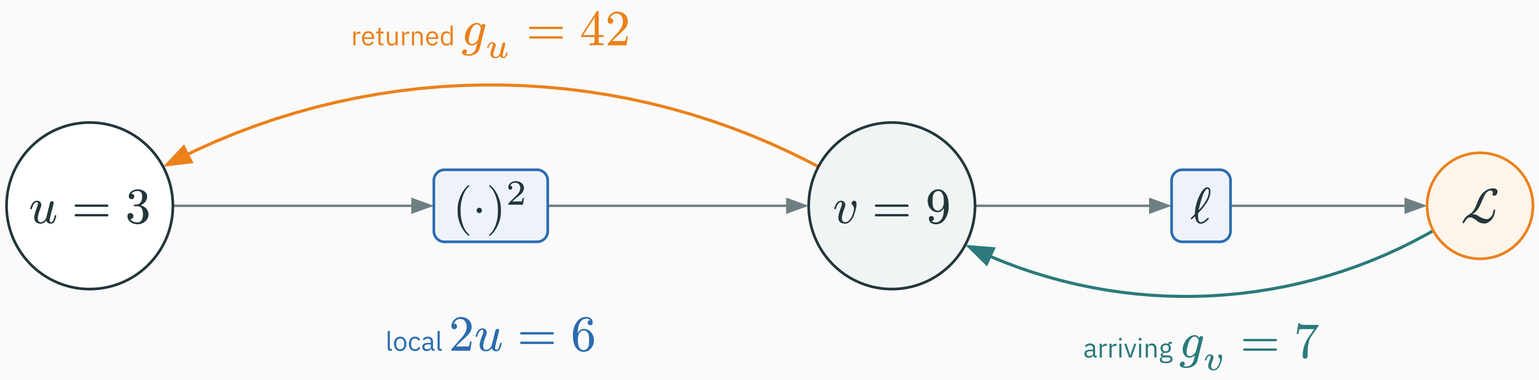


$$\frac{\partial\mathcal{L}}{\partial u} = \frac{\partial\mathcal{L}}{\partial v} \cdot 2u = 7 \cdot 6 = 42.$$

Example: a square operation

D DERIVATION

For $v = u^2$ at $u = 3$, the forward value is $v = 9$ and the local slope is $\partial v/\partial u = 2u = 6$. Suppose the rest of the graph sends $g_v = \partial\mathcal{L}/\partial v = 7$ back to this square.



$$\frac{\partial\mathcal{L}}{\partial u} = \frac{\partial\mathcal{L}}{\partial v} \cdot 2u = 7 \cdot 6 = 42.$$

> The square operation multiplies the arriving $\partial\mathcal{L}/\partial v = 7$ by its local gradient $2u = 6$ to produce $\partial\mathcal{L}/\partial u = 42$.

Addition copies the arriving gradient

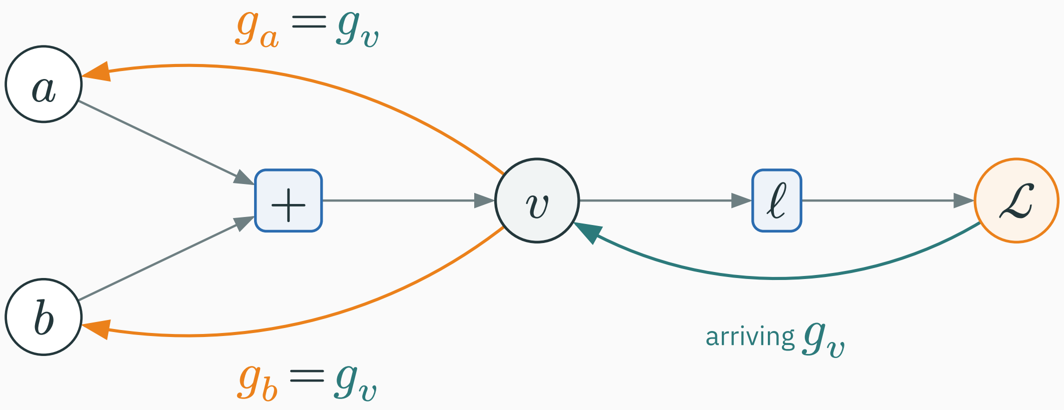
D DERIVATION

Operation $v = a + b$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = 1$, $\partial v/\partial b = 1$.

Addition copies the arriving gradient

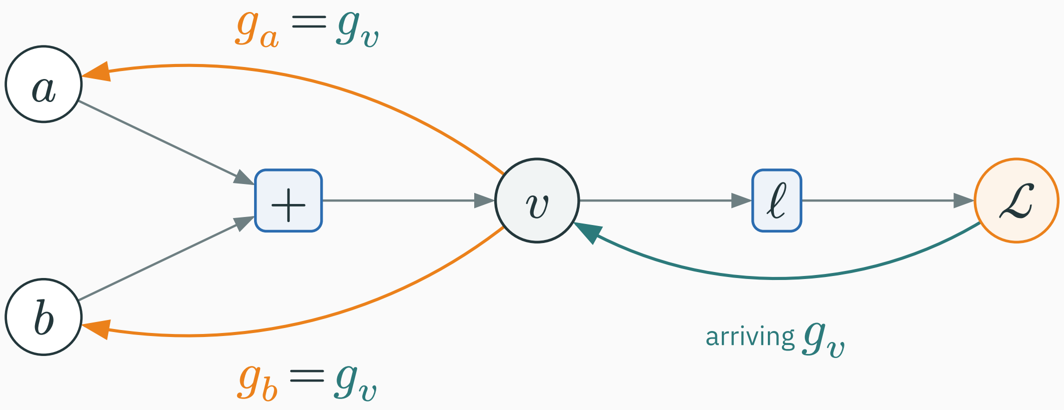
D DERIVATION

Operation $v = a + b$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = 1$, $\partial v/\partial b = 1$.



Addition copies the arriving gradient

Operation $v = a + b$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = 1$, $\partial v/\partial b = 1$.

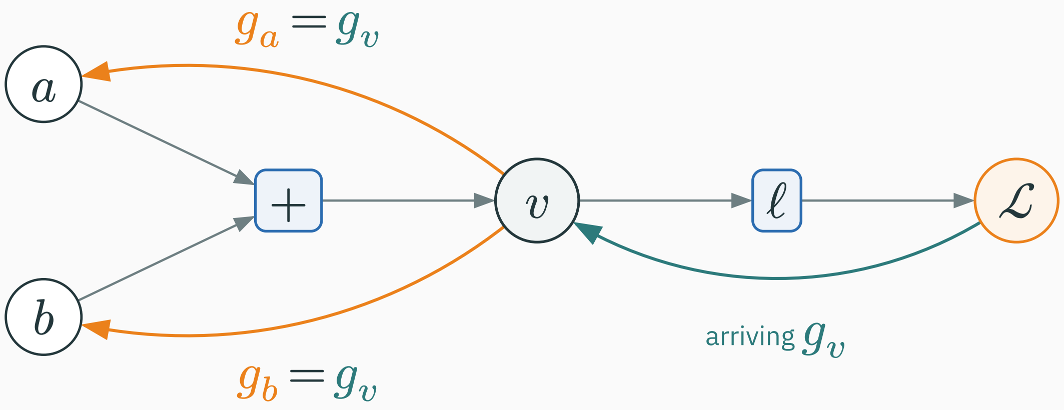


$$\frac{\partial\mathcal{L}}{\partial a} = \frac{\partial\mathcal{L}}{\partial v}\cdot 1, \qquad \frac{\partial\mathcal{L}}{\partial b} = \frac{\partial\mathcal{L}}{\partial v}\cdot 1.$$

Addition copies the arriving gradient

D DERIVATION

Operation $v = a + b$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = 1$, $\partial v/\partial b = 1$.



$$\frac{\partial\mathcal{L}}{\partial a} = \frac{\partial\mathcal{L}}{\partial v} \cdot 1, \qquad \frac{\partial\mathcal{L}}{\partial b} = \frac{\partial\mathcal{L}}{\partial v} \cdot 1.$$

$+$ copies the arriving derivative into a downstream derivative for every input.

Multiplication scales by the other input

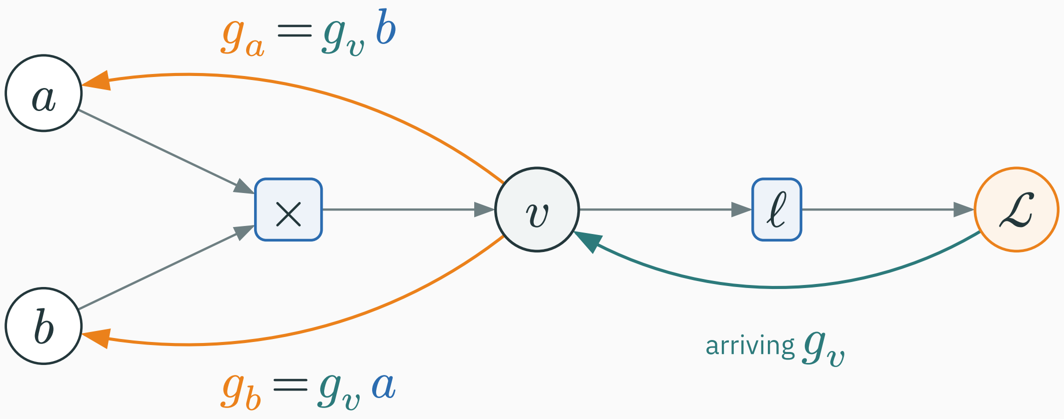
D DERIVATION

Operation $v = ab$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = b$, $\partial v/\partial b = a$.

Multiplication scales by the other input

D DERIVATION

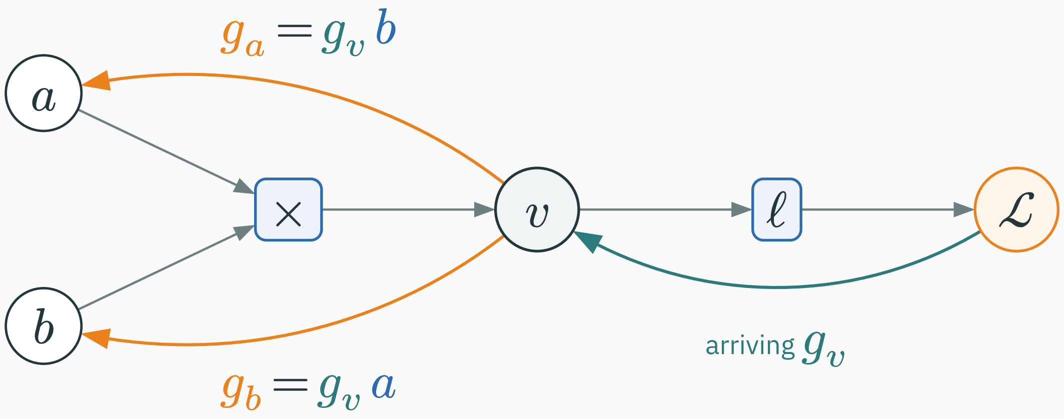
Operation $v = ab$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = b$, $\partial v/\partial b = a$.



Multiplication scales by the other input

D DERIVATION

Operation $v = ab$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = b$, $\partial v/\partial b = a$.

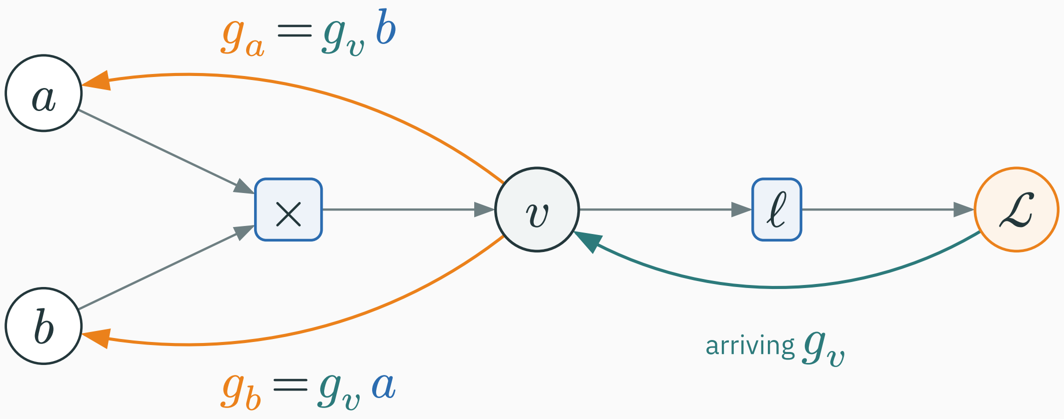


$$\frac{\partial\mathcal{L}}{\partial a} = \frac{\partial\mathcal{L}}{\partial v}\cdot b, \qquad \frac{\partial\mathcal{L}}{\partial b} = \frac{\partial\mathcal{L}}{\partial v}\cdot a.$$

Multiplication scales by the other input

D DERIVATION

Operation $v = ab$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = b$, $\partial v/\partial b = a$.

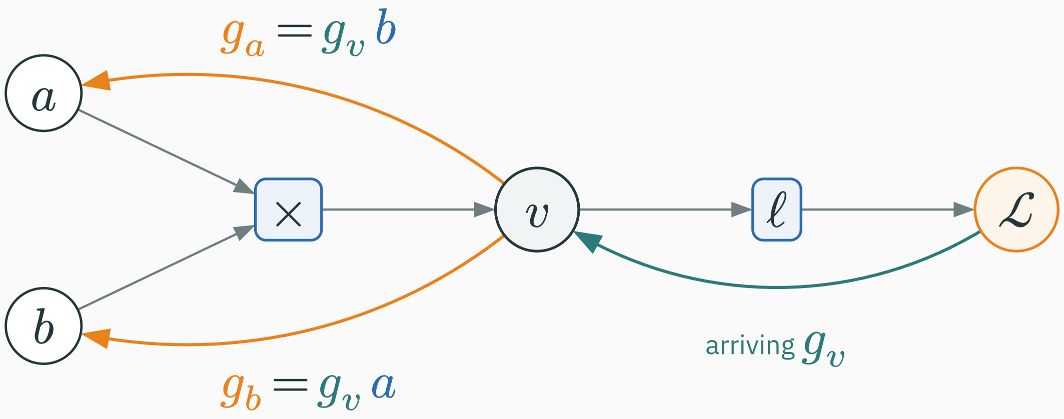


$$\frac{\partial\mathcal{L}}{\partial a} = \frac{\partial\mathcal{L}}{\partial v}\cdot b, \qquad \frac{\partial\mathcal{L}}{\partial b} = \frac{\partial\mathcal{L}}{\partial v}\cdot a.$$

$\times$ uses the other input as its local multiplier.

Multiplication scales by the other input

Operation $v = ab$. Let $g_v = \partial\mathcal{L}/\partial v$ arrive. Local derivatives: $\partial v/\partial a = b$, $\partial v/\partial b = a$.



$$\frac{\partial\mathcal{L}}{\partial a} = \frac{\partial\mathcal{L}}{\partial v}\cdot b, \qquad \frac{\partial\mathcal{L}}{\partial b} = \frac{\partial\mathcal{L}}{\partial v}\cdot a.$$

$\times$ uses the other input as its local multiplier.

> If $a = 2$, $b = 5$, and $\partial\mathcal{L}/\partial v = 3$, then $\partial\mathcal{L}/\partial a = 15$ and $\partial\mathcal{L}/\partial b = 6$.

Activation operation

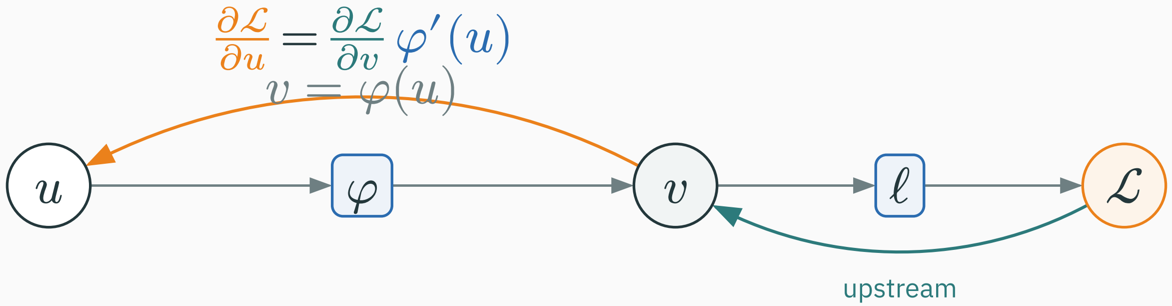
D DERIVATION

Operation $v = \varphi(u)$ (elementwise nonlinearity). Local derivative: $\partial v / \partial u = \varphi'(u)$.

Activation operation

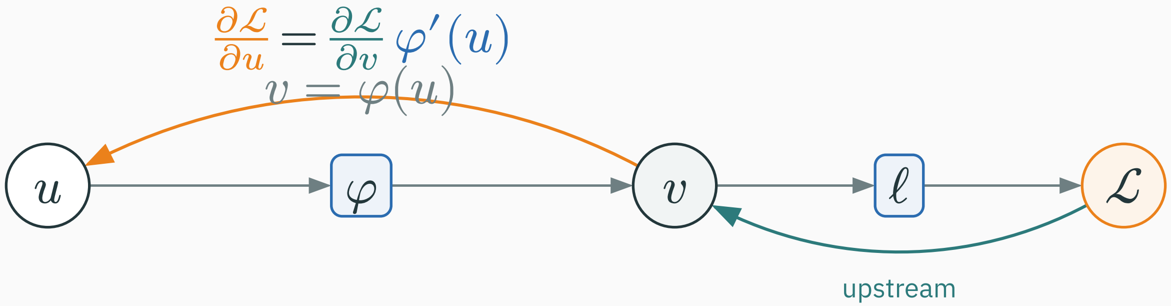
DERIVATION

Operation $v = \varphi(u)$ (elementwise nonlinearity). Local derivative: $\partial v / \partial u = \varphi'(u)$.



Activation operation

Operation $v = \varphi(u)$ (elementwise nonlinearity). Local derivative: $\partial v / \partial u = \varphi'(u)$.

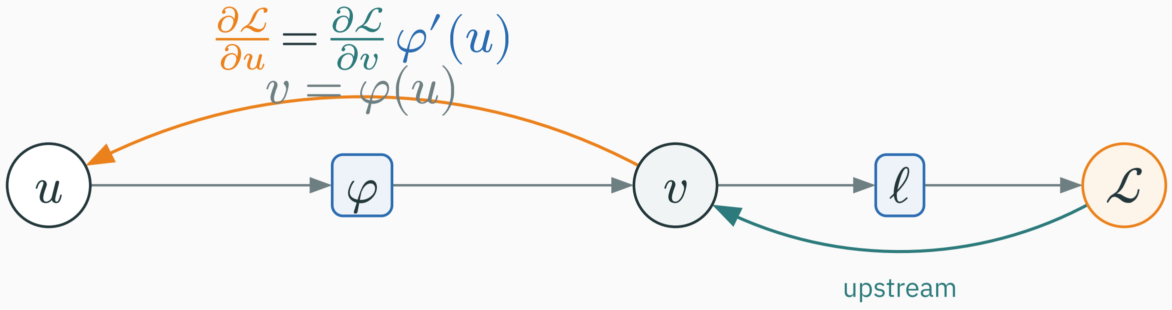


$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot \varphi'(u).$$

Activation operation

D DERIVATION

Operation $v = \varphi(u)$ (elementwise nonlinearity). Local derivative: $\partial v/\partial u = \varphi'(u)$.



$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot \varphi'(u).$$

sigmoid σ: $\varphi'(u) = \sigma(u)(1 - \sigma(u))$

ReLU: $\varphi'(u) = 1$ for $u > 0$, and 0 for $u < 0$

Pop quiz: what crosses an inactive ReLU?

Q QUESTION

Let $v = \mathrm{ReLU}(u)$. If $u = -2$ and the upstream gradient is $\partial\mathcal{L}/\partial v = 5$, what downstream gradient $\partial\mathcal{L}/\partial u$ is sent back?

A. 5

B. -10

C. 0

D. undefined because $u < 0$

Answer: an inactive ReLU blocks the gradient

A ANSWER

Correct: C — 0

Why: For $u < 0$, the local gradient is $\partial v / \partial u = 0$. Therefore downstream $=$ upstream $\times$ local $= 5 \times 0 = 0$.

Answer: an inactive ReLU blocks the gradient

A ANSWER

Correct: C — 0

Why: For $u < 0$, the local gradient is $\partial v / \partial u = 0$. Therefore downstream $=$ upstream $\times$ local $= 5 \times 0 = 0$.

Forward: $u = -2 \rightarrow v = 0$

Backward: $5 \times 0 \rightarrow \partial \mathcal{L} / \partial u = 0$

Answer: an inactive ReLU blocks the gradient

A ANSWER

Correct: C — 0

Why: For $u < 0$, the local gradient is $\partial v / \partial u = 0$. Therefore downstream $=$ upstream $\times$ local $= 5 \times 0 = 0$.

Forward: $u = -2 \to v = 0$

Backward: $5 \times 0 \to \partial \mathcal{L} / \partial u = 0$

At exactly $u = 0$, ReLU's ordinary derivative is undefined. Autodiff software must choose a convention; commonly it returns 0.

The rule table — memorize this

VISUAL

Forward operation	Contribution sent backward
$v = a + b$	$\frac{\partial \mathcal{L}}{\partial a}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 1$ $\frac{\partial \mathcal{L}}{\partial b}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 1$
$v = ab$	$\frac{\partial \mathcal{L}}{\partial a}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot b$ $\frac{\partial \mathcal{L}}{\partial b}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot a$
$v = u^2$	$\frac{\partial \mathcal{L}}{\partial u}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 2u$
$v = \varphi(u)$	$\frac{\partial \mathcal{L}}{\partial u}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot \varphi'(u)$

The rule table — memorize this

V VISUAL

Forward operation	Contribution sent backward
$v = a + b$	$\frac{\partial \mathcal{L}}{\partial a}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 1$ $\frac{\partial \mathcal{L}}{\partial b}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 1$
$v = ab$	$\frac{\partial \mathcal{L}}{\partial a}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot b$ $\frac{\partial \mathcal{L}}{\partial b}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot a$
$v = u^2$	$\frac{\partial \mathcal{L}}{\partial u}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot 2u$
$v = \varphi(u)$	$\frac{\partial \mathcal{L}}{\partial u}$ += $\frac{\partial \mathcal{L}}{\partial v} \cdot \varphi'(u)$

Use `+=`, not `=` — a value consumed by several operations accumulates gradient from **all** of them.

Gradients accumulate at branches

Why gradients add at a branch

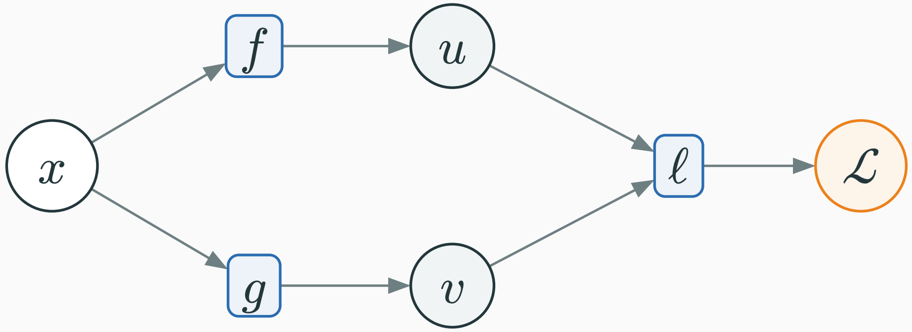
V VISUAL

Imagine nudging x from 4 to 4.01. The **same nudge** travels down both routes to the loss:

Why gradients add at a branch

V VISUAL

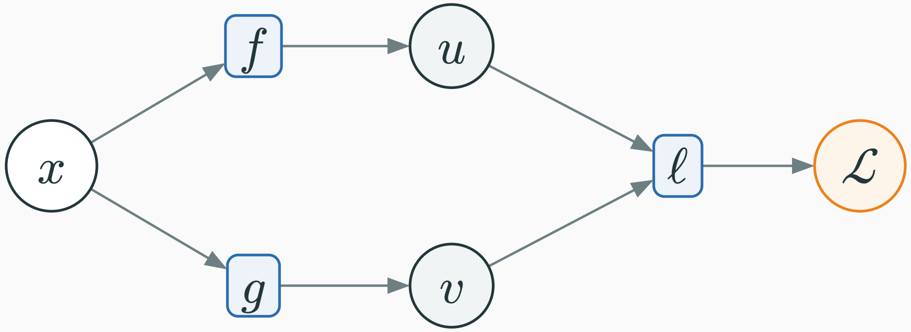
Imagine nudging x from 4 to 4.01. The **same nudge** travels down both routes to the loss:



Why gradients add at a branch

V VISUAL

Imagine nudging x from 4 to 4.01. The **same nudge** travels down both routes to the loss:



Via $u = x^2$: the local slope is $2x = 8$, so $\Delta x = 0.01$ changes the loss by about $8 \cdot 0.01 = 0.08$.

Via $v = 3x$: the local slope is 3, so the same nudge changes the loss by about $3 \cdot 0.01 = 0.03$.

$$\textbf{total slope} = \textbf{total loss change} \div \textbf{nudge} = \frac{0.08+0.03}{0.01} = 8 + 3 = 11$$

The multivariable chain rule adds every path

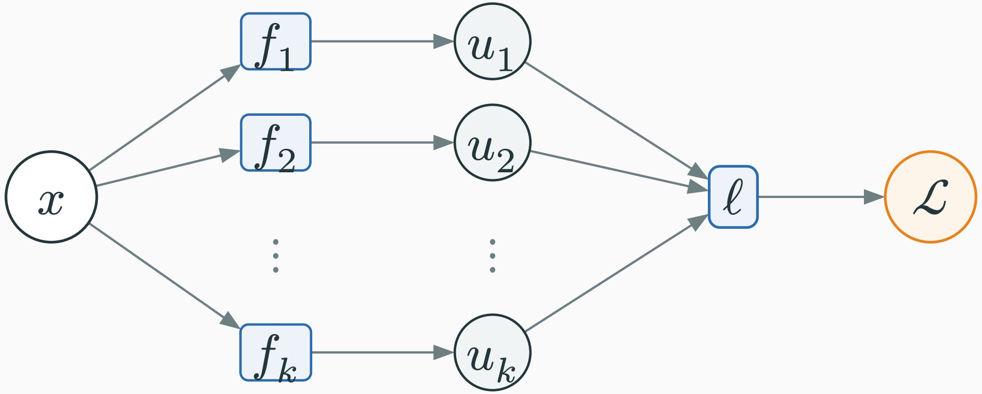
D DERIVATION

The nudge experiment generalizes. If x influences $\mathcal{L}$ through $u_1, \ldots, u_k$:

The multivariable chain rule adds every path

D DERIVATION

The nudge experiment generalizes. If x influences $\mathcal{L}$ through $u_1, \ldots, u_k$:

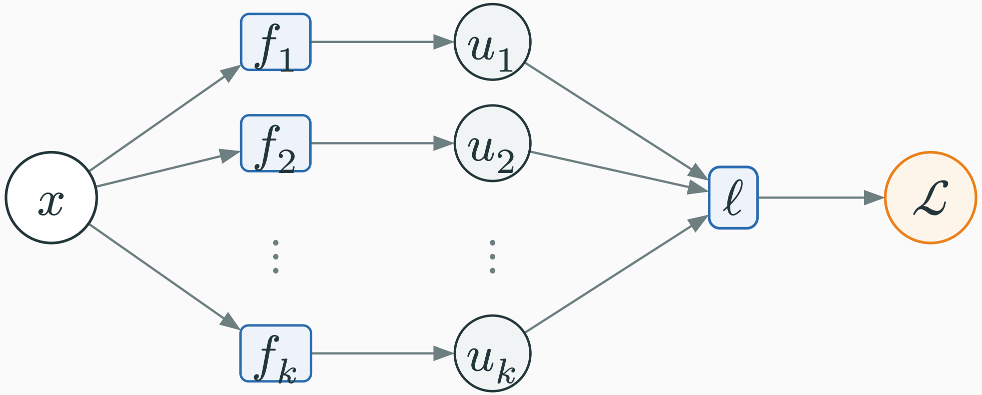


branch i contributes upstream $\partial\mathcal{L}/\partial u_i$ × local $\partial u_i/\partial x$

The multivariable chain rule adds every path

D DERIVATION

The nudge experiment generalizes. If x influences $\mathcal{L}$ through $u_1, \ldots, u_k$:



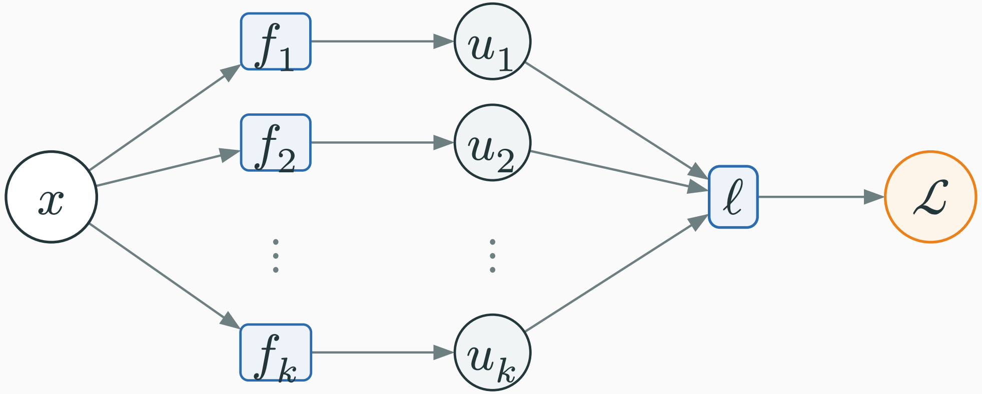
branch i contributes upstream $\partial\mathcal{L}/\partial u_i$ $\times$ local $\partial u_i/\partial x$

$$\frac{\partial\mathcal{L}}{\partial x} = \sum_{i=1}^{k} \frac{\partial\mathcal{L}}{\partial u_i}\,\frac{\partial u_i}{\partial x}.$$

The multivariable chain rule adds every path

D DERIVATION

The nudge experiment generalizes. If x influences $\mathcal{L}$ through $u_1, \ldots, u_k$:



branch i contributes upstream $\partial\mathcal{L}/\partial u_i$ $\times$ local $\partial u_i/\partial x$

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{i=1}^{k} \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial x}.$$

each path contributes one chain-rule product; the products add

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x$.

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x$. By symbolic differentiation — derive the formula first:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + 3, \quad \text{at } x = 4: \quad 2(4) + 3 = 11.$$

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x$. By symbolic differentiation — derive the formula first:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + 3, \quad \text{at } x = 4 : \quad 2(4) + 3 = 11.$$

Now let us get the **same** answer by backprop.

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $\;u = 16,\; v = 12,\; \mathcal{L} = 28.$

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $\;u = 16,\; v = 12,\; \mathcal{L} = 28$. **Backward:** seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$; the add operation gives $\partial\mathcal{L}/\partial u = 1$ and $\partial\mathcal{L}/\partial v = 1$.

Same answer, by backprop

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28$. **Backward:** seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$; the add operation gives $\partial\mathcal{L}/\partial u = 1$ and $\partial\mathcal{L}/\partial v = 1$.

$$\left(\frac{\partial\mathcal{L}}{\partial x}\right)_{\text{via } u} = 1 \cdot 2x = 8, \qquad \left(\frac{\partial\mathcal{L}}{\partial x}\right)_{\text{via } v} = 1 \cdot 3 = 3.$$

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28$. **Backward:** seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$; the add operation gives $\partial\mathcal{L}/\partial u = 1$ and $\partial\mathcal{L}/\partial v = 1$.

$$\left(\frac{\partial\mathcal{L}}{\partial x}\right)_{\text{via } u} = 1 \cdot 2x = 8, \qquad \left(\frac{\partial\mathcal{L}}{\partial x}\right)_{\text{via } v} = 1 \cdot 3 = 3.$$

$$\frac{\partial\mathcal{L}}{\partial x} = 8 + 3 = 11. \quad \checkmark$$

PyTorch adds the same two paths

INTERACTIVE

```python
import torch

x = torch.tensor(4.0, requires_grad=True)
u = x**2                        # path 1
v = 3*x                         # path 2
L = u + v
L.backward()

print(x.grad)                   # tensor(11.)
```

PyTorch adds the same two paths

INTERACTIVE

```
import torch

x = torch.tensor(4.0, requires_grad=True)
u = x**2                          # path 1
v = 3*x                           # path 2
L = u + v
L.backward()

print(x.grad)                     # tensor(11.)
```

via u: $\partial \frac{u}{\partial} x = 2x = 8$

via v: $\partial \frac{v}{\partial} x = 3$

PyTorch adds the same two paths

INTERACTIVE

```python
import torch

x = torch.tensor(4.0, requires_grad=True)
u = x**2                        # path 1
v = 3*x                         # path 2
L = u + v
L.backward()

print(x.grad)                   # tensor(11.)
```

via u: $\partial\frac{u}{\partial}x = 2x = 8$

via v: $\partial\frac{v}{\partial}x = 3$

autograd accumulates both downstream contributions: $8 + 3 = 11$

The central worked example

A scalar linear model

D DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$.

A scalar linear model

D DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$. Concrete numbers:

$$x = 3, \quad w = 2, \quad b = 1, \quad y = 10.$$

A scalar linear model

DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$. Concrete numbers:

$$x = 3, \quad w = 2, \quad b = 1, \quad y = 10.$$

> Compute $\partial\mathcal{L}/\partial w$, $\partial\mathcal{L}/\partial x$, $\partial\mathcal{L}/\partial b$, and $\partial\mathcal{L}/\partial y$ by backpropagation; verify the four values with PyTorch.

Break into primitives

Decompose the loss into elementary operations and name each stored result:

$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

Break into primitives

Decompose the loss into elementary operations and name each stored result:

$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

value m **after** $\times$ · **value** a **after** $+$ · **value** e **after** $-$ · **value** $\mathcal{L}$ **after** $(\cdot)^2$

Break into primitives

Decompose the loss into elementary operations and name each stored result:

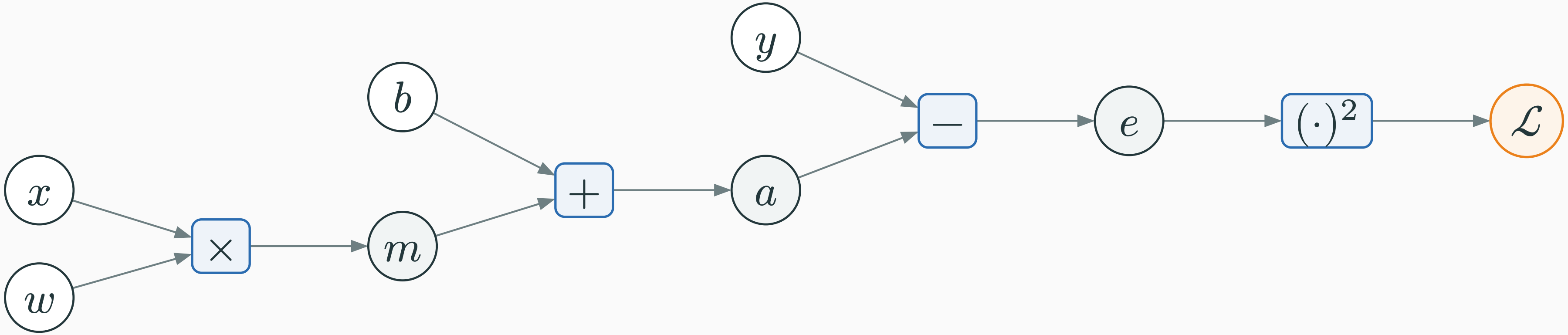
$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$

value m **after** $\times$ **· value** a **after** $+$ **· value** e **after** $-$ **· value** $\mathcal{L}$ **after** $(\cdot)^2$

The symbols m, a, and e name stored intermediate values. They are circles in the graph; the four operations are boxes.

The computation graph

V VISUAL



Given values w, x, b, y	**Stored values** m, a, e
Operations $\times, +, -, (\cdot)^2$	**Scalar loss** $\mathcal{L}$

Forward pass

D DERIVATION

Stored value	Operation that produces it	Numeric value
m	$w \times x$	$2 \cdot 3 = 6$
a	$m + b$	$6 + 1 = 7$
e	$a - y$	$7 - 10 = -3$
$\mathcal{L}$	e^2	$(-3)^2 = 9$

Forward pass

D DERIVATION

Stored value	Operation that produces it	Numeric value
m	$w \times x$	$2 \cdot 3 = 6$
a	$m + b$	$6 + 1 = 7$
e	$a - y$	$7 - 10 = -3$
$\mathcal{L}$	e^2	$(-3)^2 = 9$

$\mathcal{L} = 9$ **— now run the graph backward**

Backward: seed and the square operation

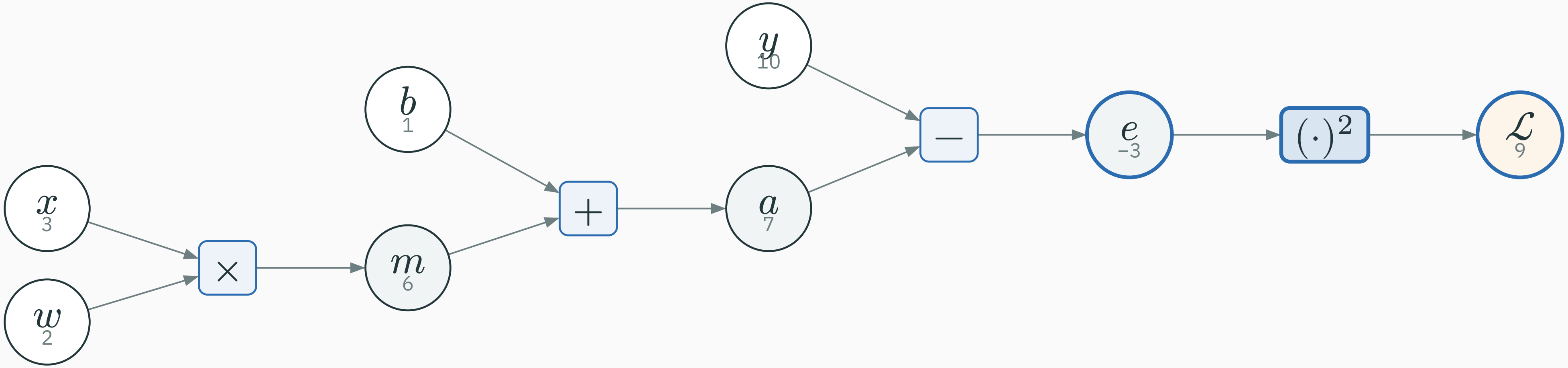
D DERIVATION

Seed the output: $\partial\mathcal{L}/\partial\mathcal{L} = 1$.

Backward: seed and the square operation

D DERIVATION

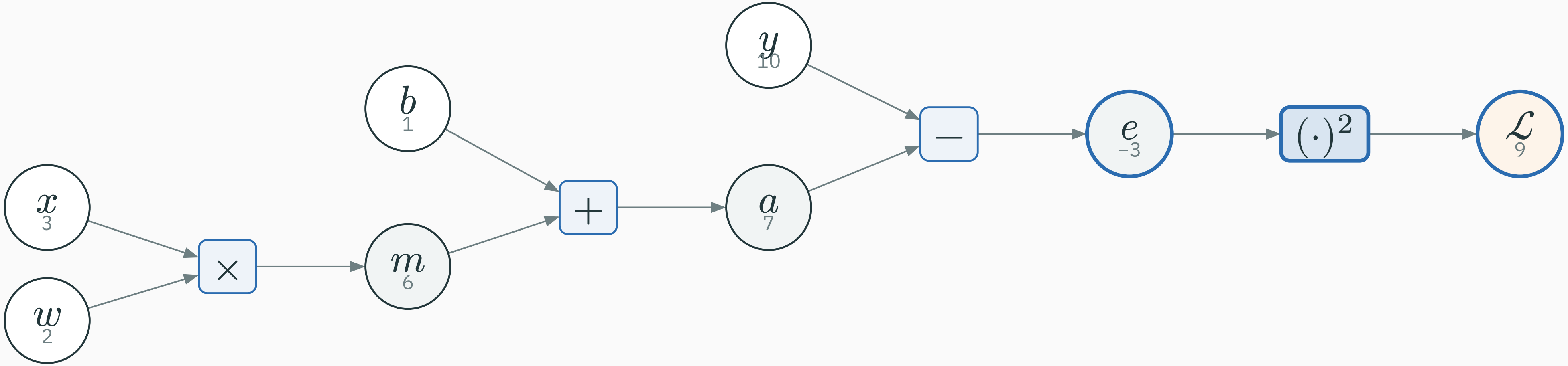
Seed the output: $\partial\mathcal{L}/\partial\mathcal{L} = 1$.



Backward: seed and the square operation

D DERIVATION

Seed the output: $\partial\mathcal{L}/\partial\mathcal{L} = 1$.



Square operation $\mathcal{L} = e^2$, local $\partial\mathcal{L}/\partial e = 2e$:

$$\frac{\partial\mathcal{L}}{\partial e} = 1 \cdot 2e = 1 \cdot 2(-3) = -6.$$

Backward: the subtraction operation

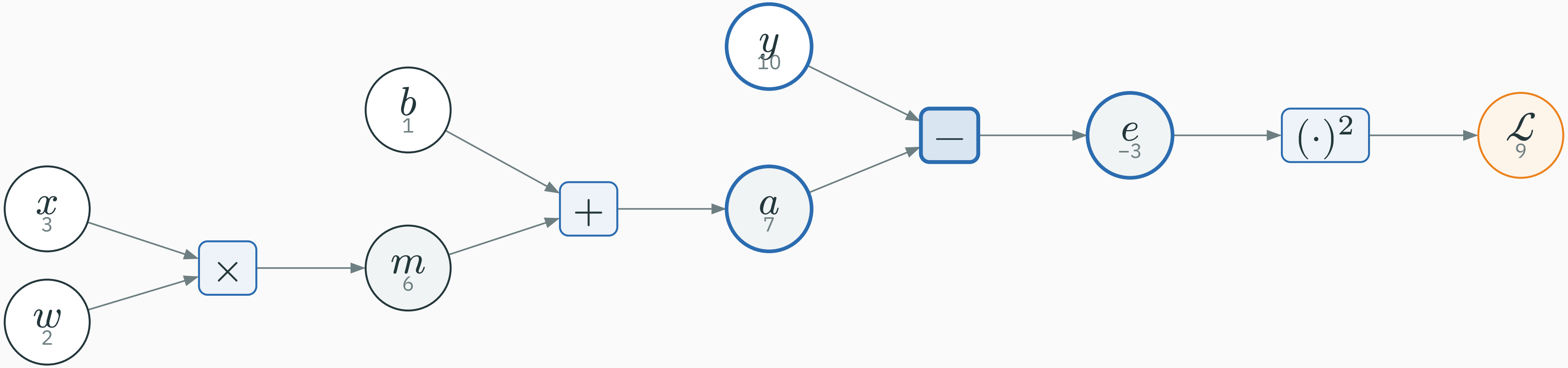
D DERIVATION

Subtraction operation $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

Backward: the subtraction operation

D DERIVATION

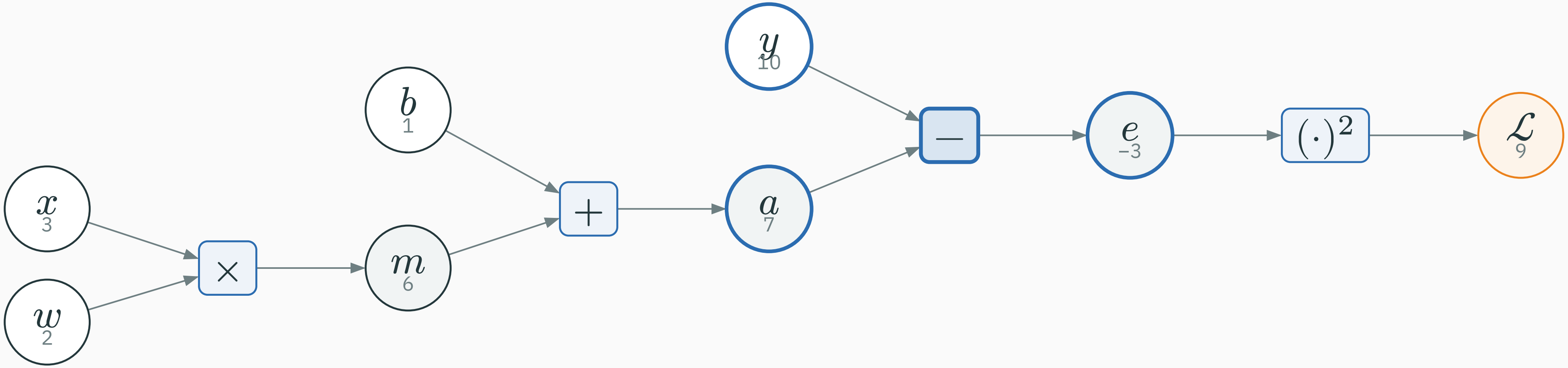
Subtraction operation $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:



Backward: the subtraction operation

D DERIVATION

Subtraction operation $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

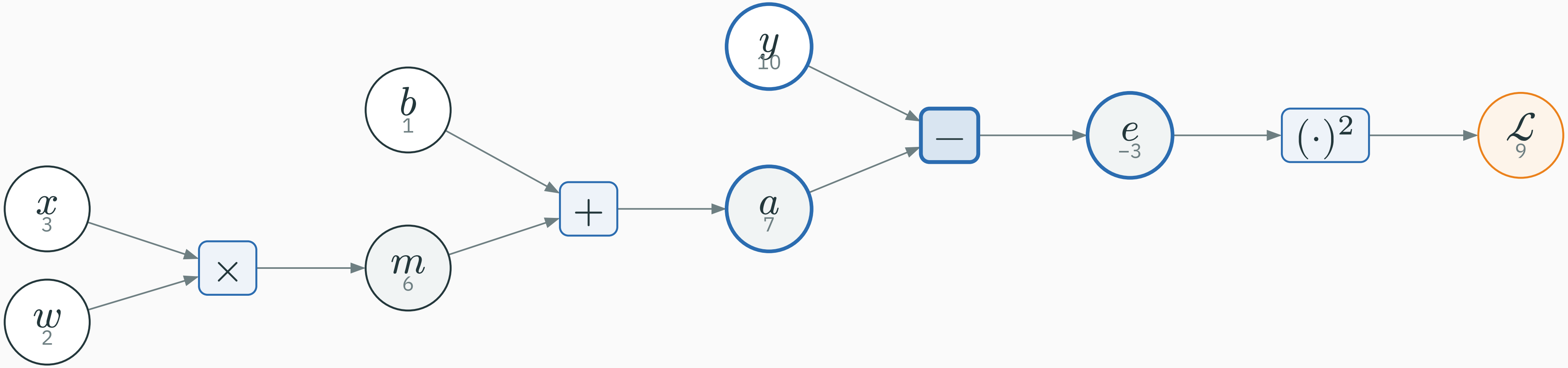


$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial e} \cdot 1 = -6$$

Backward: the subtraction operation

D DERIVATION

Subtraction operation $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

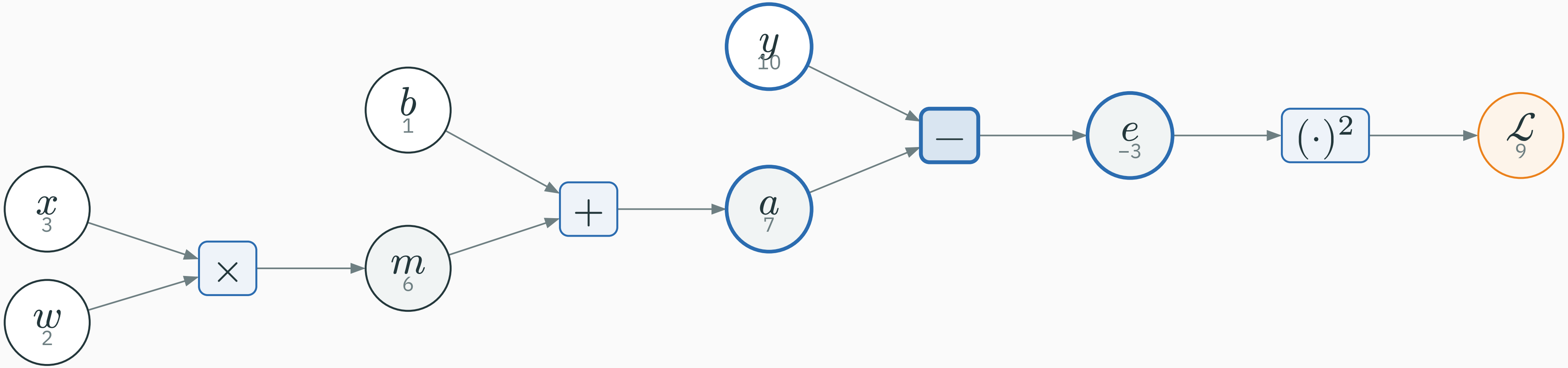


$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial e} \cdot 1 = -6 \qquad \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial e} \cdot (-1) = 6$$

Backward: the subtraction operation

D DERIVATION

Subtraction operation $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:



$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial e} \cdot 1 = -6 \qquad \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial e} \cdot (-1) = 6$$

> The target y also receives a derivative. We would use $\partial \mathcal{L}/\partial y$ if y were itself a learned quantity.

Backward: the addition operation

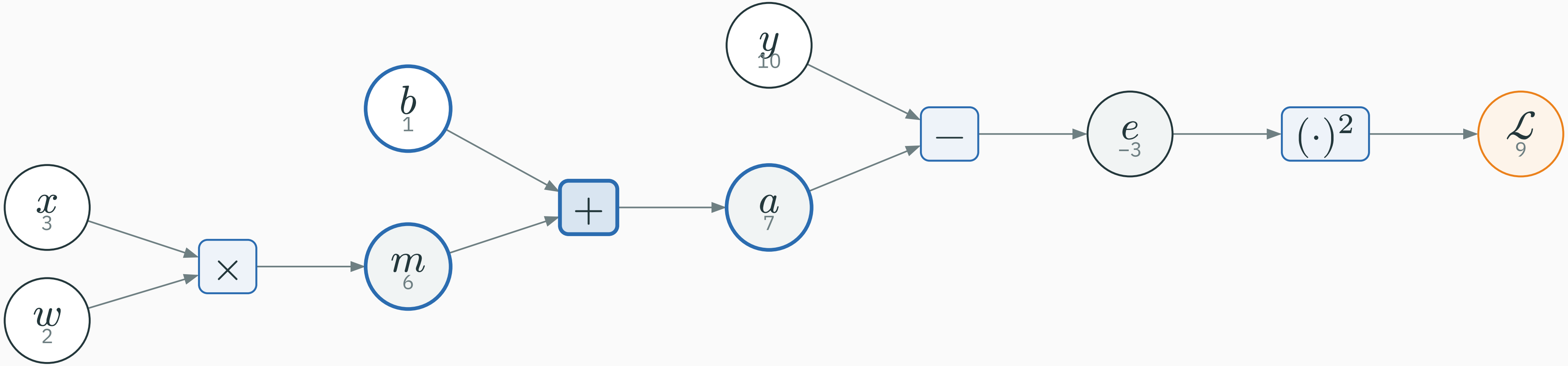
D DERIVATION

Addition operation $a = m + b$ **copies** its gradient:

Backward: the addition operation

D DERIVATION

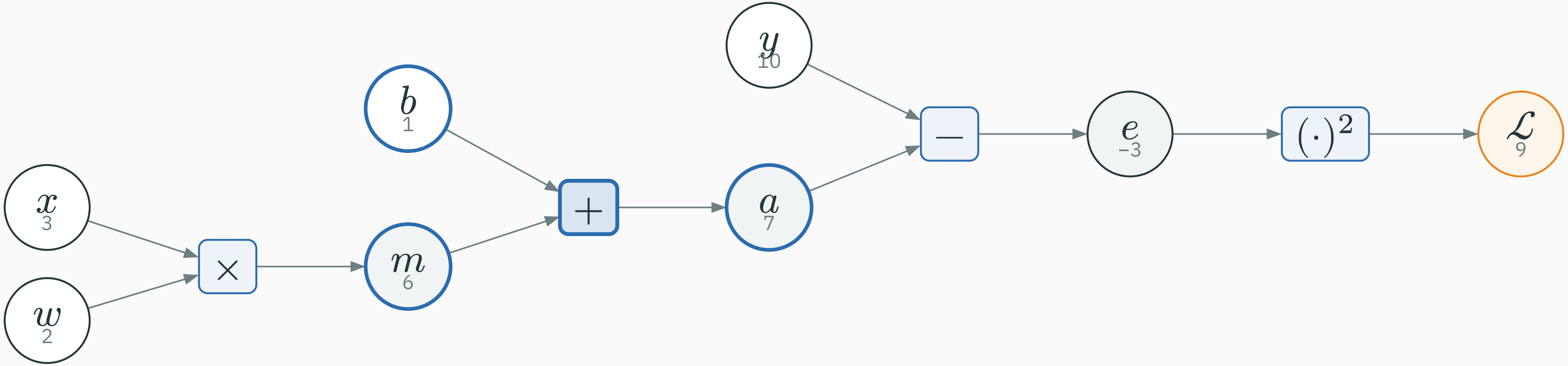
Addition operation $a = m + b$ **copies** its gradient:



Backward: the addition operation

D DERIVATION

Addition operation $a = m + b$ **copies** its gradient:

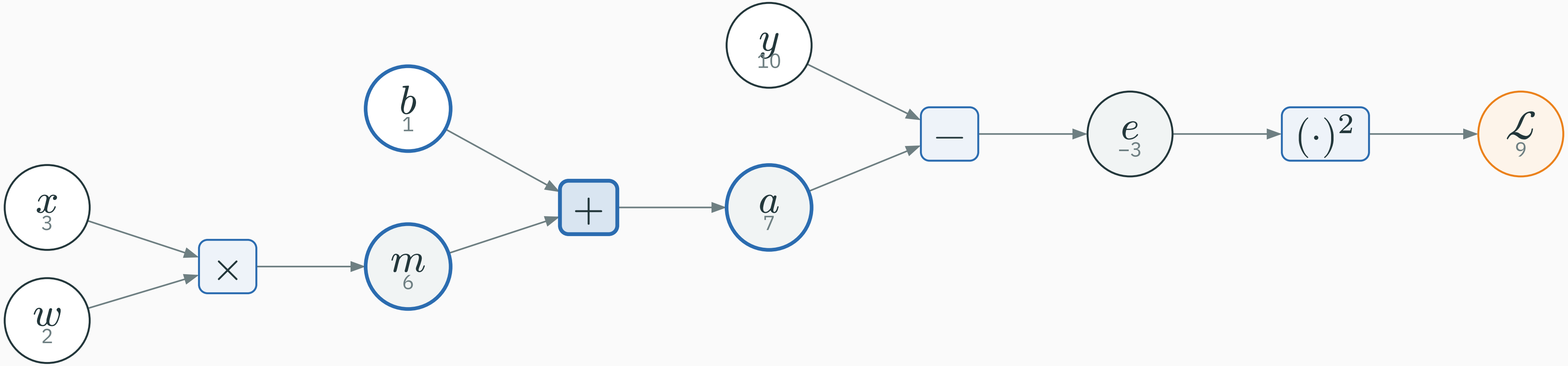


$$\frac{\partial \mathcal{L}}{\partial m} = \frac{\partial \mathcal{L}}{\partial a} = -6, \quad \frac{\partial \mathcal{L}}{\partial b} = -6.$$

Backward: the addition operation

D DERIVATION

Addition operation $a = m + b$ **copies** its gradient:



$$\frac{\partial \mathcal{L}}{\partial m} = \frac{\partial \mathcal{L}}{\partial a} = -6, \quad \frac{\partial \mathcal{L}}{\partial b} = -6.$$

So $\partial \mathcal{L}/\partial b = -6$ is one of our final answers.

Backward: the multiplication operation

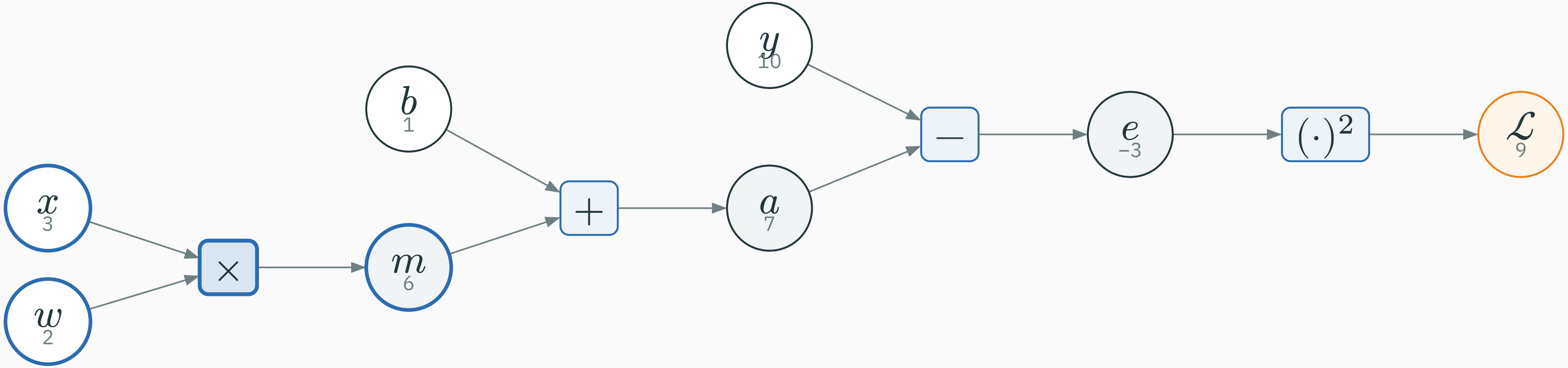
D DERIVATION

Multiplication operation $m = wx$ **sends the other input**:

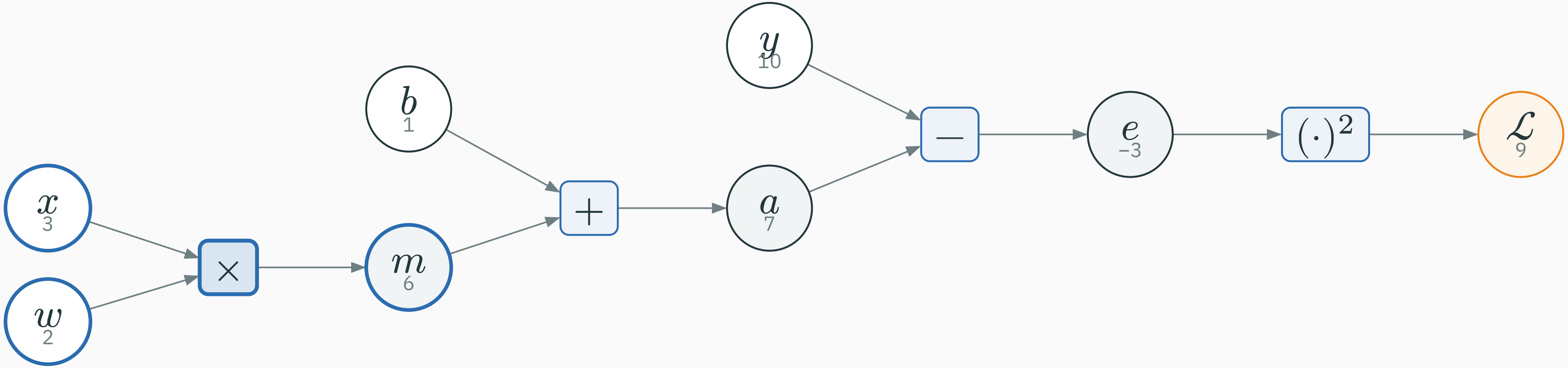
Backward: the multiplication operation

D DERIVATION

Multiplication operation $m = wx$ **sends the other input**:



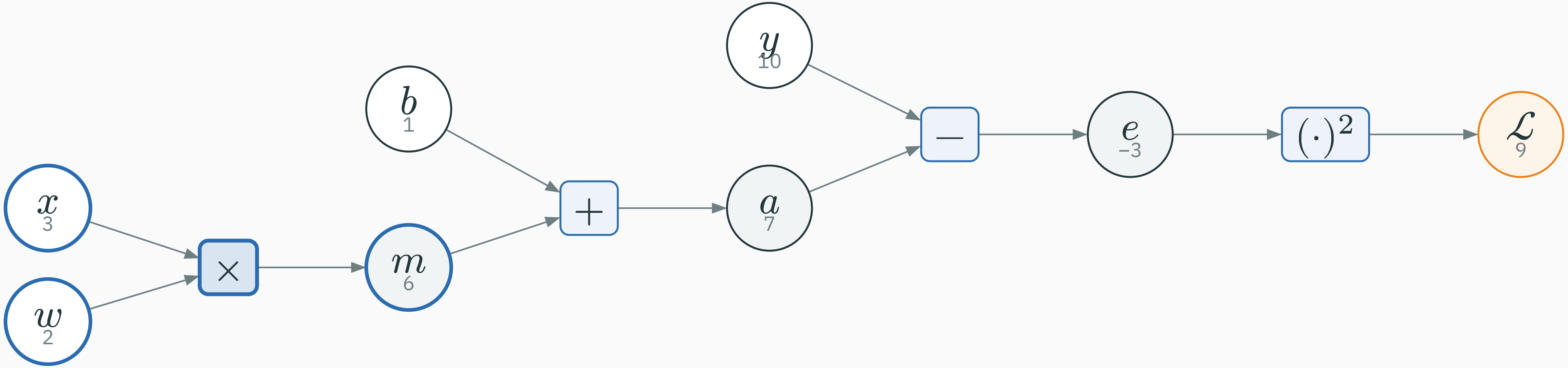
Multiplication operation $m = wx$ **sends the other input**:



$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial m} \cdot x = -6 \cdot 3 = -18$$

Backward: the multiplication operation

Multiplication operation $m = wx$ **sends the other input**:

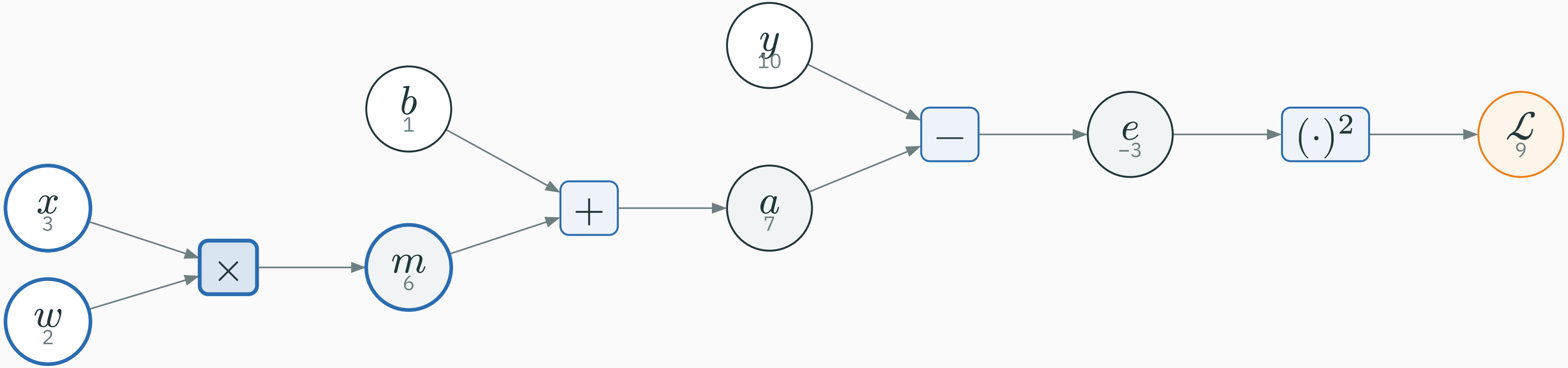


$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial m} \cdot x = -6 \cdot 3 = -18 \qquad \frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial m} \cdot w = -6 \cdot 2 = -12$$

Backward: the multiplication operation

D DERIVATION

Multiplication operation $m = wx$ **sends the other input**:



$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial m} \cdot x = -6 \cdot 3 = -18 \qquad \frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial m} \cdot w = -6 \cdot 2 = -12$$

$$\partial \mathcal{L}/\partial w = -18, \quad \partial \mathcal{L}/\partial x = -12$$

Final gradients

V VISUAL

$\partial\mathcal{L}/\partial w$	$\partial\mathcal{L}/\partial b$	$\partial\mathcal{L}/\partial x$	$\partial\mathcal{L}/\partial y$
-18	-6	-12	6

Final gradients

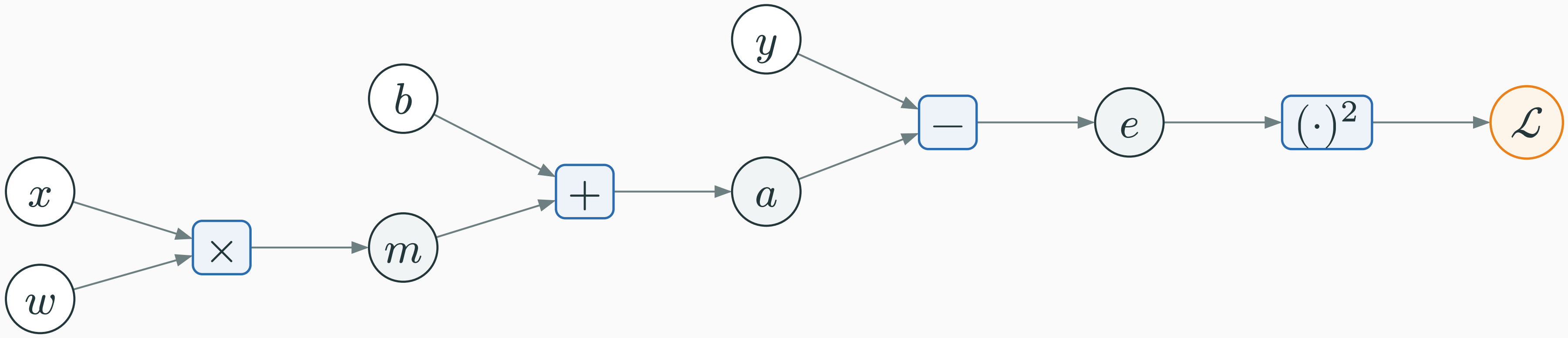
V VISUAL

$\partial\mathcal{L}/\partial w$	$\partial\mathcal{L}/\partial b$	$\partial\mathcal{L}/\partial x$	$\partial\mathcal{L}/\partial y$
-18	-6	-12	6

> One backward sweep produced **all four** partial derivatives — no formula re-derivation per parameter.

All stored values and their gradients

VISUAL



first half	w	x	m	b
forward value	2	3	6	1
gradient	$g_w = -18$	$g_x = -12$	$g_m = -6$	$g_b = -6$

second half	a	y	e	$\mathcal{L}$
forward value	7	10	-3	9
gradient	$g_a = -6$	$g_y = 6$	$g_e = -6$	$g_{\mathcal{L}} = 1$

Check with symbolic differentiation

D DERIVATION

Keep w, x, b, y symbolic, derive the formulas, and only then substitute $wx + b - y = -3$:

Check with symbolic differentiation

D DERIVATION

Keep w, x, b, y symbolic, derive the formulas, and only then substitute $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

Check with symbolic differentiation

Keep w, x, b, y symbolic, derive the formulas, and only then substitute $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

Check with symbolic differentiation

D DERIVATION

Keep w, x, b, y symbolic, derive the formulas, and only then substitute $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

Backprop and the symbolic formulas agree at this point.

PyTorch executes the same graph

I INTERACTIVE

```python
import torch

w = torch.tensor(2.0, requires_grad=True)
x = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
# Targets normally do not track gradients; enable it here to inspect dL/dy.
y = torch.tensor(10.0, requires_grad=True)

m = w * x
a = m + b
e = a - y
L = e**2
for value in (m, a, e, L):
    value.retain_grad()      # keep intermediate grads for inspection

L.backward()
print([t.item() for t in (m, a, e, L)])
# [6.0, 7.0, -3.0, 9.0]
print([t.grad.item() for t in (L, e, a, m, w, x, b, y)])
# [1.0, -6.0, -6.0, -6.0, -18.0, -12.0, -6.0, 6.0]
```

Hand computation → PyTorch autograd

V VISUAL

In our graph	In PyTorch	Meaning
stored scalar value	`Tensor`	one stored forward value
local backward rule	`grad_fn`	operation that produced a non-leaf tensor
saved forward value	saved by the operation	operand needed by a local derivative
reverse sweep	`L.backward()`	seed at $\mathcal{L}$; visit operations in reverse order
leaf derivative	`p.grad`	$\partial\mathcal{L}/\partial p$, accumulated with `+=`
intermediate derivative	`retain_grad()`	keep non-leaf `.grad` for inspection

Hand computation → PyTorch autograd

V VISUAL

In our graph	In PyTorch	Meaning
stored scalar value	`Tensor`	one stored forward value
local backward rule	`grad_fn`	operation that produced a non-leaf tensor
saved forward value	saved by the operation	operand needed by a local derivative
reverse sweep	`L.backward()`	seed at $\mathcal{L}$; visit operations in reverse order
leaf derivative	`p.grad`	$\partial\mathcal{L}/\partial p$, accumulated with `+=`
intermediate derivative	`retain_grad()`	keep non-leaf `.grad` for inspection

reverse mode = direction · backprop = algorithm · autograd = engine: record → save → replay

Checkpoint: read the sign of a gradient

Q QUESTION

We found $\partial\mathcal{L}/\partial w = -18$. For a small positive learning rate, which way does gradient descent move w?

A. Decrease w

B. Increase w

C. Leave w unchanged

D. The sign tells us nothing

Answer: a negative gradient means increase the parameter

A ANSWER

Correct: B — Increase w

Why: Gradient descent subtracts the gradient: $w \leftarrow w - \eta(-18) = w + 18\eta$.

A small gradient step reduces this loss

DERIVATION

Take $\eta = 0.01$ and update:

$$w \leftarrow 2 - 0.01(-18) = 2.18, \qquad b \leftarrow 1 - 0.01(-6) = 1.06.$$

A small gradient step reduces this loss

D DERIVATION

Take $\eta = 0.01$ and update:

$$w \leftarrow 2 - 0.01(-18) = 2.18, \qquad b \leftarrow 1 - 0.01(-6) = 1.06.$$

New prediction: $\hat{y} = 2.18 \cdot 3 + 1.06 = 7.60$.

A small gradient step reduces this loss

D DERIVATION

Take $\eta = 0.01$ and update:

$$w \leftarrow 2 - 0.01(-18) = 2.18, \qquad b \leftarrow 1 - 0.01(-6) = 1.06.$$

New prediction: $\hat{y} = 2.18 \cdot 3 + 1.06 = 7.60.$

loss falls from 9 **to** $(7.60 - 10)^2 = 5.76$

A small gradient step reduces this loss

D DERIVATION

Take $\eta = 0.01$ and update:

$$w \leftarrow 2 - 0.01(-18) = 2.18, \qquad b \leftarrow 1 - 0.01(-6) = 1.06.$$

New prediction: $\hat{y} = 2.18 \cdot 3 + 1.06 = 7.60.$

loss falls from 9 **to** $(7.60 - 10)^2 = 5.76$

> Backprop supplied the direction. The learning rate controls how far we move; a later optimization lecture studies that choice.

Interactive: the computation graph

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Build a scalar computation graph, run the forward pass, then click **backward** to watch each $\partial\mathcal{L}/\partial u$ fill in — exactly the $(wx + b - y)^2$ example above.

Interactive: the computation graph

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Build a scalar computation graph, run the forward pass, then click **backward** to watch each $\partial\mathcal{L}/\partial u$ fill in — exactly the $(wx+b-y)^2$ example above.

Trace which local rule changes when the final squared-error operation is replaced with $|e|$.

Practice the scalar graph three ways

I INTERACTIVE

See it move

Step through values and loss derivatives in the interactive graph ↗.

PyTorch → build autograd

Open the scalar-engine Colab ↓: use plain PyTorch, trace every edge, then compare fused and atomic sigmoid.

Continue through the lecture

Use the complete Colab ↓ for branches, vectors, the dense VJP, batches, one update, and the tiny MLP.

Practice the scalar graph three ways

I INTERACTIVE

See it move

Step through values and loss derivatives in the interactive graph ↗.

PyTorch → build autograd

Open the scalar-engine Colab ↓: use plain PyTorch, trace every edge, then compare fused and atomic sigmoid.

Continue through the lecture

Use the complete Colab ↓ for branches, vectors, the dense VJP, batches, one update, and the tiny MLP.

Easy	change one input and recompute the forward pass
Medium	derive all four gradients before calling `backward()`
Hard	replace the square by $\lvert e\rvert$; state what happens at $e = 0$

From scalars to vectors

Derivative, gradient, Jacobian: the shape tells us

V VISUAL

We store gradients as column vectors. The name of the local object depends on the input and output shapes:

Derivative, gradient, Jacobian: the shape tells us

V VISUAL

We store gradients as column vectors. The name of the local object depends on the input and output shapes:

Operation shape	Local object	Name
$u \in \mathbb{R} \to v \in \mathbb{R}$	$\partial v / \partial u \in \mathbb{R}$	derivative
$\boldsymbol{x} \in \mathbb{R}^d \to z \in \mathbb{R}$	$\partial z / \partial \boldsymbol{x} \in \mathbb{R}^d$	gradient
$\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$	$J_{ij} = \partial z_i / \partial x_j$	Jacobian $\boldsymbol{J} \in \mathbb{R}^{m \times d}$

Derivative, gradient, Jacobian: the shape tells us

V VISUAL

We store gradients as column vectors. The name of the local object depends on the input and output shapes:

Operation shape	Local object	Name
$u \in \mathbb{R} \to v \in \mathbb{R}$	$\partial v / \partial u \in \mathbb{R}$	derivative
$\boldsymbol{x} \in \mathbb{R}^d \to z \in \mathbb{R}$	$\partial z / \partial \boldsymbol{x} \in \mathbb{R}^d$	gradient
$\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$	$J_{ij} = \partial z_i / \partial x_j$	Jacobian $\boldsymbol{J} \in \mathbb{R}^{m \times d}$

a Jacobian contains one local derivative for every output–input pair

A 2×2 Jacobian is four familiar derivatives

V VISUAL

Take a vector operation with two inputs and two outputs:

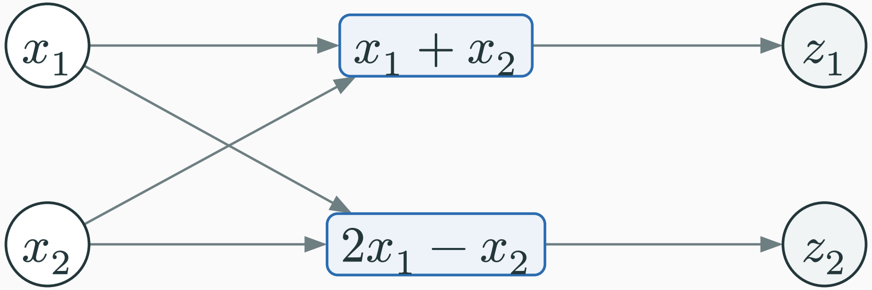
$$z_1 = x_1 + x_2, \quad z_2 = 2x_1 - x_2.$$

A 2×2 Jacobian is four familiar derivatives

V VISUAL

Take a vector operation with two inputs and two outputs:

$$z_1 = x_1 + x_2, \quad z_2 = 2x_1 - x_2.$$

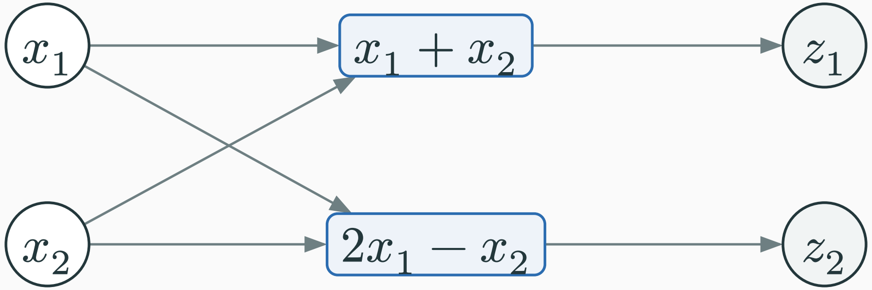


A 2×2 Jacobian is four familiar derivatives

V VISUAL

Take a vector operation with two inputs and two outputs:

$$z_1 = x_1 + x_2, \quad z_2 = 2x_1 - x_2.$$

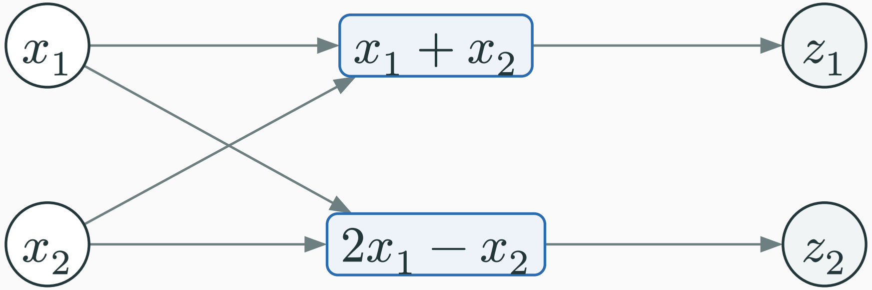


Output	**Input x_1**	**Input x_2**
z_1	$\partial z_1/\partial x_1 = 1$	$\partial z_1/\partial x_2 = 1$
z_2	$\partial z_2/\partial x_1 = 2$	$\partial z_2/\partial x_2 = -1$

A 2×2 Jacobian is four familiar derivatives

Take a vector operation with two inputs and two outputs:

$$z_1 = x_1 + x_2, \quad z_2 = 2x_1 - x_2.$$



Output	Input x_1	Input x_2
z_1	$\partial z_1/\partial x_1 = 1$	$\partial z_1/\partial x_2 = 1$
z_2	$\partial z_2/\partial x_1 = 2$	$\partial z_2/\partial x_2 = -1$

$J = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$ · **rows are outputs · columns are inputs**

Vector chain rule: make the shapes fit

D DERIVATION

For $\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$, let the local Jacobian be

$$\boldsymbol{J}_{ij} = \partial z_i / \partial x_j, \quad \boldsymbol{J} \in \mathbb{R}^{m \times d}.$$

Vector chain rule: make the shapes fit

D DERIVATION

For $\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$, let the local Jacobian be

$$\boldsymbol{J}_{ij} = \partial z_i / \partial x_j, \quad \boldsymbol{J} \in \mathbb{R}^{m \times d}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$$

Vector chain rule: make the shapes fit

D DERIVATION

For $\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$, let the local Jacobian be

$$\boldsymbol{J}_{ij} = \partial z_i / \partial x_j, \quad \boldsymbol{J} \in \mathbb{R}^{m \times d}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \ \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$$

$$(d \times 1) \quad = \quad (d \times m) \quad (m \times 1)$$

Vector chain rule: make the shapes fit

D DERIVATION

For $\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$, let the local Jacobian be

$$\boldsymbol{J}_{ij} = \partial z_i / \partial x_j, \quad \boldsymbol{J} \in \mathbb{R}^{m \times d}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$$

$$(d \times 1) \quad = \quad (d \times m) \quad (m \times 1)$$

downstream gradient = local Jacobian transpose × upstream gradient

Vector chain rule: make the shapes fit

D DERIVATION

For $\boldsymbol{x} \in \mathbb{R}^d \to \boldsymbol{z} \in \mathbb{R}^m$, let the local Jacobian be

$$\boldsymbol{J}_{ij} = \partial z_i / \partial x_j, \quad \boldsymbol{J} \in \mathbb{R}^{m \times d}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$$

$$(d \times 1) \quad = \quad (d \times m) \quad (m \times 1)$$

downstream gradient = local Jacobian transpose × upstream gradient

> When $d = m = 1$, $\boldsymbol{J}$ is one number and this is the scalar chain rule from earlier.

The transpose adds every output's contribution

D DERIVATION

For the previous operation, suppose the arriving gradient is

$$\boldsymbol{g}_z = \partial \mathcal{L} / \partial \boldsymbol{z} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

The transpose adds every output's contribution

D DERIVATION

For the previous operation, suppose the arriving gradient is

$$\boldsymbol{g}_z = \partial \mathcal{L} / \partial \boldsymbol{z} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \boldsymbol{g}_z = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

The transpose adds every output's contribution

D DERIVATION

For the previous operation, suppose the arriving gradient is

$$\boldsymbol{g}_z = \partial \mathcal{L} / \partial \boldsymbol{z} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{J}^\top \boldsymbol{g}_z = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

$\partial \mathcal{L} / \partial x_1$	$3(1) + 4(2) = 11$
$\partial \mathcal{L} / \partial x_2$	$3(1) + 4(-1) = -1$

The transpose adds every output's contribution

D DERIVATION

For the previous operation, suppose the arriving gradient is

$$\boldsymbol{g}_z = \partial\mathcal{L}/\partial\boldsymbol{z} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{x}} = \boldsymbol{J}^\top \boldsymbol{g}_z = \begin{pmatrix} 1 & 2 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 11 \\ -1 \end{pmatrix}$$

$\partial\mathcal{L}/\partial x_1$	$3(1) + 4(2) = 11$
$\partial\mathcal{L}/\partial x_2$	$3(1) + 4(-1) = -1$

the vector rule is the scalar branch rule applied to every input

An affine operation adds scalar products and a bias

V VISUAL

Affine means a weighted sum plus a bias:

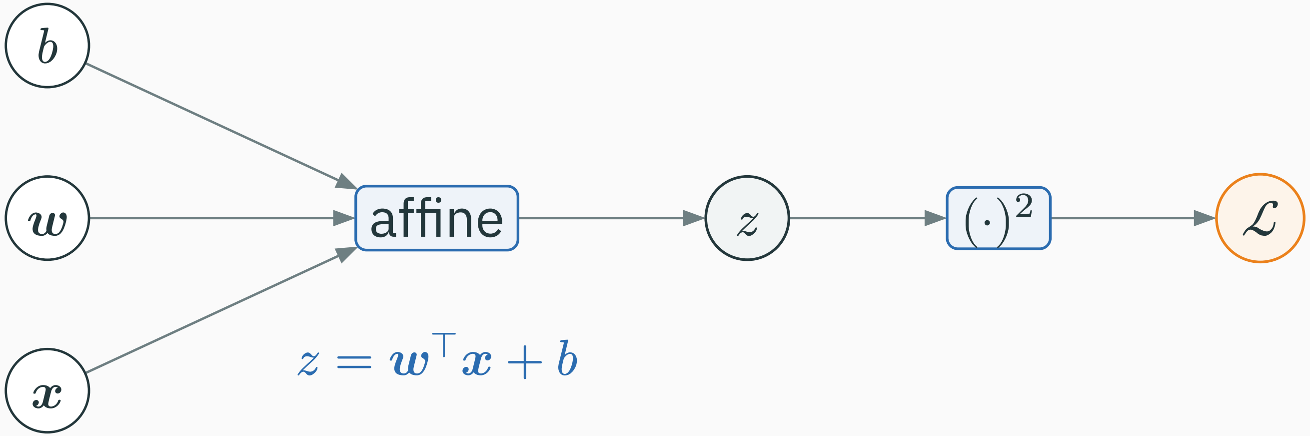
$$z = \boldsymbol{w}^\top \boldsymbol{x} + b \quad = \quad \sum_{j=1}^{d} w_j x_j + b.$$

An affine operation adds scalar products and a bias

V VISUAL

Affine means a weighted sum plus a bias:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b \quad = \quad \sum_{j=1}^{d} w_j x_j + b.$$



Change one scalar while holding every other value fixed

D DERIVATION

Start from $z = \sum_j w_j x_j + b$. Change only one input by a small amount Δ:

Change one scalar while holding every other value fixed

D DERIVATION

Start from $z = \sum_j w_j x_j + b$. Change only one input by a small amount Δ:

Change	Resulting change in z	Divide by Δ
$w_i \to w_i + \Delta$	$((w_i + \Delta)x_i) - w_i x_i = x_i \Delta$	$\partial z / \partial w_i = x_i$
$x_i \to x_i + \Delta$	$w_i(x_i + \Delta) - w_i x_i = w_i \Delta$	$\partial z / \partial x_i = w_i$
$b \to b + \Delta$	$(b + \Delta) - b = \Delta$	$\partial z / \partial b = 1$

Change one scalar while holding every other value fixed

D DERIVATION

Start from $z = \sum_j w_j x_j + b$. Change only one input by a small amount Δ:

Change	**Resulting change in z**	**Divide by Δ**
$w_i \to w_i + \Delta$	$((w_i + \Delta)x_i) - w_i x_i = x_i \Delta$	$\partial z / \partial w_i = x_i$
$x_i \to x_i + \Delta$	$w_i(x_i + \Delta) - w_i x_i = w_i \Delta$	$\partial z / \partial x_i = w_i$
$b \to b + \Delta$	$(b + \Delta) - b = \Delta$	$\partial z / \partial b = 1$

local derivative = change in the operation's output ÷ change in one input

Stack the coordinate derivatives into gradient vectors

D DERIVATION

The derivatives with respect to the coordinates of a vector are stored in one column:

Stack the coordinate derivatives into gradient vectors

D DERIVATION

The derivatives with respect to the coordinates of a vector are stored in one column:

$$\frac{\partial z}{\partial \boldsymbol{w}} := \begin{pmatrix} \partial z / \partial w_1 \\ \vdots \\ \partial z / \partial w_d \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} = \boldsymbol{x}$$

Stack the coordinate derivatives into gradient vectors

D DERIVATION

The derivatives with respect to the coordinates of a vector are stored in one column:

$$\frac{\partial z}{\partial \boldsymbol{w}} := \begin{pmatrix} \partial z/\partial w_1 \\ \vdots \\ \partial z/\partial w_d \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} = \boldsymbol{x}$$

$$\frac{\partial z}{\partial \boldsymbol{x}} := \begin{pmatrix} \partial z/\partial x_1 \\ \vdots \\ \partial z/\partial x_d \end{pmatrix} = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix} = \boldsymbol{w}$$

Stack the coordinate derivatives into gradient vectors

D DERIVATION

The derivatives with respect to the coordinates of a vector are stored in one column:

$$\frac{\partial z}{\partial \boldsymbol{w}} := \begin{pmatrix} \partial z/\partial w_1 \\ \vdots \\ \partial z/\partial w_d \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} = \boldsymbol{x}$$

$$\frac{\partial z}{\partial \boldsymbol{x}} := \begin{pmatrix} \partial z/\partial x_1 \\ \vdots \\ \partial z/\partial x_d \end{pmatrix} = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix} = \boldsymbol{w}$$

With $\boldsymbol{x} = \binom{2}{-1}$ and $\boldsymbol{w} = \binom{1}{3}$:

$$\frac{\partial z}{\partial \boldsymbol{w}} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \frac{\partial z}{\partial \boldsymbol{x}} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \frac{\partial z}{\partial b} = 1.$$

Stack the coordinate derivatives into gradient vectors

D DERIVATION

The derivatives with respect to the coordinates of a vector are stored in one column:

$$\frac{\partial z}{\partial \boldsymbol{w}} := \begin{pmatrix} \partial z/\partial w_1 \\ \vdots \\ \partial z/\partial w_d \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} = \boldsymbol{x}$$

$$\frac{\partial z}{\partial \boldsymbol{x}} := \begin{pmatrix} \partial z/\partial x_1 \\ \vdots \\ \partial z/\partial x_d \end{pmatrix} = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix} = \boldsymbol{w}$$

With $\boldsymbol{x} = \binom{2}{-1}$ and $\boldsymbol{w} = \binom{1}{3}$:

$$\frac{\partial z}{\partial \boldsymbol{w}} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \frac{\partial z}{\partial \boldsymbol{x}} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \frac{\partial z}{\partial b} = 1.$$

the vector formulas are just the scalar coordinate derivatives stacked together

Affine forward: multiply componentwise, then add

D DERIVATION

Use $x = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$, $w = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$, and $b = 0$.

Affine forward: multiply componentwise, then add

D DERIVATION

Use $x = \binom{2}{-1}$, $w = \binom{1}{3}$, and $b = 0$.

Coordinate	x_i	w_i	$w_i x_i$
1	2	1	$1 \cdot 2 = 2$
2	-1	3	$3 \cdot (-1) = -3$

Affine forward: multiply componentwise, then add

DERIVATION

Use $\boldsymbol{x} = \binom{2}{-1}$, $\boldsymbol{w} = \binom{1}{3}$, and $b = 0$.

Coordinate	x_i	w_i	$w_i x_i$
1	2	1	$1 \cdot 2 = 2$
2	-1	3	$3 \cdot (-1) = -3$

$$\boldsymbol{w}^\top \boldsymbol{x} = 2 + (-3) = -1, \quad z = -1 + 0 = -1.$$

Affine forward: multiply componentwise, then add

D DERIVATION

Use $\boldsymbol{x} = \binom{2}{-1}$, $\boldsymbol{w} = \binom{1}{3}$, and $b = 0$.

Coordinate	x_i	w_i	$w_i x_i$
1	2	1	$1 \cdot 2 = 2$
2	-1	3	$3 \cdot (-1) = -3$

$$\boldsymbol{w}^\top \boldsymbol{x} = 2 + (-3) = -1, \quad z = -1 + 0 = -1.$$

$$\mathcal{L} = z^2 = (-1)^2 = 1.$$

Affine forward: multiply componentwise, then add

D DERIVATION

Use $\boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$, $\boldsymbol{w} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$, and $b = 0$.

Coordinate	x_i	w_i	$w_i x_i$
1	2	1	$1 \cdot 2 = 2$
2	-1	3	$3 \cdot (-1) = -3$

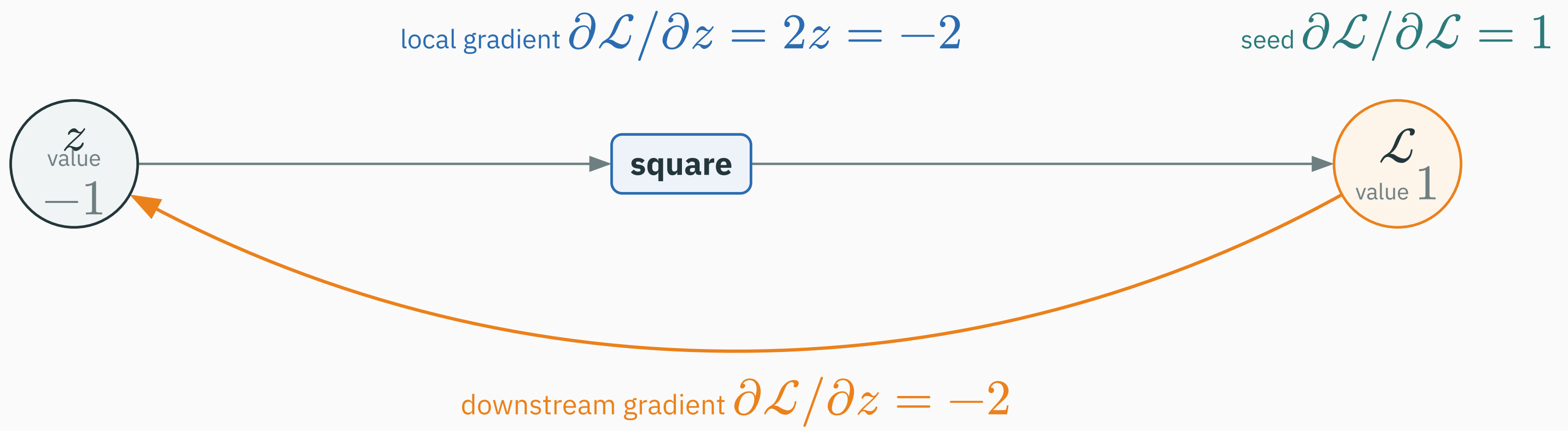
$$\boldsymbol{w}^\top \boldsymbol{x} = 2 + (-3) = -1, \quad z = -1 + 0 = -1.$$

$$\mathcal{L} = z^2 = (-1)^2 = 1.$$

dot product = multiply matching entries, then sum

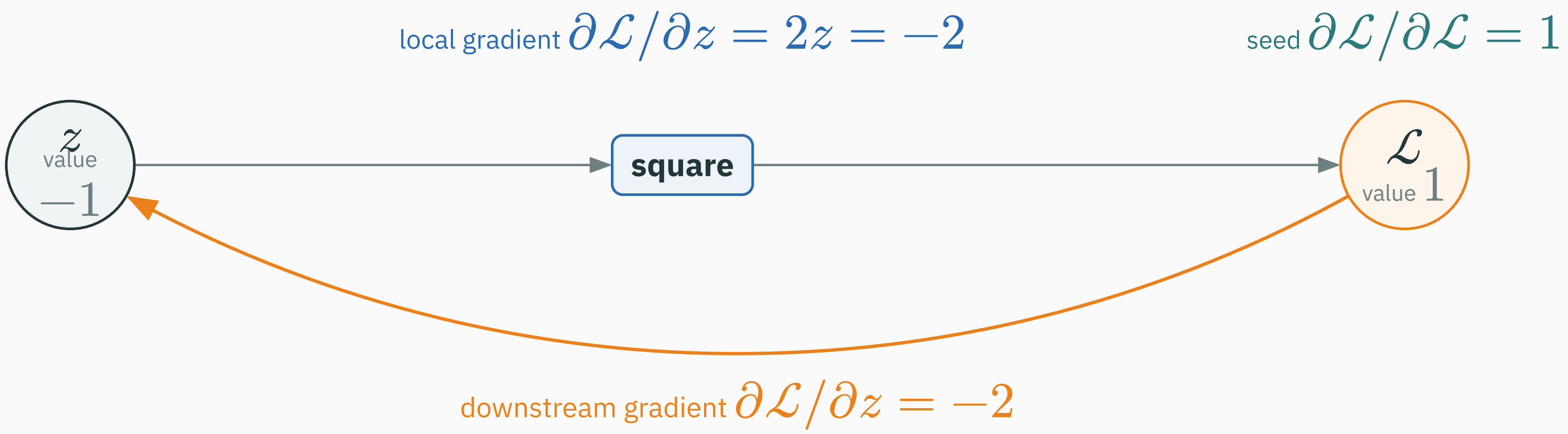
First backward step: the square operation

D DERIVATION



First backward step: the square operation

D DERIVATION

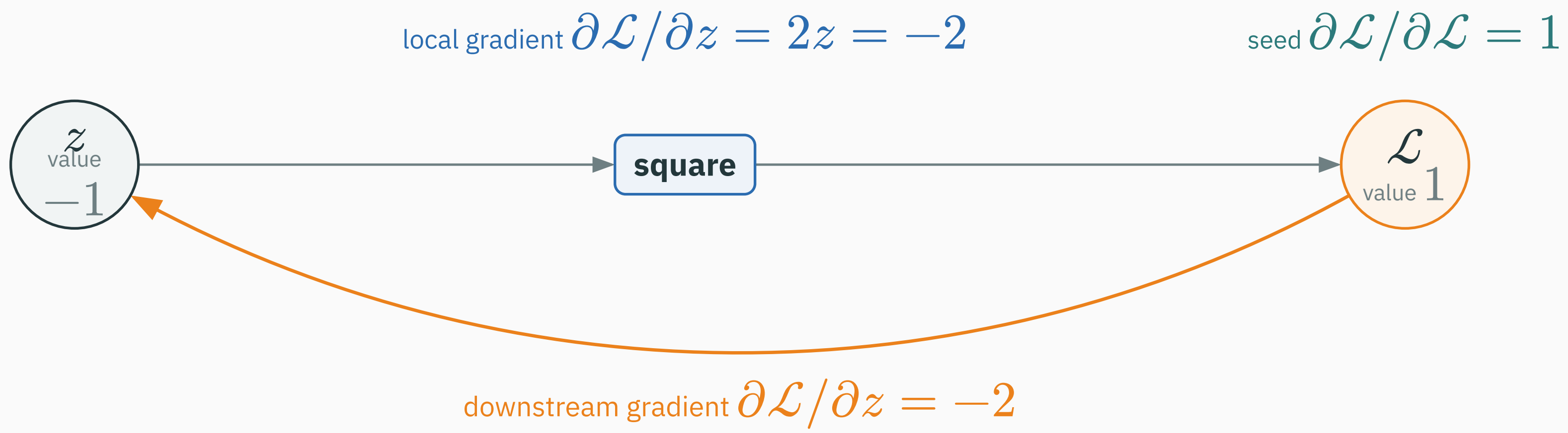


Seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$. The square has local derivative $2z = -2$, so

$$\frac{\partial\mathcal{L}}{\partial z} = 1 \cdot 2z = -2.$$

First backward step: the square operation

D DERIVATION



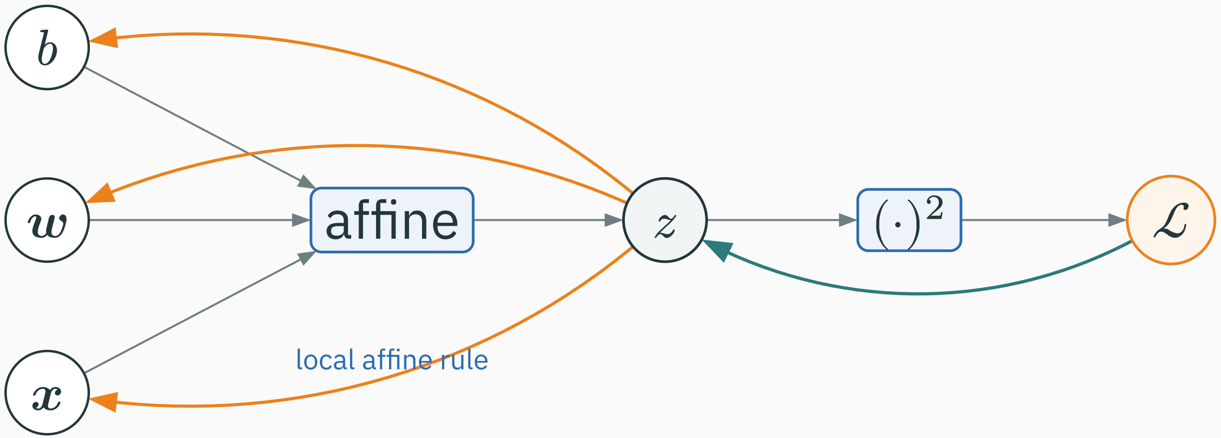
Seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$. The square has local derivative $2z = -2$, so

$$\frac{\partial\mathcal{L}}{\partial z} = 1 \cdot 2z = -2.$$

the square returns -2; it now arrives at the affine operation

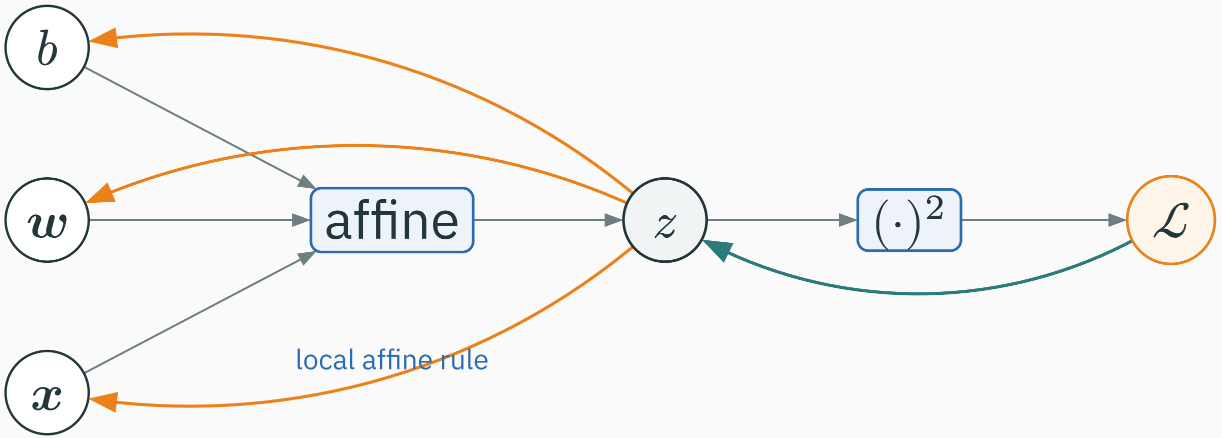
Affine backward: apply the three local gradients

D DERIVATION



Affine backward: apply the three local gradients

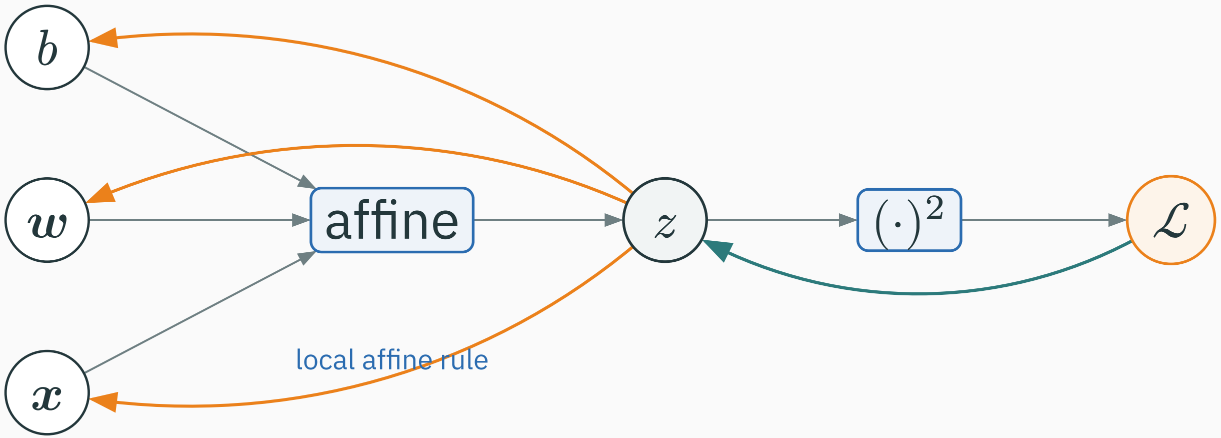
D DERIVATION



Let $g_z = \partial\mathcal{L}/\partial z = -2$ arrive at the affine operation. Its local gradients send back:

Affine backward: apply the three local gradients

D DERIVATION

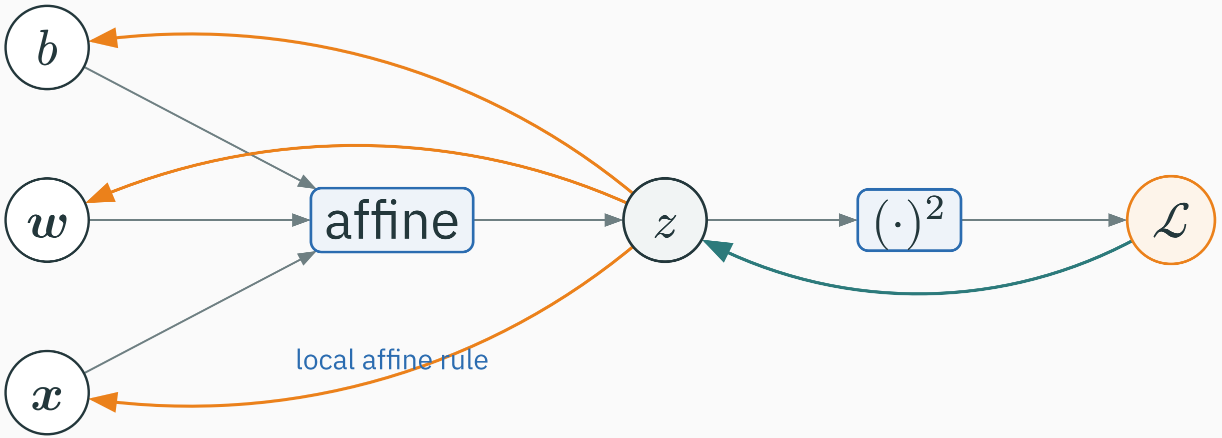


Let $g_z = \partial\mathcal{L}/\partial z = -2$ arrive at the affine operation. Its local gradients send back:

Input	**Local gradient**	**Downstream gradient**
$\boldsymbol{w}$	$\partial z/\partial \boldsymbol{w} = \boldsymbol{x}$	$\partial\mathcal{L}/\partial \boldsymbol{w} = (-2)\boldsymbol{x} = (-4, 2)^\top$
$\boldsymbol{x}$	$\partial z/\partial \boldsymbol{x} = \boldsymbol{w}$	$\partial\mathcal{L}/\partial \boldsymbol{x} = (-2)\boldsymbol{w} = (-2, -6)^\top$
b	$\partial z/\partial b = 1$	$\partial\mathcal{L}/\partial b = -2$

Affine backward: apply the three local gradients

D DERIVATION



Let $g_z = \partial\mathcal{L}/\partial z = -2$ arrive at the affine operation. Its local gradients send back:

Input	Local gradient	Downstream gradient
$\boldsymbol{w}$	$\partial z/\partial \boldsymbol{w} = \boldsymbol{x}$	$\partial\mathcal{L}/\partial \boldsymbol{w} = (-2)\boldsymbol{x} = (-4, 2)^\top$
$\boldsymbol{x}$	$\partial z/\partial \boldsymbol{x} = \boldsymbol{w}$	$\partial\mathcal{L}/\partial \boldsymbol{x} = (-2)\boldsymbol{w} = (-2, -6)^\top$
b	$\partial z/\partial b = 1$	$\partial\mathcal{L}/\partial b = -2$

one arriving gradient g_z produces three separate downstream gradients

PyTorch gives the same vector gradients

INTERACTIVE

```python
import torch

x = torch.tensor([2., -1.], requires_grad=True)
w = torch.tensor([1.,  3.], requires_grad=True)
b = torch.tensor(0., requires_grad=True)

z = w @ x + b
L = z**2
L.backward()

print(z.item(), L.item())  # -1.0, 1.0
print(w.grad)              # tensor([-4.,  2.])
print(x.grad, b.grad)      # tensor([-2., -6.]), tensor(-2.)
```

PyTorch gives the same vector gradients

INTERACTIVE

```python
import torch

x = torch.tensor([2., -1.], requires_grad=True)
w = torch.tensor([1.,  3.], requires_grad=True)
b = torch.tensor(0., requires_grad=True)

z = w @ x + b
L = z**2
L.backward()

print(z.item(), L.item())  # -1.0, 1.0
print(w.grad)              # tensor([-4.,  2.])
print(x.grad, b.grad)      # tensor([-2., -6.]), tensor(-2.)
```

`@` performs the dot product; `backward()` still applies the same local rules

Under the hood: the affine backward rule

D DERIVATION

For this one-output affine node, PyTorch sends the upstream gradient `grad_z` into a backward rule. A readable teaching equivalent is:

```
class Affine(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, w, b):
        ctx.save_for_backward(x, w)   # needed later
        return w @ x + b

    @staticmethod
    def backward(ctx, grad_z):        # grad_z = dL/dz
        x, w = ctx.saved_tensors
        grad_x = grad_z * w           # dL/dx = dL/dz * dz/dx
        grad_w = grad_z * x           # dL/dw = dL/dz * dz/dw
        grad_b = grad_z               # dL/db = dL/dz * 1
        return grad_x, grad_w, grad_b # one gradient per input
```

Under the hood: the affine backward rule

D DERIVATION

For this one-output affine node, PyTorch sends the upstream gradient `grad_z` into a backward rule. A readable teaching equivalent is:

```
class Affine(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, w, b):
        ctx.save_for_backward(x, w)   # needed later
        return w @ x + b

    @staticmethod
    def backward(ctx, grad_z):        # grad_z = dL/dz
        x, w = ctx.saved_tensors
        grad_x = grad_z * w           # dL/dx = dL/dz * dz/dx
        grad_w = grad_z * x           # dL/dw = dL/dz * dz/dw
        grad_b = grad_z               # dL/db = dL/dz * 1
        return grad_x, grad_w, grad_b # one gradient per input
```

`backward` receives $\partial\frac{\mathcal{L}}{\partial}z$ and returns $\partial\frac{\mathcal{L}}{\partial}x$, $\partial\frac{\mathcal{L}}{\partial}w$, and $\partial\frac{\mathcal{L}}{\partial}b$

Under the hood: the affine backward rule

D DERIVATION

For this one-output affine node, PyTorch sends the upstream gradient `grad_z` into a backward rule. A readable teaching equivalent is:

```
class Affine(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, w, b):
        ctx.save_for_backward(x, w)   # needed later
        return w @ x + b

    @staticmethod
    def backward(ctx, grad_z):        # grad_z = dL/dz
        x, w = ctx.saved_tensors
        grad_x = grad_z * w           # dL/dx = dL/dz * dz/dx
        grad_w = grad_z * x           # dL/dw = dL/dz * dz/dw
        grad_b = grad_z               # dL/db = dL/dz * 1
        return grad_x, grad_w, grad_b # one gradient per input
```

`backward` receives $\partial \frac{\mathcal{L}}{\partial} z$ and returns $\partial \frac{\mathcal{L}}{\partial} x$, $\partial \frac{\mathcal{L}}{\partial} w$, and $\partial \frac{\mathcal{L}}{\partial} b$

Teaching equivalent. `nn.Linear.forward` calls `F.linear`; production autograd uses generated native kernels. Linear source ↗ · derivative rules ↗

From one scalar output to a vector output

One neuron still produces one scalar

V VISUAL

Start with the affine rule we already know:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b.$$

One neuron still produces one scalar

V VISUAL

Start with the affine rule we already know:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b.$$

Quantity	$\boldsymbol{w}^\top$	$\boldsymbol{x}$	z
Shape	$(1 \times d)$	$(d \times 1)$	(1×1)

One neuron still produces one scalar

V VISUAL

Start with the affine rule we already know:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b.$$

Quantity	$\boldsymbol{w}^\top$	$\boldsymbol{x}$	z
Shape	$(1 \times d)$	$(d \times 1)$	(1×1)

$$(1 \times d)(d \times 1) = (1 \times 1)$$

One neuron still produces one scalar

V VISUAL

Start with the affine rule we already know:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b.$$

Quantity	$\boldsymbol{w}^\top$	$\boldsymbol{x}$	z
Shape	$(1 \times d)$	$(d \times 1)$	(1×1)

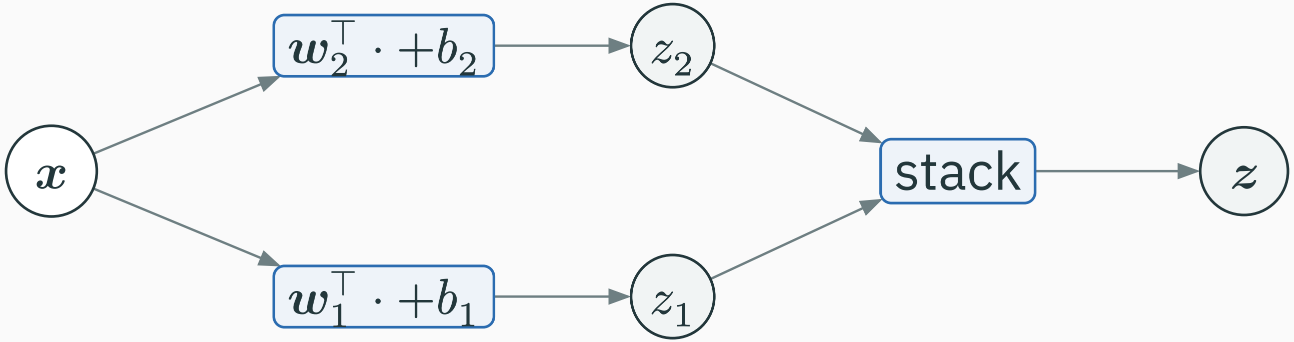
$$(1 \times d)(d \times 1) = (1 \times 1)$$

one weight row dotted with one input vector produces one number

Two neurons produce two scalars

V VISUAL

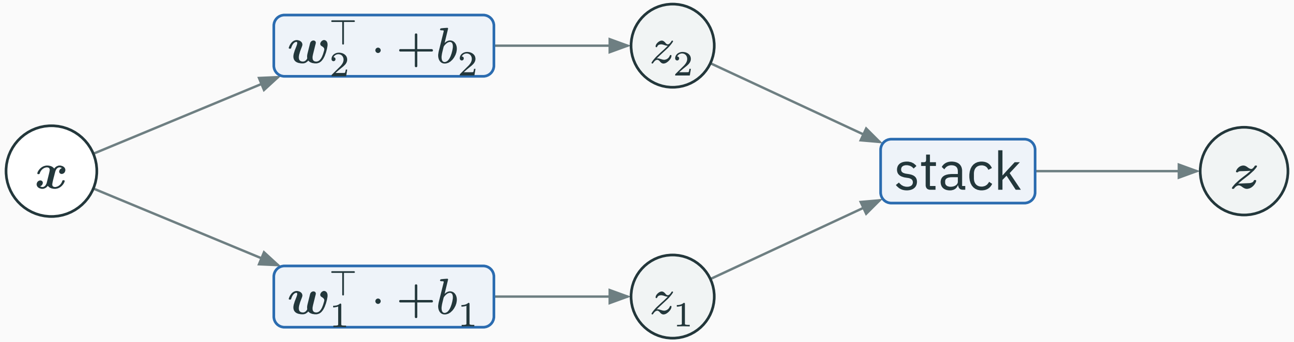
Give each neuron its own weight row and bias; both read the same $\boldsymbol{x}$.



Two neurons produce two scalars

V VISUAL

Give each neuron its own weight row and bias; both read the same $\boldsymbol{x}$.

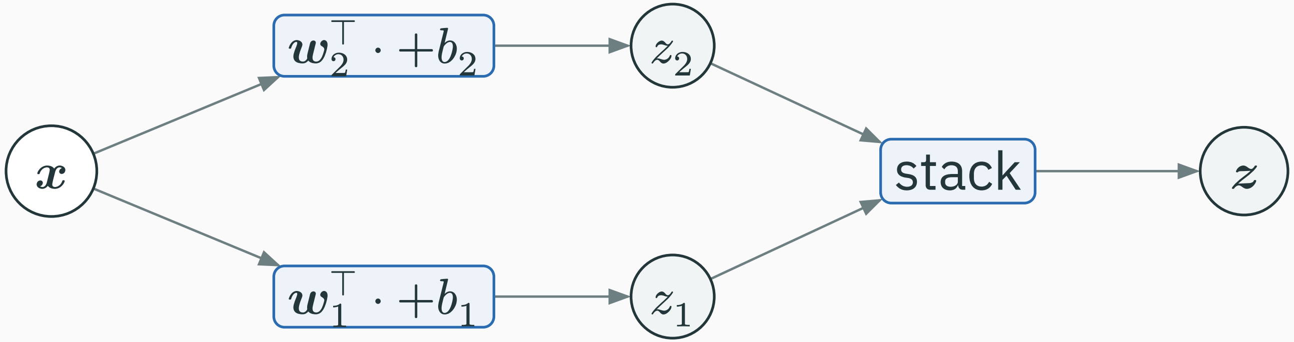


$$z_1 = \boldsymbol{w}_1^\top \boldsymbol{x} + b_1, \quad z_2 = \boldsymbol{w}_2^\top \boldsymbol{x} + b_2.$$

Two neurons produce two scalars

VISUAL

Give each neuron its own weight row and bias; both read the same $\boldsymbol{x}$.



$$z_1 = \boldsymbol{w}_1^\top \boldsymbol{x} + b_1, \quad z_2 = \boldsymbol{w}_2^\top \boldsymbol{x} + b_2.$$

stack the two numbers into $\boldsymbol{z} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix}$

Stacking the two neurons gives one dense layer

D DERIVATION

$$\boldsymbol{W} = \begin{pmatrix} \boldsymbol{w}_1^\top \\ \boldsymbol{w}_2^\top \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \quad \boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

Stacking the two neurons gives one dense layer

D DERIVATION

$$\boldsymbol{W} = \begin{pmatrix} \boldsymbol{w}_1^\top \\ \boldsymbol{w}_2^\top \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \quad \boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

$\boldsymbol{W}$	$\boldsymbol{x}$	$\boldsymbol{b}$	$\boldsymbol{z}$
$(m \times d)$	$(d \times 1)$	$(m \times 1)$	$(m \times 1)$

Stacking the two neurons gives one dense layer

D DERIVATION

$$\boldsymbol{W} = \begin{pmatrix} \boldsymbol{w}_1^\top \\ \boldsymbol{w}_2^\top \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \quad \boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

$\boldsymbol{W}$	$\boldsymbol{x}$	$\boldsymbol{b}$	$\boldsymbol{z}$
$(m \times d)$	$(d \times 1)$	$(m \times 1)$	$(m \times 1)$

$$(m \times d)(d \times 1) + (m \times 1) = (m \times 1)$$

Stacking the two neurons gives one dense layer

D DERIVATION

$$\boldsymbol{W} = \begin{pmatrix} \boldsymbol{w}_1^\top \\ \boldsymbol{w}_2^\top \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \quad \boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

$\boldsymbol{W}$	$\boldsymbol{x}$	$\boldsymbol{b}$	$\boldsymbol{z}$
$(m \times d)$	$(d \times 1)$	$(m \times 1)$	$(m \times 1)$

$$(m \times d)(d \times 1) + (m \times 1) = (m \times 1)$$

> d is the number of input features; m is the number of output neurons.

Work out output 1

D DERIVATION

Use

$$\boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \boldsymbol{w}_1^\top = (1, 3), \quad b_1 = 0.$$

Work out output 1

D DERIVATION

Use

$$\boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \boldsymbol{w}_1^\top = (1, 3), \quad b_1 = 0.$$

$$z_1 = 1(2) + 3(-1) + 0 = -1.$$

Work out output 1

D DERIVATION

Use

$$\boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \boldsymbol{w}_1^\top = (1, 3), \quad b_1 = 0.$$

$$z_1 = 1(2) + 3(-1) + 0 = -1.$$

$$(1 \times 2)(2 \times 1) + (1 \times 1) = (1 \times 1)$$

Work out output 1

D DERIVATION

Use

$$\boldsymbol{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \boldsymbol{w}_1^\top = (1, 3), \quad b_1 = 0.$$

$$z_1 = 1(2) + 3(-1) + 0 = -1.$$

$$(1 \times 2)(2 \times 1) + (1 \times 1) = (1 \times 1)$$

multiply matching entries, add them, then add the bias

Work out output 2, then stack

D DERIVATION

The second neuron uses a different row:

$$\boldsymbol{w}_2^\top = (-2, 1), \quad b_2 = 1.$$

Work out output 2, then stack

D DERIVATION

The second neuron uses a different row:

$$\boldsymbol{w}_2^\top = (-2, 1), \quad b_2 = 1.$$

$$z_2 = (-2)(2) + 1(-1) + 1 = -4.$$

Work out output 2, then stack

D DERIVATION

The second neuron uses a different row:

$$\boldsymbol{w}_2^\top = (-2, 1), \quad b_2 = 1.$$

$$z_2 = (-2)(2) + 1(-1) + 1 = -4.$$

$$\boldsymbol{z} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} -1 \\ -4 \end{pmatrix}, \quad \boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}.$$

Work out output 2, then stack

D DERIVATION

The second neuron uses a different row:

$$\boldsymbol{w}_2^\top = (-2, 1), \quad b_2 = 1.$$

$$z_2 = (-2)(2) + 1(-1) + 1 = -4.$$

$$\boldsymbol{z} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} -1 \\ -4 \end{pmatrix}, \quad \boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}.$$

matrix notation only stacks the two familiar dot products

Backward starts with one arrival per output

D DERIVATION

Suppose the later loss returns

$$\boldsymbol{g}_z = \partial\mathcal{L}/\partial\boldsymbol{z} = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

Backward starts with one arrival per output

D DERIVATION

Suppose the later loss returns

$$\boldsymbol{g}_z = \partial\mathcal{L}/\partial\boldsymbol{z} = \binom{4}{-2}.$$

Read this vector as two scalar messages:

Output	Arriving gradient	Meaning
z_1	$g_1 = \partial\mathcal{L}/\partial z_1 = 4$	nudge z_1 upward; loss rises at rate 4
z_2	$g_2 = \partial\mathcal{L}/\partial z_2 = -2$	nudge z_2 upward; loss falls at rate 2

Backward starts with one arrival per output

D DERIVATION

Suppose the later loss returns

$$\boldsymbol{g}_z = \partial\mathcal{L}/\partial\boldsymbol{z} = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

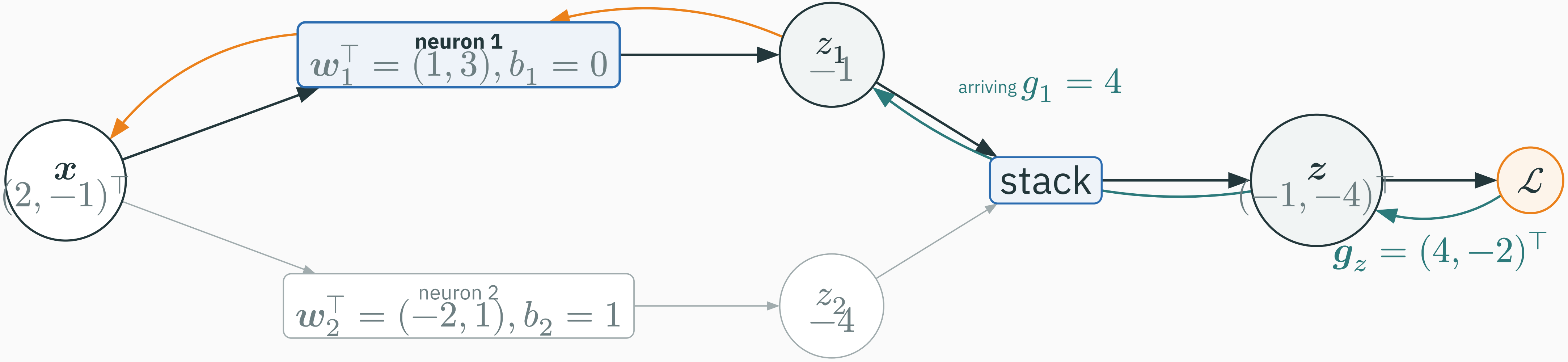
Read this vector as two scalar messages:

Output	Arriving gradient	Meaning
z_1	$g_1 = \partial\mathcal{L}/\partial z_1 = 4$	nudge z_1 upward; loss rises at rate 4
z_2	$g_2 = \partial\mathcal{L}/\partial z_2 = -2$	nudge z_2 upward; loss falls at rate 2

a vector gradient is simply one arriving number for each output coordinate

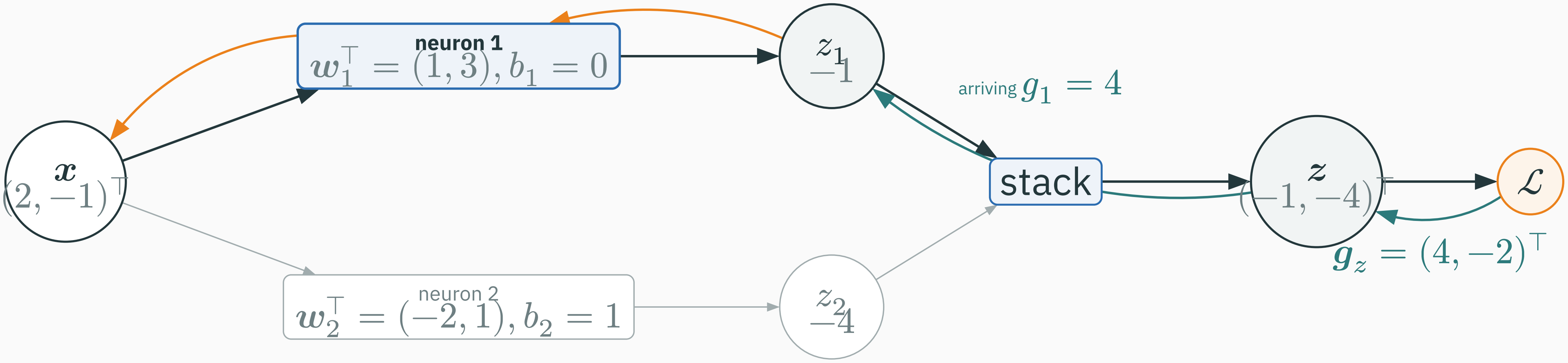
Backward through neuron 1: trace one scalar path

V VISUAL



Backward through neuron 1: trace one scalar path

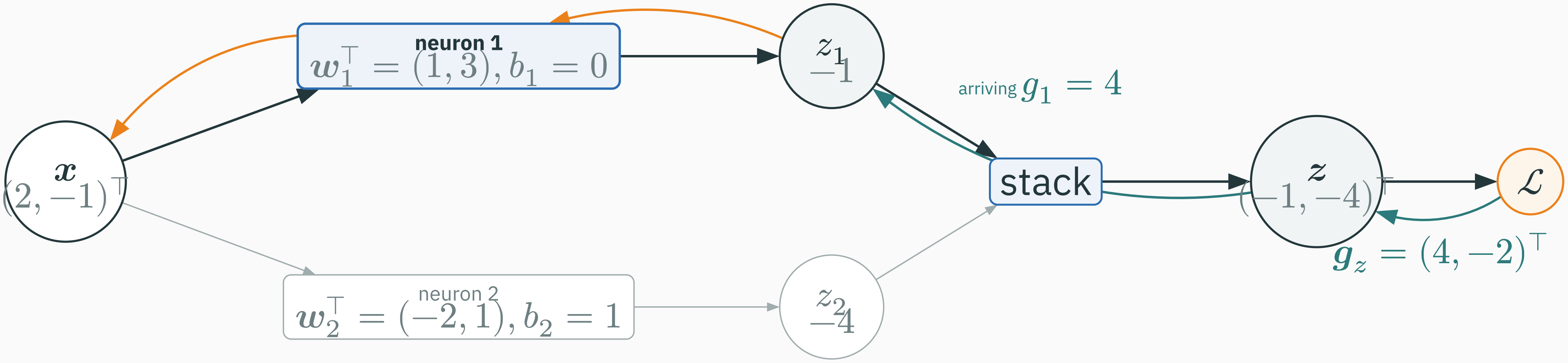
V VISUAL



$$\boldsymbol{W}_{[1,:]} = \boldsymbol{w}_1^\top = (1, 3) \quad \boldsymbol{b}_{[1]} = b_1 = 0 \quad \boldsymbol{z}_{[1]} = z_1 = -1 \quad \boldsymbol{g}_{\boldsymbol{z}[1]} = g_1 = 4$$

Backward through neuron 1: trace one scalar path

V VISUAL

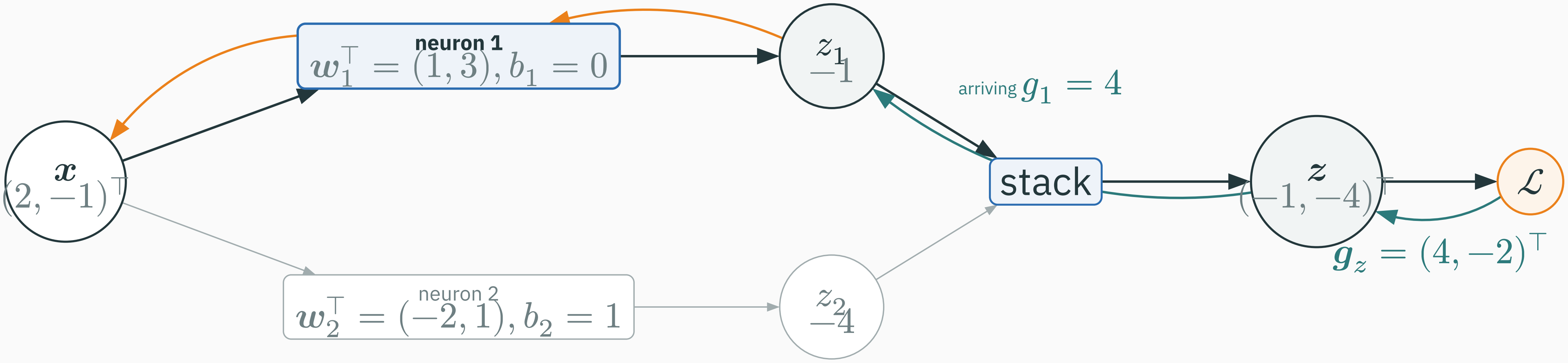


$$\boldsymbol{W}_{[1,:]} = \boldsymbol{w}_1^\top = (1,3) \quad \boldsymbol{b}_{[1]} = b_1 = 0 \quad \boldsymbol{z}_{[1]} = z_1 = -1 \quad \boldsymbol{g}_{z[1]} = g_1 = 4$$

orange path to $\boldsymbol{x}$: $g_1 \boldsymbol{w}_1 = 4(1,3)^\top = (4,12)^\top$

Backward through neuron 1: trace one scalar path

V VISUAL



$$\boldsymbol{W}_{[1,:]} = \boldsymbol{w}_1^\top = (1,3) \quad \boldsymbol{b}_{[1]} = b_1 = 0 \quad \boldsymbol{z}_{[1]} = z_1 = -1 \quad \boldsymbol{g}_{z[1]} = g_1 = 4$$

orange path to $\boldsymbol{x}$: $g_1 \boldsymbol{w}_1 = 4(1,3)^\top = (4,12)^\top$

one matrix row and one vector coordinate recover the scalar neuron we already know

Neuron 1 returns gradients to its row, bias, and input

D DERIVATION

The highlighted node receives $g_1 = 4$. Reuse the scalar affine rule:

Neuron 1 returns gradients to its row, bias, and input

D DERIVATION

The highlighted node receives $g_1 = 4$. Reuse the scalar affine rule:

Recipient	Local derivative	Gradient returned
$\boldsymbol{w}_1$	$\partial z_1/\partial \boldsymbol{w}_1 = \boldsymbol{x}$	$g_1\boldsymbol{x} = 4\begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 8 \\ -4 \end{pmatrix}$
b_1	$\partial z_1/\partial b_1 = 1$	$g_1 = 4$
$\boldsymbol{x}$	$\partial z_1/\partial \boldsymbol{x} = \boldsymbol{w}_1$	$g_1\boldsymbol{w}_1 = 4\begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 4 \\ 12 \end{pmatrix}$

Neuron 1 returns gradients to its row, bias, and input

D DERIVATION

The highlighted node receives $g_1 = 4$. Reuse the scalar affine rule:

Recipient	Local derivative	Gradient returned
$\boldsymbol{w}_1$	$\partial z_1/\partial \boldsymbol{w}_1 = \boldsymbol{x}$	$g_1\boldsymbol{x} = 4\begin{pmatrix}2\\-1\end{pmatrix} = \begin{pmatrix}8\\-4\end{pmatrix}$
b_1	$\partial z_1/\partial b_1 = 1$	$g_1 = 4$
$\boldsymbol{x}$	$\partial z_1/\partial \boldsymbol{x} = \boldsymbol{w}_1$	$g_1\boldsymbol{w}_1 = 4\begin{pmatrix}1\\3\end{pmatrix} = \begin{pmatrix}4\\12\end{pmatrix}$

neuron 1 updates only row 1 of W, but it also returns a contribution to the shared input

Backward through neuron 2

D DERIVATION

Neuron 2 receives $g_2 = -2$. Apply the same rule again:

Backward through neuron 2

D DERIVATION

Neuron 2 receives $g_2 = -2$. Apply the same rule again:

Recipient	Local derivative	Gradient returned
$\boldsymbol{w}_2$	$\partial z_2/\partial \boldsymbol{w}_2 = \boldsymbol{x}$	$g_2\boldsymbol{x} = (-2)(2,-1)^\top = (-4,2)^\top$
b_2	$\partial z_2/\partial b_2 = 1$	$g_2 = -2$
$\boldsymbol{x}$	$\partial z_2/\partial \boldsymbol{x} = \boldsymbol{w}_2$	$g_2\boldsymbol{w}_2 = (-2)(-2,1)^\top = (4,-2)^\top$

Backward through neuron 2

D DERIVATION

Neuron 2 receives $g_2 = -2$. Apply the same rule again:

Recipient	Local derivative	Gradient returned
$\boldsymbol{w}_2$	$\partial z_2/\partial \boldsymbol{w}_2 = \boldsymbol{x}$	$g_2\boldsymbol{x} = (-2)(2,-1)^\top = (-4,2)^\top$
b_2	$\partial z_2/\partial b_2 = 1$	$g_2 = -2$
$\boldsymbol{x}$	$\partial z_2/\partial \boldsymbol{x} = \boldsymbol{w}_2$	$g_2\boldsymbol{w}_2 = (-2)(-2,1)^\top = (4,-2)^\top$

neuron 2 updates row 2 and returns its own contribution to x

The shared input receives both contributions

D DERIVATION

The same x fed both neurons, so its two backward paths add:

The shared input receives both contributions

D DERIVATION

The same x fed both neurons, so its two backward paths add:

$$\boldsymbol{g}_x = \underbrace{\begin{pmatrix} 4 \\ 12 \end{pmatrix}}_{\text{from neuron 1}} + \underbrace{\begin{pmatrix} 4 \\ -2 \end{pmatrix}}_{\text{from neuron 2}} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}.$$

The shared input receives both contributions

The same x fed both neurons, so its two backward paths add:

$$\boldsymbol{g}_x = \underbrace{\begin{pmatrix} 4 \\ 12 \end{pmatrix}}_{\text{from neuron 1}} + \underbrace{\begin{pmatrix} 4 \\ -2 \end{pmatrix}}_{\text{from neuron 2}} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}.$$

$$\boldsymbol{g}_x = \boldsymbol{W}^\top \boldsymbol{g}_z$$

The shared input receives both contributions

The same x fed both neurons, so its two backward paths add:

$$\boldsymbol{g}_x = \underbrace{\begin{pmatrix} 4 \\ 12 \end{pmatrix}}_{\text{from neuron 1}} + \underbrace{\begin{pmatrix} 4 \\ -2 \end{pmatrix}}_{\text{from neuron 2}} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}.$$

$$\boldsymbol{g}_x = \boldsymbol{W}^\top \boldsymbol{g}_z$$

$$(2 \times 1) = (2 \times 2)(2 \times 1)$$

The shared input receives both contributions

D DERIVATION

The same x fed both neurons, so its two backward paths add:

$$\boldsymbol{g}_x = \underbrace{\begin{pmatrix} 4 \\ 12 \end{pmatrix}}_{\text{from neuron 1}} + \underbrace{\begin{pmatrix} 4 \\ -2 \end{pmatrix}}_{\text{from neuron 2}} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}.$$

$$\boldsymbol{g}_x = \boldsymbol{W}^\top \boldsymbol{g}_z$$

$$(2 \times 1) = (2 \times 2)(2 \times 1)$$

> The transpose lines up the two output contributions for each input coordinate; the matrix multiplication performs the required addition.

Parameter gradients keep the two rows separate

D DERIVATION

$$\boldsymbol{g}_W = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}, \quad \boldsymbol{g}_b = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

Parameter gradients keep the two rows separate

D DERIVATION

$$\boldsymbol{g}_W = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}, \quad \boldsymbol{g}_b = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

The compact rules are

$$\boldsymbol{g}_W = \boldsymbol{g}_z \, \boldsymbol{x}^\top \qquad \boldsymbol{g}_b = \boldsymbol{g}_z.$$

Parameter gradients keep the two rows separate

D DERIVATION

$$\boldsymbol{g}_W = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}, \quad \boldsymbol{g}_b = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

The compact rules are

$$\boldsymbol{g}_W = \boldsymbol{g}_z \, \boldsymbol{x}^\top \qquad \boldsymbol{g}_b = \boldsymbol{g}_z.$$

$$(2 \times 2) = (2 \times 1)(1 \times 2), \quad (2 \times 1) = (2 \times 1)$$

Parameter gradients keep the two rows separate

D DERIVATION

$$\boldsymbol{g}_W = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}, \quad \boldsymbol{g}_b = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

The compact rules are

$$\boldsymbol{g}_W = \boldsymbol{g}_z\, \boldsymbol{x}^\top \qquad \boldsymbol{g}_b = \boldsymbol{g}_z.$$

$$(2 \times 2) = (2 \times 1)(1 \times 2), \quad (2 \times 1) = (2 \times 1)$$

one output coordinate creates one row of the weight gradient

Dense backward: three rules, with shapes

V VISUAL

Returned to	Rule	Shape
$\boldsymbol{x}$	$\boldsymbol{g}_x = \boldsymbol{W}^\top \boldsymbol{g}_z$	$(d \times 1) = (d \times m)(m \times 1)$
$\boldsymbol{W}$	$\boldsymbol{g}_W = \boldsymbol{g}_z \boldsymbol{x}^\top$	$(m \times d) = (m \times 1)(1 \times d)$
$\boldsymbol{b}$	$\boldsymbol{g}_b = \boldsymbol{g}_z$	$(m \times 1) = (m \times 1)$

Dense backward: three rules, with shapes

V VISUAL

Returned to	Rule	Shape
$\boldsymbol{x}$	$\boldsymbol{g}_x = \boldsymbol{W}^\top \boldsymbol{g}_z$	$(d \times 1) = (d \times m)(m \times 1)$
$\boldsymbol{W}$	$\boldsymbol{g}_W = \boldsymbol{g}_z \, \boldsymbol{x}^\top$	$(m \times d) = (m \times 1)(1 \times d)$
$\boldsymbol{b}$	$\boldsymbol{g}_b = \boldsymbol{g}_z$	$(m \times 1) = (m \times 1)$

> Do the scalar reasoning first. Use the shapes as a check that the stacked formula has the correct orientation.

PyTorch stores gradients in matching shapes

INTERACTIVE

```python
import torch

x = torch.tensor([2., -1.], requires_grad=True)        # (2,)
W = torch.tensor([[1., 3.], [-2., 1.]],
                 requires_grad=True)                    # (2, 2)
b = torch.tensor([0., 1.], requires_grad=True)         # (2,)

z = W @ x + b                                          # (2,)
z.backward(torch.tensor([4., -2.]))                    # arriving g_z

print(x.grad)  # tensor([ 8., 10.])
print(W.grad)  # tensor([[ 8., -4.], [-4.,  2.]])
print(b.grad)  # tensor([ 4., -2.])
```

PyTorch stores gradients in matching shapes

INTERACTIVE

```
import torch

x = torch.tensor([2., -1.], requires_grad=True)       # (2,)
W = torch.tensor([[1., 3.], [-2., 1.]],
                 requires_grad=True)                    # (2, 2)
b = torch.tensor([0., 1.], requires_grad=True)        # (2,)

z = W @ x + b                                           # (2,)
z.backward(torch.tensor([4., -2.]))                     # arriving g_z

print(x.grad)  # tensor([ 8., 10.])
print(W.grad)  # tensor([[ 8., -4.], [-4.,  2.]])
print(b.grad)  # tensor([ 4., -2.])
```

every tensor and its gradient have the same shape

From one example to a batch

A batch is a stack of examples

V VISUAL

Use two examples so every calculation remains visible:

$$\boldsymbol{X} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

A batch is a stack of examples

V VISUAL

Use two examples so every calculation remains visible:

$$\boldsymbol{X} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Row	Example	Meaning
1	$\boldsymbol{x}^{(1)^\top} = (2, -1)$	first training example
2	$\boldsymbol{x}^{(2)^\top} = (-1, 2)$	second training example

A batch is a stack of examples

V VISUAL

Use two examples so every calculation remains visible:

$$\boldsymbol{X} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Row	Example	Meaning
1	$\boldsymbol{x}^{(1)\top} = (2, -1)$	first training example
2	$\boldsymbol{x}^{(2)\top} = (-1, 2)$	second training example

$$\boldsymbol{X} \in \mathbb{R}^{B \times d} = \mathbb{R}^{2 \times 2}$$

A batch is a stack of examples

V VISUAL

Use two examples so every calculation remains visible:

$$\boldsymbol{X} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Row	Example	Meaning
1	$\boldsymbol{x}^{(1)^\top} = (2, -1)$	first training example
2	$\boldsymbol{x}^{(2)^\top} = (-1, 2)$	second training example

$$\boldsymbol{X} \in \mathbb{R}^{B \times d} = \mathbb{R}^{2 \times 2}$$

the new first axis counts examples; it does not create new parameters

The same dense layer processes both rows

Keep the same parameters

$$\boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The same dense layer processes both rows

D DERIVATION

Keep the same parameters

$$\boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}^\top + \mathbf{1}_B \boldsymbol{b}^\top = \begin{pmatrix} -1 & -4 \\ 5 & 5 \end{pmatrix}.$$

The same dense layer processes both rows

D DERIVATION

Keep the same parameters

$$\boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}^\top + \mathbf{1}_B \boldsymbol{b}^\top = \begin{pmatrix} -1 & -4 \\ 5 & 5 \end{pmatrix}.$$

$$(B \times m) = (B \times d)(d \times m) + (B \times 1)(1 \times m)$$

The same dense layer processes both rows

D DERIVATION

Keep the same parameters

$$\boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}^\top + \mathbf{1}_B \boldsymbol{b}^\top = \begin{pmatrix} -1 & -4 \\ 5 & 5 \end{pmatrix}.$$

$$(B \times m) = (B \times d)(d \times m) + (B \times 1)(1 \times m)$$

row 1: $(2, -1)\boldsymbol{W}^\top + \boldsymbol{b}^\top = (-1, -4)$
row 2: $(-1, 2)\boldsymbol{W}^\top + \boldsymbol{b}^\top = (5, 5)$

The same dense layer processes both rows

D DERIVATION

Keep the same parameters

$$\boldsymbol{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}^\top + \mathbf{1}_B \boldsymbol{b}^\top = \begin{pmatrix} -1 & -4 \\ 5 & 5 \end{pmatrix}.$$

$$(B \times m) = (B \times d)(d \times m) + (B \times 1)(1 \times m)$$

row 1: $(2, -1)\boldsymbol{W}^\top + \boldsymbol{b}^\top = (-1, -4)$

row 2: $(-1, 2)\boldsymbol{W}^\top + \boldsymbol{b}^\top = (5, 5)$

each row runs the same forward calculation with the same W, b

Each example contributes one loss

D DERIVATION

Choose targets $\boldsymbol{Y} = \begin{pmatrix} -5 & -2 \\ 7 & 1 \end{pmatrix}$ and use half-squared error.

Each example contributes one loss

D DERIVATION

Choose targets $\boldsymbol{Y} = \begin{pmatrix} -5 & -2 \\ 7 & 1 \end{pmatrix}$ and use half-squared error.

Example	$\boldsymbol{z}$	$\boldsymbol{y}$	$\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{y}$	ℓ
1	$(-1, -4)$	$(-5, -2)$	$(4, -2)$	10
2	$(5, 5)$	$(7, 1)$	$(-2, 4)$	10

Each example contributes one loss

D DERIVATION

Choose targets $\boldsymbol{Y} = \begin{pmatrix} -5 & -2 \\ 7 & 1 \end{pmatrix}$ and use half-squared error.

Example	$\boldsymbol{z}$	$\boldsymbol{y}$	$\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{y}$	ℓ
1	$(-1, -4)$	$(-5, -2)$	$(4, -2)$	10
2	$(5, 5)$	$(7, 1)$	$(-2, 4)$	10

$$\ell_n = \tfrac{1}{2}\|\boldsymbol{r}^n\|^2, \quad \partial \ell_n / \partial \boldsymbol{z}^n = \boldsymbol{r}^n.$$

Each example contributes one loss

D DERIVATION

Choose targets $\boldsymbol{Y} = \begin{pmatrix} -5 & -2 \\ 7 & 1 \end{pmatrix}$ and use half-squared error.

Example	$\boldsymbol{z}$	$\boldsymbol{y}$	$\boldsymbol{r} = \boldsymbol{z} - \boldsymbol{y}$	ℓ
1	$(-1, -4)$	$(-5, -2)$	$(4, -2)$	10
2	$(5, 5)$	$(7, 1)$	$(-2, 4)$	10

$$\ell_n = \tfrac{1}{2}\|\boldsymbol{r}^n\|^2, \quad \partial \ell_n / \partial \boldsymbol{z}^n = \boldsymbol{r}^n.$$

for squared error, the residual is the gradient returned by that example's loss

The mean loss scales both arriving gradients

D DERIVATION

$$\mathcal{L} = \frac{1}{2}(\ell_1 + \ell_2) = 10.$$

The mean loss scales both arriving gradients

D DERIVATION

$$\mathcal{L} = \tfrac{1}{2}(\ell_1 + \ell_2) = 10.$$

Therefore

$$\boldsymbol{G}_Z = \partial \mathcal{L}/\partial \boldsymbol{Z} = \frac{1}{2}\begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

The mean loss scales both arriving gradients

D DERIVATION

$$\mathcal{L} = \tfrac{1}{2}(\ell_1 + \ell_2) = 10.$$

Therefore

$$\boldsymbol{G}_Z = \partial\mathcal{L}/\partial\boldsymbol{Z} = \frac{1}{2}\begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

$$\boldsymbol{G}_Z \in \mathbb{R}^{B\times m} = \mathbb{R}^{2\times 2}$$

The mean loss scales both arriving gradients

D DERIVATION

$$\mathcal{L} = \tfrac{1}{2}(\ell_1 + \ell_2) = 10.$$

Therefore

$$\boldsymbol{G}_Z = \partial\mathcal{L}/\partial\boldsymbol{Z} = \tfrac{1}{2}\begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

$$\boldsymbol{G}_Z \in \mathbb{R}^{B\times m} = \mathbb{R}^{2\times 2}$$

> Rows still correspond to examples; columns still correspond to output neurons. The factor $\frac{1}{B}$ comes only from taking the mean.

Example 1 proposes a parameter gradient

D DERIVATION

Before averaging, example 1 has $\boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$ and $\boldsymbol{x}^1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$.

Example 1 proposes a parameter gradient

D DERIVATION

Before averaging, example 1 has $\boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$ and $\boldsymbol{x}^1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$.

$$\boldsymbol{G}_W^1 = \boldsymbol{r}^1 (\boldsymbol{x}^1)^\top = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}.$$

Example 1 proposes a parameter gradient

D DERIVATION

Before averaging, example 1 has $\boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$ and $\boldsymbol{x}^1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$.

$$\boldsymbol{G}_W^1 = \boldsymbol{r}^1 (\boldsymbol{x}^1)^\top = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}.$$

$$(m \times d) = (m \times 1)(1 \times d)$$

Example 1 proposes a parameter gradient

D DERIVATION

Before averaging, example 1 has $\boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$ and $\boldsymbol{x}^1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$.

$$\boldsymbol{G}_W^1 = \boldsymbol{r}^1 (\boldsymbol{x}^1)^\top = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}.$$

$$(m \times d) = (m \times 1)(1 \times d)$$

$$\boldsymbol{g}_b^1 = \boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

Example 1 proposes a parameter gradient

D DERIVATION

Before averaging, example 1 has $\boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$ and $\boldsymbol{x}^1 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$.

$$\boldsymbol{G}_W^1 = \boldsymbol{r}^1 (\boldsymbol{x}^1)^\top = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix}.$$

$$(m \times d) = (m \times 1)(1 \times d)$$

$$\boldsymbol{g}_b^1 = \boldsymbol{r}^1 = \begin{pmatrix} 4 \\ -2 \end{pmatrix}.$$

this is the same single-example outer product derived on the previous slides

Example 2 proposes another gradient

D DERIVATION

Example 2 has $\boldsymbol{r}^2 = \binom{-2}{4}$ and $\boldsymbol{x}^2 = \binom{-1}{2}$.

Example 2 proposes another gradient

D DERIVATION

Example 2 has $\boldsymbol{r}^2 = \binom{-2}{4}$ and $\boldsymbol{x}^2 = \binom{-1}{2}$.

$$\boldsymbol{G}_W^2 = \boldsymbol{r}^2 (\boldsymbol{x}^2)^\top = \begin{pmatrix} 2 & -4 \\ -4 & 8 \end{pmatrix}.$$

Example 2 proposes another gradient

D DERIVATION

Example 2 has $\boldsymbol{r}^2 = \begin{pmatrix} -2 \\ 4 \end{pmatrix}$ and $\boldsymbol{x}^2 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$.

$$\boldsymbol{G}_W^2 = \boldsymbol{r}^2 (\boldsymbol{x}^2)^\top = \begin{pmatrix} 2 & -4 \\ -4 & 8 \end{pmatrix}.$$

Both examples used the same parameters, so their paths add; the mean divides by 2:

$$\boldsymbol{g}_W = \tfrac{1}{2}(\boldsymbol{G}_W^1 + \boldsymbol{G}_W^2) = \begin{pmatrix} 5 & -4 \\ -4 & 5 \end{pmatrix}.$$

Example 2 proposes another gradient

Example 2 has $\boldsymbol{r}^2 = \begin{pmatrix} -2 \\ 4 \end{pmatrix}$ and $\boldsymbol{x}^2 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$.

$$\boldsymbol{G}_W^2 = \boldsymbol{r}^2 (\boldsymbol{x}^2)^\top = \begin{pmatrix} 2 & -4 \\ -4 & 8 \end{pmatrix}.$$

Both examples used the same parameters, so their paths add; the mean divides by 2:

$$\boldsymbol{g}_W = \tfrac{1}{2}(\boldsymbol{G}_W^1 + \boldsymbol{G}_W^2) = \begin{pmatrix} 5 & -4 \\ -4 & 5 \end{pmatrix}.$$

$$\boldsymbol{g}_b = \tfrac{1}{2}\left(\begin{pmatrix} 4 \\ -2 \end{pmatrix} + \begin{pmatrix} -2 \\ 4 \end{pmatrix}\right) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Batch matrix multiplication performs the same sum

D DERIVATION

Let $\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$. Then

$$\boldsymbol{g}_W = \boldsymbol{G}_Z^\top \boldsymbol{X}.$$

Batch matrix multiplication performs the same sum

D DERIVATION

Let $\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$. Then

$$\boldsymbol{g}_W = \boldsymbol{G}_Z^\top \boldsymbol{X}.$$

$$(m \times d) = (m \times B)(B \times d)$$

Batch matrix multiplication performs the same sum

D DERIVATION

Let $\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$. Then

$$\boldsymbol{g}_W = \boldsymbol{G}_Z^\top \boldsymbol{X}.$$

$$(m \times d) = (m \times B)(B \times d)$$

$$\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 5 & -4 \\ -4 & 5 \end{pmatrix}.$$

Batch matrix multiplication performs the same sum

D DERIVATION

Let $\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$. Then

$$\boldsymbol{g}_W = \boldsymbol{G}_Z^\top \boldsymbol{X}.$$

$$(m \times d) = (m \times B)(B \times d)$$

$$\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 5 & -4 \\ -4 & 5 \end{pmatrix}.$$

> The middle dimension B disappears because matrix multiplication sums the contribution from every example.

Batch gradients have one simple shape story

VISUAL

Returned to	Rule	Shape
$\boldsymbol{W}$	$g_W = \boldsymbol{G}_Z^\top \boldsymbol{X}$	$(m \times d) = (m \times B)(B \times d)$
$\boldsymbol{b}$	$g_b = \boldsymbol{G}_Z^\top \mathbf{1}_B$	$(m \times 1) = (m \times B)(B \times 1)$
$\boldsymbol{X}$	$g_X = \boldsymbol{G}_Z \boldsymbol{W}$	$(B \times d) = (B \times m)(m \times d)$

Batch gradients have one simple shape story

VISUAL

Returned to	Rule	Shape
$\boldsymbol{W}$	$g_W = \boldsymbol{G}_Z^\top \boldsymbol{X}$	$(m \times d) = (m \times B)(B \times d)$
$\boldsymbol{b}$	$g_b = \boldsymbol{G}_Z^\top \mathbf{1}_B$	$(m \times 1) = (m \times B)(B \times 1)$
$\boldsymbol{X}$	$g_X = \boldsymbol{G}_Z \boldsymbol{W}$	$(B \times d) = (B \times m)(m \times d)$

$$g_X = \begin{pmatrix} 4 & 5 \\ -5 & -1 \end{pmatrix}.$$

Batch gradients have one simple shape story

VISUAL

Returned to	Rule	Shape
$\boldsymbol{W}$	$g_W = \boldsymbol{G}_Z^\top \boldsymbol{X}$	$(m \times d) = (m \times B)(B \times d)$
$\boldsymbol{b}$	$g_b = \boldsymbol{G}_Z^\top \mathbf{1}_B$	$(m \times 1) = (m \times B)(B \times 1)$
$\boldsymbol{X}$	$g_X = \boldsymbol{G}_Z \boldsymbol{W}$	$(B \times d) = (B \times m)(m \times d)$

$$g_X = \begin{pmatrix} 4 & 5 \\ -5 & -1 \end{pmatrix}.$$

shared parameter gradients add across rows; input gradients remain one row per example

Sum and mean differ only by scale

D DERIVATION

Reduction	Arriving gradient	Weight gradient
sum	$\boldsymbol{G}_Z = \boldsymbol{R}$	$g_W = \boldsymbol{R}^\top \boldsymbol{X}$
mean	$\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$	$g_W = \boldsymbol{R}^\top \frac{\boldsymbol{X}}{B}$

Sum and mean differ only by scale

D DERIVATION

Reduction	Arriving gradient	Weight gradient
sum	$G_Z = \boldsymbol{R}$	$g_W = \boldsymbol{R}^\top \boldsymbol{X}$
mean	$G_Z = \frac{\boldsymbol{R}}{B}$	$g_W = \boldsymbol{R}^\top \frac{\boldsymbol{X}}{B}$

```
loss = 0.5 * ((Z - Y)**2).sum(dim=1).mean()
# sum output coordinates inside each example; mean over examples
```

Sum and mean differ only by scale

D DERIVATION

Reduction	Arriving gradient	Weight gradient
sum	$\boldsymbol{G}_Z = \boldsymbol{R}$	$g_W = \boldsymbol{R}^\top \boldsymbol{X}$
mean	$\boldsymbol{G}_Z = \frac{\boldsymbol{R}}{B}$	$g_W = \boldsymbol{R}^\top \frac{\boldsymbol{X}}{B}$

```
loss = 0.5 * ((Z - Y)**2).sum(dim=1).mean()
# sum output coordinates inside each example; mean over examples
```

the dense-layer rule is unchanged; the final scalar reduction chooses the scale

PyTorch reproduces the two-example calculation

INTERACTIVE

```
X = torch.tensor([[ 2., -1.], [-1., 2.]], requires_grad=True)  # (B=2, d=2)
Y = torch.tensor([[-5., -2.], [ 7., 1.]])                     # (2, 2)
W = torch.tensor([[1., 3.], [-2., 1.]], requires_grad=True)   # (m=2, d=2)
b = torch.tensor([0., 1.], requires_grad=True)                # (2,)

Z = X @ W.T + b                                               # (B, m)
loss = 0.5 * ((Z - Y)**2).sum(dim=1).mean()                   # scalar
loss.backward()

print(loss.item())  # 10.0
print(W.grad)        # [[ 5., -4.], [-4., 5.]]
print(b.grad)        # [1., 1.]
print(X.grad)        # [[ 4., 5.], [-5., -1.]]
```

PyTorch reproduces the two-example calculation

I INTERACTIVE

```
X = torch.tensor([[ 2., -1.], [-1., 2.]], requires_grad=True)  # (B=2, d=2)
Y = torch.tensor([[-5., -2.], [ 7., 1.]])                      # (2, 2)
W = torch.tensor([[1., 3.], [-2., 1.]], requires_grad=True)    # (m=2, d=2)
b = torch.tensor([0., 1.], requires_grad=True)                 # (2,)

Z = X @ W.T + b                                                # (B, m)
loss = 0.5 * ((Z - Y)**2).sum(dim=1).mean()                    # scalar
loss.backward()

print(loss.item())  # 10.0
print(W.grad)        # [[ 5., -4.], [-4., 5.]]
print(b.grad)        # [1., 1.]
print(X.grad)        # [[ 4., 5.], [-5., -1.]]
```

`backward()` carries out the same rowwise contributions and additions

The training loop repeats this batch calculation

INTERACTIVE

```
for X, Y in loader:
    optimizer.zero_grad()       # discard gradients from the previous update
    Z = model(X)                # batch forward: (B, d) -> (B, m)
    loss = loss_fn(Z, Y)        # reduce B examples to one scalar
    loss.backward()             # add this batch's parameter contributions
    optimizer.step()            # update parameters once
```

The training loop repeats this batch calculation

I INTERACTIVE

```
for X, Y in loader:
    optimizer.zero_grad()       # discard gradients from the previous update
    Z = model(X)                # batch forward: (B, d) -> (B, m)
    loss = loss_fn(Z, Y)        # reduce B examples to one scalar
    loss.backward()             # add this batch's parameter contributions
    optimizer.step()            # update parameters once
```

clear → forward → scalar loss → backward → update

The training loop repeats this batch calculation

INTERACTIVE

```
for X, Y in loader:
    optimizer.zero_grad()       # discard gradients from the previous update
    Z = model(X)                # batch forward: (B, d) -> (B, m)
    loss = loss_fn(Z, Y)        # reduce B examples to one scalar
    loss.backward()             # add this batch's parameter contributions
    optimizer.step()            # update parameters once
```

clear $\rightarrow$ forward $\rightarrow$ scalar loss $\rightarrow$ backward $\rightarrow$ update

> `step()` uses the gradients; it does not compute or clear them. That is why the next iteration begins with `zero_grad()`.

Microbatches split one batch without changing the goal

D DERIVATION

```
optimizer.zero_grad()
(loss_example_1 / 2).backward()   # first half of the mean gradient
(loss_example_2 / 2).backward()   # second half accumulates
optimizer.step()
```

Microbatches split one batch without changing the goal

D DERIVATION

```
optimizer.zero_grad()
(loss_example_1 / 2).backward()   # first half of the mean gradient
(loss_example_2 / 2).backward()   # second half accumulates
optimizer.step()
```

$$\boldsymbol{W}.\mathrm{grad}: \quad \frac{G_W^1}{2} \to \frac{G_W^1 + G_W^2}{2}$$

Microbatches split one batch without changing the goal

D DERIVATION

```
optimizer.zero_grad()
(loss_example_1 / 2).backward()   # first half of the mean gradient
(loss_example_2 / 2).backward()   # second half accumulates
optimizer.step()
```

$$\boldsymbol{W}.\mathrm{grad}: \quad \frac{\boldsymbol{G}_W^1}{2} \to \frac{\boldsymbol{G}_W^1 + \boldsymbol{G}_W^2}{2}$$

> For equivalence, keep parameters fixed, preserve the same overall scaling, and do not call `zero_grad()` or `step()` between pieces.

Microbatches split one batch without changing the goal

D DERIVATION

```
optimizer.zero_grad()
(loss_example_1 / 2).backward()   # first half of the mean gradient
(loss_example_2 / 2).backward()   # second half accumulates
optimizer.step()
```

$$\boldsymbol{W}.\text{grad}: \quad \frac{G_W^1}{2} \rightarrow \frac{G_W^1 + G_W^2}{2}$$

> For equivalence, keep parameters fixed, preserve the same overall scaling, and do not call `zero_grad()` or `step()` between pieces.

one update window may contain one full batch or several correctly scaled pieces

A tiny neural network

ReLU is what prevents two dense layers from collapsing into one

V VISUAL

Without a nonlinearity, two affine maps are still one affine map:

$$\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2$$

$$= (\boldsymbol{W}_2\boldsymbol{W}_1)\boldsymbol{x} + (\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2).$$

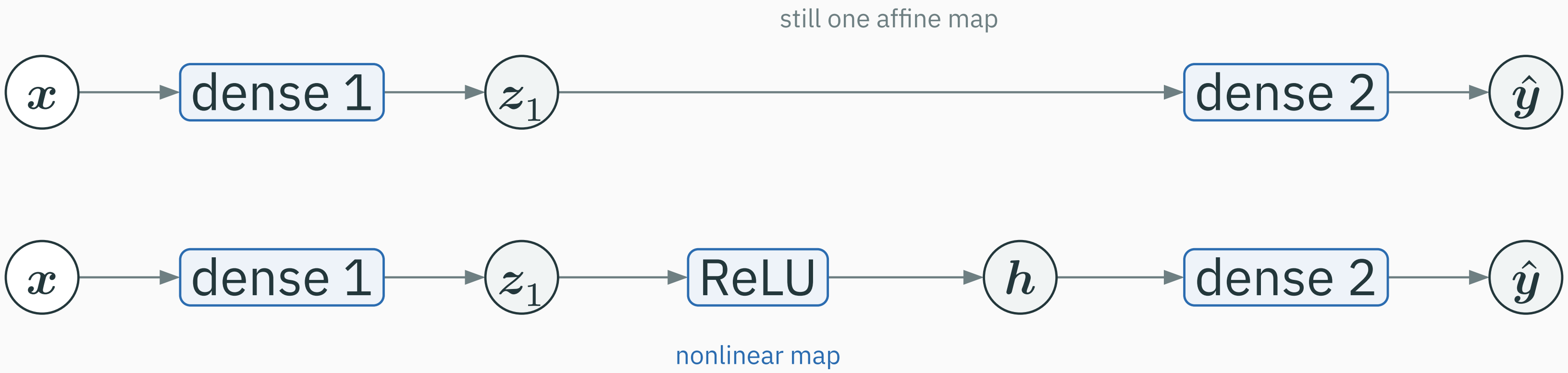
ReLU is what prevents two dense layers from collapsing into one

V VISUAL

Without a nonlinearity, two affine maps are still one affine map:

$$\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2$$

$$= (\boldsymbol{W}_2\boldsymbol{W}_1)\boldsymbol{x} + (\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2).$$



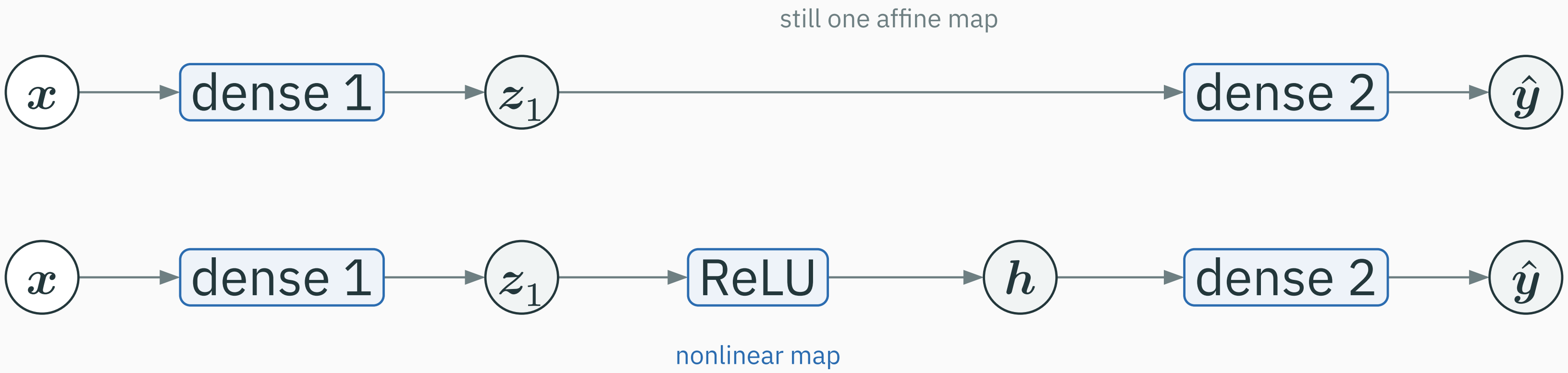
ReLU is what prevents two dense layers from collapsing into one

V VISUAL

Without a nonlinearity, two affine maps are still one affine map:

$$\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2$$

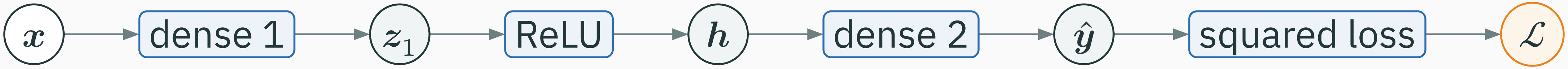
$$= (\boldsymbol{W}_2\boldsymbol{W}_1)\boldsymbol{x} + (\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2).$$



ReLU makes the composition nonlinear; depth can now represent a richer function

A tiny MLP composes three vector blocks

V VISUAL



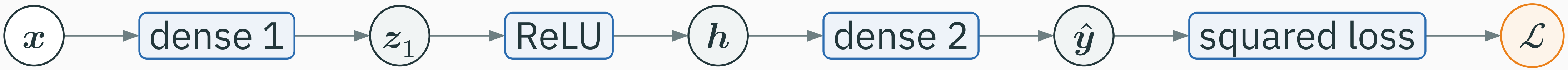
A tiny MLP composes three vector blocks

V VISUAL

$\boldsymbol{x} \rightarrow$ dense 1 $\rightarrow \boldsymbol{z}_1 \rightarrow$ ReLU $\rightarrow \boldsymbol{h} \rightarrow$ dense 2 $\rightarrow \hat{\boldsymbol{y}} \rightarrow$ squared loss $\rightarrow \mathcal{L}$

Value	Rule	Shape
$\boldsymbol{z}_1$	$\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$	$(h \times d)(d \times 1) + (h \times 1) = h \times 1$
$\boldsymbol{h}$	$\mathrm{ReLU}(\boldsymbol{z}_1)$	$h \times 1$
$\hat{\boldsymbol{y}}$	$\boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2$	$(m \times h)(h \times 1) + (m \times 1) = m \times 1$
$\mathcal{L}$	$\sum_i (\hat{y}_i - y_i)^2$	scalar

A tiny MLP composes three vector blocks



Value	Rule	Shape
$\boldsymbol{z}_1$	$\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$	$(h \times d)(d \times 1) + (h \times 1) = h \times 1$
$\boldsymbol{h}$	$\mathrm{ReLU}(\boldsymbol{z}_1)$	$h \times 1$
$\hat{\boldsymbol{y}}$	$\boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2$	$(m \times h)(h \times 1) + (m \times 1) = m \times 1$
$\mathcal{L}$	$\sum_i (\hat{y}_i - y_i)^2$	scalar

d **input features → h hidden features → m output coordinates → one loss**

MLP forward, one block at a time

D DERIVATION

$$\boldsymbol{z}_1 = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^{h \times 1}$$

MLP forward, one block at a time

D DERIVATION

$$\boldsymbol{z}_1 = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^{h \times 1}$$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}_1) \in \mathbb{R}^{h \times 1}$$

MLP forward, one block at a time

D DERIVATION

$$\boldsymbol{z}_1 = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^{h\times 1}$$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}_1) \in \mathbb{R}^{h\times 1}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2 \in \mathbb{R}^{m\times 1}$$

MLP forward, one block at a time

D DERIVATION

$$\boldsymbol{z}_1 = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^{h \times 1}$$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}_1) \in \mathbb{R}^{h \times 1}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{W}_2 \boldsymbol{h} + \boldsymbol{b}_2 \in \mathbb{R}^{m \times 1}$$

$$\mathcal{L} = \sum_i (\hat{y}_i - y_i)^2 \in \mathbb{R}$$

MLP forward, one block at a time

D DERIVATION

$$\boldsymbol{z}_1 = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^{h \times 1}$$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}_1) \in \mathbb{R}^{h \times 1}$$

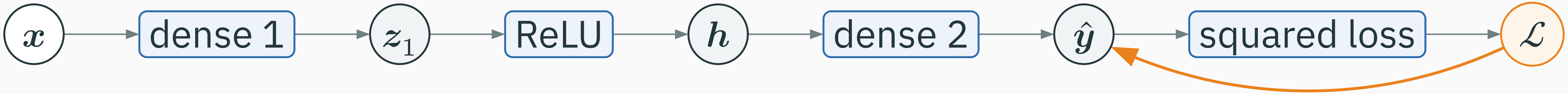
$$\hat{\boldsymbol{y}} = \boldsymbol{W}_2 \boldsymbol{h} + \boldsymbol{b}_2 \in \mathbb{R}^{m \times 1}$$

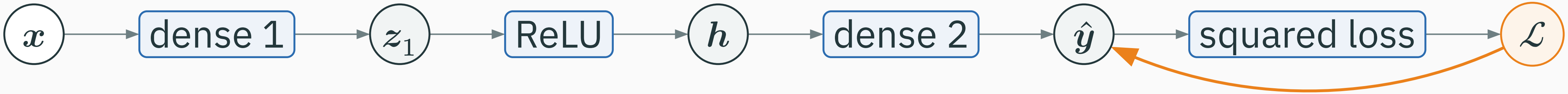
$$\mathcal{L} = \sum_i (\hat{y}_i - y_i)^2 \in \mathbb{R}$$

forward changes the representation shape; the loss finally collapses it to one number

Backward, part 1: loss → predictions

D DERIVATION





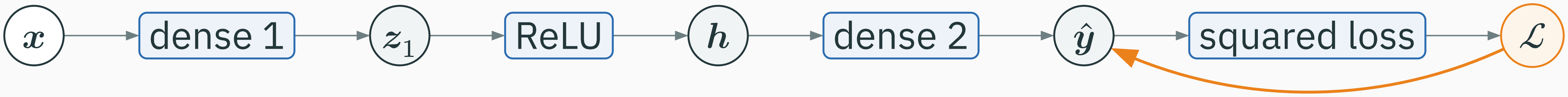
1. Squared loss. Start with the seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$. The loss operation returns

$$\boldsymbol{g}_{\hat{y}} = 2(\hat{\boldsymbol{y}} - \boldsymbol{y}).$$

$$\boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}$$

Backward, part 1: loss → predictions

D DERIVATION



1. Squared loss. Start with the seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$. The loss operation returns

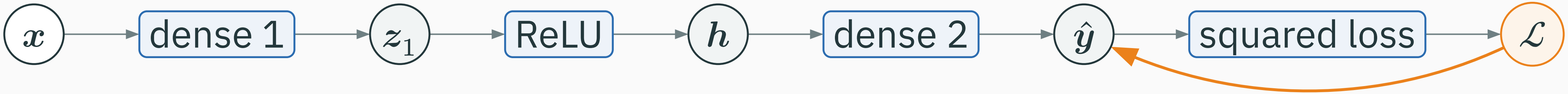
$$\boldsymbol{g}_{\hat{y}} = 2(\hat{\boldsymbol{y}} - \boldsymbol{y}).$$

$$\boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}$$

> There is one returned number for each prediction coordinate: positive means increasing that prediction increases the loss; negative means it decreases the loss.

Backward, part 1: loss → predictions

D DERIVATION



1. Squared loss. Start with the seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$. The loss operation returns

$$\boldsymbol{g}_{\hat{y}} = 2(\hat{\boldsymbol{y}} - \boldsymbol{y}).$$

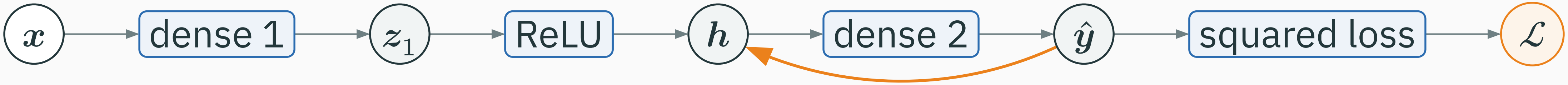
$$\boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}$$

> There is one returned number for each prediction coordinate: positive means increasing that prediction increases the loss; negative means it decreases the loss.

$\boldsymbol{g}_{\hat{y}}$ now arrives at the second dense layer

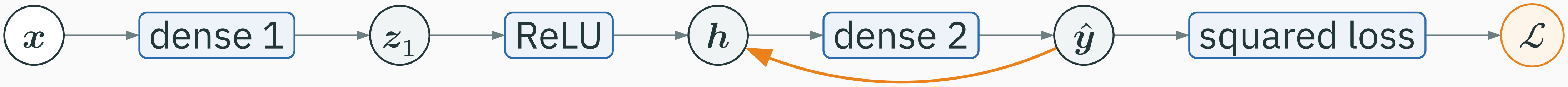
Backward, part 2: dense 2 → hidden activations

D DERIVATION



Backward, part 2: dense 2 → hidden activations

D DERIVATION



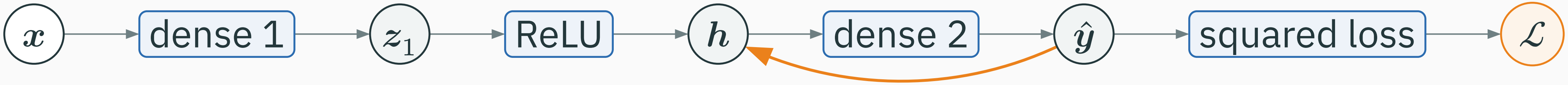
2. Second dense layer. It receives $\boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}$ and returns a gradient to its input $\boldsymbol{h}$:

$$\boldsymbol{g}_h = \boldsymbol{W}_2^\top \, \boldsymbol{g}_{\hat{y}}.$$

$$(h \times 1) = (h \times m)(m \times 1)$$

Backward, part 2: dense 2 → hidden activations

D DERIVATION



2. Second dense layer. It receives $\boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}$ and returns a gradient to its input $\boldsymbol{h}$:

$$\boldsymbol{g}_h = \boldsymbol{W}_2^\top \boldsymbol{g}_{\hat{y}}.$$

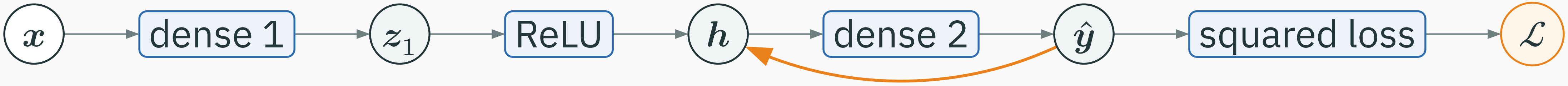
$$(h \times 1) = (h \times m)(m \times 1)$$

It also returns parameter gradients

$$\partial\mathcal{L}/\partial\boldsymbol{W}_2 = \boldsymbol{g}_{\hat{y}}\boldsymbol{h}^\top \in \mathbb{R}^{m\times h}, \quad \partial\mathcal{L}/\partial\boldsymbol{b}_2 = \boldsymbol{g}_{\hat{y}} \in \mathbb{R}^{m\times 1}.$$

Backward, part 2: dense 2 → hidden activations

D DERIVATION



2. Second dense layer. It receives $g_{\hat{y}} \in \mathbb{R}^{m \times 1}$ and returns a gradient to its input h:

$$g_h = W_2^\top g_{\hat{y}}.$$

$$(h \times 1) = (h \times m)(m \times 1)$$

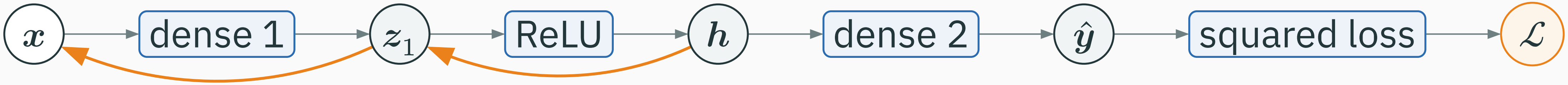
It also returns parameter gradients

$$\partial \mathcal{L} / \partial W_2 = g_{\hat{y}} h^\top \in \mathbb{R}^{m \times h}, \quad \partial \mathcal{L} / \partial b_2 = g_{\hat{y}} \in \mathbb{R}^{m \times 1}.$$

the returned g_h is now the arriving gradient at ReLU

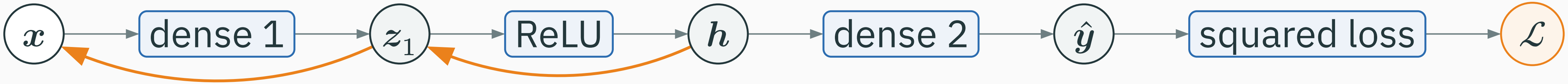
Backward, part 3: ReLU → dense 1 → input

D DERIVATION



Backward, part 3: ReLU → dense 1 → input

D DERIVATION



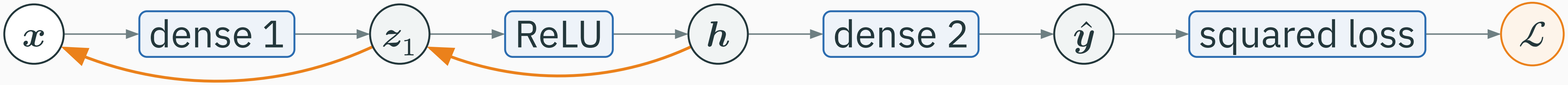
3. ReLU. Apply its local gate coordinate by coordinate:

$$\boldsymbol{g}_{z_1} = \boldsymbol{g}_h \odot \mathbb{1}[\boldsymbol{z}_1 > 0].$$

$$(h \times 1) = (h \times 1) \odot (h \times 1)$$

Backward, part 3: ReLU → dense 1 → input

D DERIVATION



3. ReLU. Apply its local gate coordinate by coordinate:

$$\boldsymbol{g}_{z_1} = \boldsymbol{g}_h \odot \mathbb{1}[\boldsymbol{z}_1 > 0].$$

$$(h \times 1) = (h \times 1) \odot (h \times 1)$$

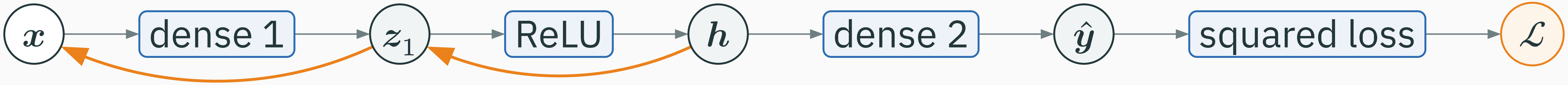
4. First dense layer. It receives $\boldsymbol{g}_{z_1}$ and returns

$$\boldsymbol{g}_x = \boldsymbol{W}_1^\top \boldsymbol{g}_{z_1}, \qquad \partial\mathcal{L}/\partial\boldsymbol{W}_1 = \boldsymbol{g}_{z_1} \boldsymbol{x}^\top, \qquad \partial\mathcal{L}/\partial\boldsymbol{b}_1 = \boldsymbol{g}_{z_1}.$$

$$\boldsymbol{g}_x : d \times 1 \quad \cdot \quad \partial\mathcal{L}/\partial\boldsymbol{W}_1 : h \times d \quad \cdot \quad \partial\mathcal{L}/\partial\boldsymbol{b}_1 : h \times 1$$

Backward, part 3: ReLU → dense 1 → input

D DERIVATION



3. ReLU. Apply its local gate coordinate by coordinate:

$$\boldsymbol{g}_{z_1} = \boldsymbol{g}_h \odot \mathbb{1}[\boldsymbol{z}_1 > 0].$$

$$(h \times 1) = (h \times 1) \odot (h \times 1)$$

4. First dense layer. It receives $\boldsymbol{g}_{z_1}$ and returns

$$\boldsymbol{g}_x = \boldsymbol{W}_1^\top \boldsymbol{g}_{z_1}, \qquad \partial\mathcal{L}/\partial\boldsymbol{W}_1 = \boldsymbol{g}_{z_1}\boldsymbol{x}^\top, \qquad \partial\mathcal{L}/\partial\boldsymbol{b}_1 = \boldsymbol{g}_{z_1}.$$

$$\boldsymbol{g}_x : d \times 1 \quad \cdot \quad \partial\mathcal{L}/\partial\boldsymbol{W}_1 : h \times d \quad \cdot \quad \partial\mathcal{L}/\partial\boldsymbol{b}_1 : h \times 1$$

each returned gradient becomes the arriving gradient for the block immediately to its left

The same tiny MLP in PyTorch

INTERACTIVE

```python
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(2, 3), nn.ReLU(), nn.Linear(3, 2)
)
x = torch.tensor([2., -1.])
y = torch.tensor([1., 0.])

y_hat = model(x)
loss = ((y_hat - y)**2).sum()
loss.backward()

for name, p in model.named_parameters():
    print(name, tuple(p.shape), tuple(p.grad.shape))
```

The same tiny MLP in PyTorch

I INTERACTIVE

```python
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(2, 3), nn.ReLU(), nn.Linear(3, 2)
)
x = torch.tensor([2., -1.])
y = torch.tensor([1., 0.])

y_hat = model(x)
loss = ((y_hat - y)**2).sum()
loss.backward()

for name, p in model.named_parameters():
    print(name, tuple(p.shape), tuple(p.grad.shape))
```

PyTorch records the three blocks and applies their local rules in reverse

Interactive: follow one vector gradient

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Step through the forward values, then follow one arriving gradient through a dense operation, a ReLU, and the preceding dense operation.

Interactive: follow one vector gradient

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Step through the forward values, then follow one arriving gradient through a dense operation, a ReLU, and the preceding dense operation.

For practice, predict each gradient's shape before revealing its value in the interactive.

When gradients cross many layers

Deep networks repeat the same multiplication

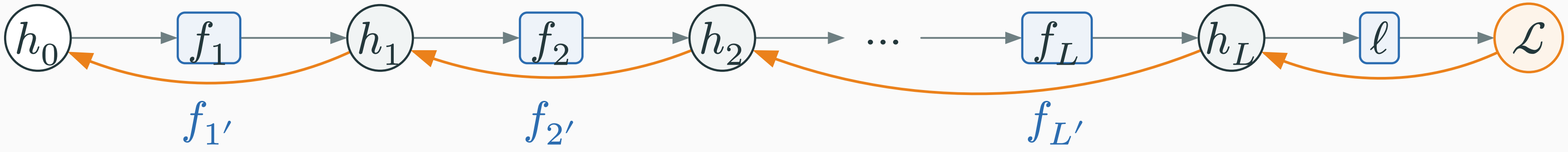
D DERIVATION

For a scalar chain $h_0 \to h_1 \to \ldots \to h_L \to \mathcal{L}$:

Deep networks repeat the same multiplication

D DERIVATION

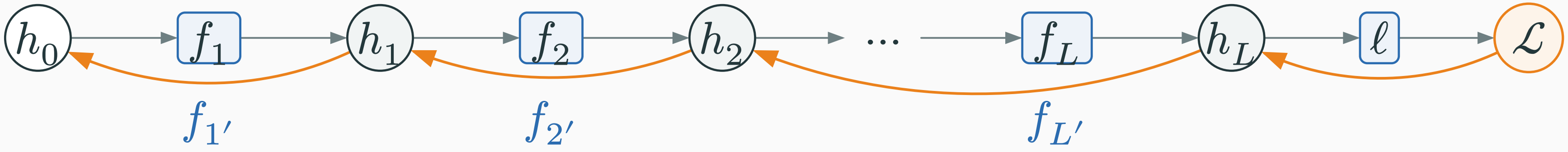
For a scalar chain $h_0 \to h_1 \to \ldots \to h_L \to \mathcal{L}$:



Deep networks repeat the same multiplication

D DERIVATION

For a scalar chain $h_0 \to h_1 \to \dots \to h_L \to \mathcal{L}$:

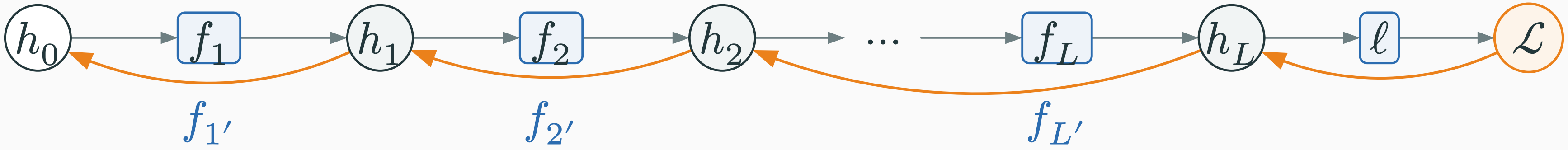


$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot f_{L'}(h_{L-1}) \dots f_{1'}(h_0).$$

Deep networks repeat the same multiplication

D DERIVATION

For a scalar chain $h_0 \to h_1 \to \ldots \to h_L \to \mathcal{L}$:



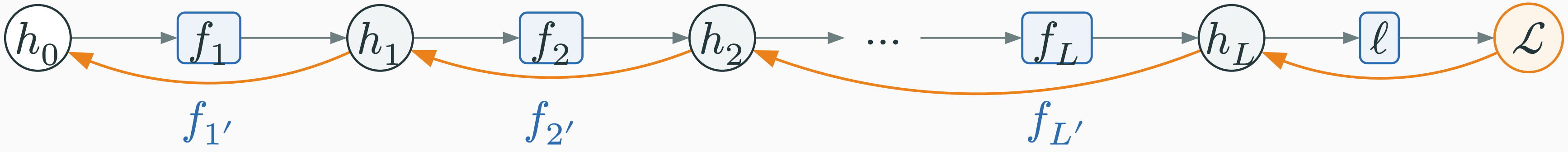
$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot f_{L'}(h_{L-1}) \ldots f_{1'}(h_0).$$

depth creates a product of many local slopes

Deep networks repeat the same multiplication

D DERIVATION

For a scalar chain $h_0 \to h_1 \to \ldots \to h_L \to \mathcal{L}$:



$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot f_{L'}(h_{L-1}) \ldots f_{1'}(h_0).$$

depth creates a product of many local slopes

> Vector layers do the same operation with matrix–vector products. The intuition is unchanged: repeatedly small factors shrink a gradient; repeatedly large factors grow it.

In a vector layer, the returned shape matches the previous value

D DERIVATION

For layer ℓ,

$$\boldsymbol{z}_\ell = \boldsymbol{W}_\ell \boldsymbol{h}_{\ell-1} + \boldsymbol{b}_\ell.$$

In a vector layer, the returned shape matches the previous value

D DERIVATION

For layer ℓ,

$$\boldsymbol{z}_\ell = \boldsymbol{W}_\ell \boldsymbol{h}_{\ell-1} + \boldsymbol{b}_\ell.$$

Quantity	Shape	Backward role
$\boldsymbol{g}_{z_\ell}$	$n_\ell \times 1$	arrives from the layer to the right
$\boldsymbol{W}_\ell^\top$	$n_{\ell-1} \times n_\ell$	local linear map
$\boldsymbol{g}_{h_{\ell-1}}$	$n_{\ell-1} \times 1$	returns to the previous layer

In a vector layer, the returned shape matches the previous value

D DERIVATION

For layer ℓ,

$$\boldsymbol{z}_\ell = \boldsymbol{W}_\ell \boldsymbol{h}_{\ell-1} + \boldsymbol{b}_\ell.$$

Quantity	Shape	Backward role
$\boldsymbol{g}_{z_\ell}$	$n_\ell \times 1$	arrives from the layer to the right
$\boldsymbol{W}_\ell^\top$	$n_{\ell-1} \times n_\ell$	local linear map
$\boldsymbol{g}_{h_{\ell-1}}$	$n_{\ell-1} \times 1$	returns to the previous layer

$$\boldsymbol{g}_{h_{\ell-1}} = \boldsymbol{W}_\ell^\top \boldsymbol{g}_{z_\ell}.$$

In a vector layer, the returned shape matches the previous value

D DERIVATION

For layer ℓ,

$$\boldsymbol{z}_\ell = \boldsymbol{W}_\ell \boldsymbol{h}_{\ell-1} + \boldsymbol{b}_\ell.$$

Quantity	Shape	Backward role
$\boldsymbol{g}_{z_\ell}$	$n_\ell \times 1$	arrives from the layer to the right
$\boldsymbol{W}_\ell^\top$	$n_{\ell-1} \times n_\ell$	local linear map
$\boldsymbol{g}_{h_{\ell-1}}$	$n_{\ell-1} \times 1$	returns to the previous layer

$$\boldsymbol{g}_{h_{\ell-1}} = \boldsymbol{W}_\ell^\top \, \boldsymbol{g}_{z_\ell}.$$

each layer receives a gradient shaped like its output and returns one shaped like its input

Ten ordinary-looking slopes can create three regimes

V VISUAL

Start with an arriving loss derivative of 1 and multiply the same local slope ten times:

Ten ordinary-looking slopes can create three regimes

V VISUAL

Start with an arriving loss derivative of 1 and multiply the same local slope ten times:

Local slope	After 10 layers	Effect
0.5	$0.5^{10} \approx 0.001$	gradient nearly disappears
1.0	$1^{10} = 1$	gradient is preserved
1.5	$1.5^{10} \approx 57.7$	gradient grows rapidly

Ten ordinary-looking slopes can create three regimes

V VISUAL

Start with an arriving loss derivative of 1 and multiply the same local slope ten times:

Local slope	After 10 layers	Effect
0.5	$0.5^{10} \approx 0.001$	gradient nearly disappears
1.0	$1^{10} = 1$	gradient is preserved
1.5	$1.5^{10} \approx 57.7$	gradient grows rapidly

the chain rule explains both vanishing and exploding gradients

Sigmoid saturation

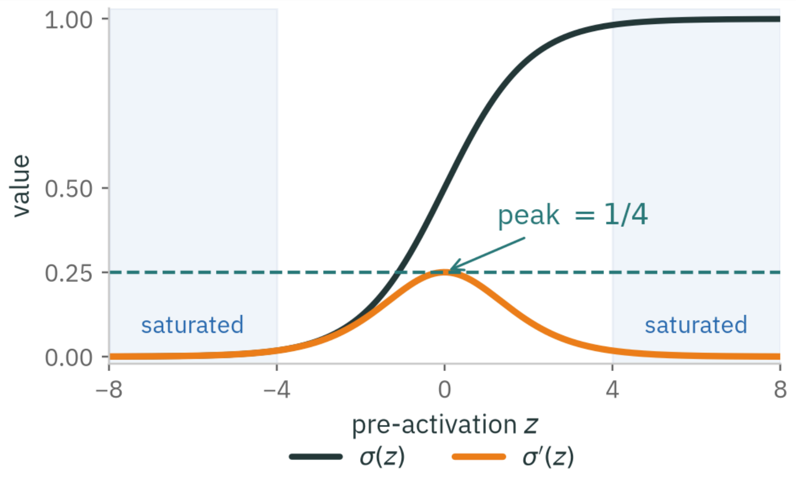
V VISUAL

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:

Sigmoid saturation

V VISUAL

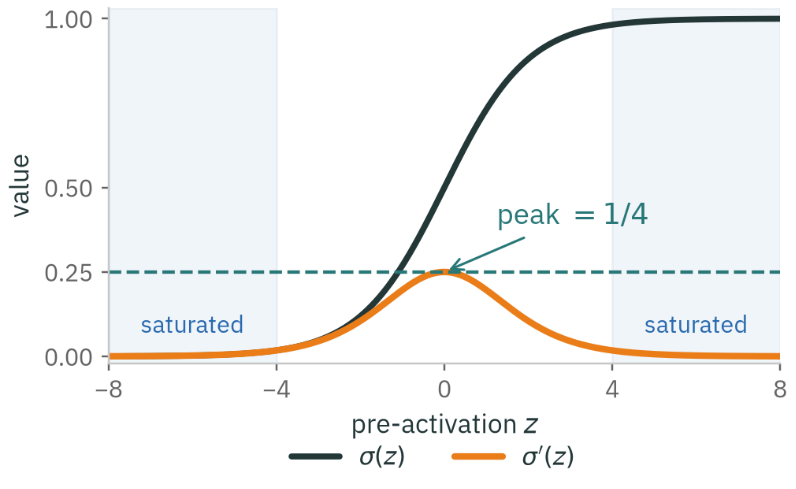
$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:



Sigmoid saturation

V VISUAL

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:

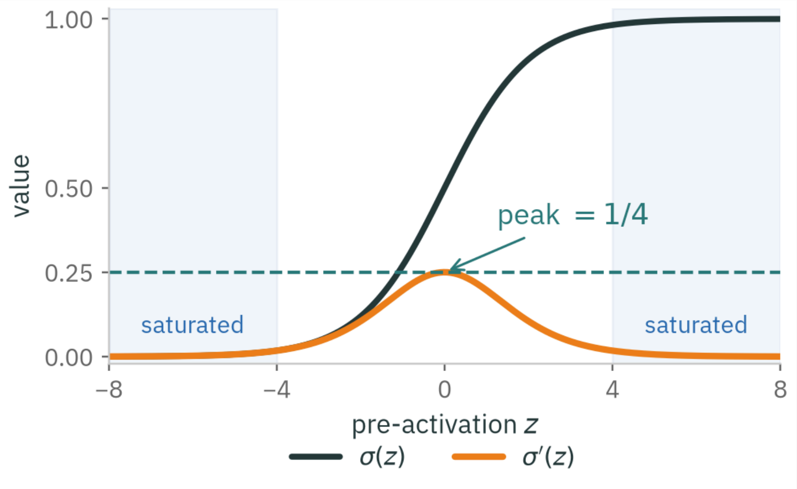


each sigmoid layer multiplies the gradient by at most $1/4$

Sigmoid saturation

V VISUAL

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:



each sigmoid layer multiplies the gradient by at most $1/4$

> Repeated sigmoid derivatives can shrink the signal quickly. ReLU has slope 1 on active units and 0 on inactive units; weights and normalization also shape the full Jacobian.

Interactive: gradient flow

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Interactive: gradient flow

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Sweep depth to see repeated small local slopes rapidly erase an early-layer gradient.

Summary

Backprop in one sentence

Seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$, then walk the graph **backward**, at each operation computing

downstream gradient $=$ **upstream gradient** $\times$ **local derivative**,

and **adding** gradients wherever a variable branched.

Backprop in one sentence

Seed $\partial\mathcal{L}/\partial\mathcal{L} = 1$, then walk the graph **backward**, at each operation computing

downstream gradient $=$ **upstream gradient** $\times$ **local derivative**,

and **adding** gradients wherever a variable branched.

For vector operations, the local derivative can be a vector or matrix; the same color-coded chain rule applies.

The formulas to keep

V VISUAL

Setting	Backward rule
scalar operation	$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v} \cdot \partial v / \partial u$
dense layer	$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}} \boldsymbol{x}^\top \; \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \boldsymbol{W}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$
activation	$\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}} \odot \varphi'(\boldsymbol{z})$
branch point	gradients **add**

We can compute $\partial\mathcal{L}/\partial\theta_j$ for every parameter.

Next: turn those gradients into reliable parameter updates.